AI Engineering Handbook

Part 6 — Advanced Computer Vision & CNN Architectures

Contents

Chapter 6.1 — The Computer Vision Pipeline **8**
6.1.1 How Images Are Represented . . . 9
6.1.2 Image Tensors and Batch Dimensions . . . 9
6.1.3 Normalization and Resizing . . . 10
6.1.4 Color-Space Conversion, Cropping, Padding . . . 10
6.1.5 Loading Images, Saving Images, Video Input . . . 11
Practice Tasks . . . 11

Chapter 6.2 — OpenCV Fundamentals **13**
6.2.1 Reading, Writing, Displaying . . . 13
6.2.2 Resizing, Cropping, Rotation, Flipping . . . 13
6.2.3 Color Conversion and Thresholding . . . 14
6.2.4 Blurring and Edge Detection . . . 14
6.2.5 Contours and Morphology . . . 15
6.2.6 Drawing and Text . . . 15
6.2.7 Common Mistakes and Debugging . . . 15
6.2.8 Mini-Project: Document Edge Detector . . . 15
Practice Tasks . . . 16

Chapter 6.3 — Image Preprocessing for Deep Learning **18**
6.3.1 Resize Strategies . . . 18
6.3.2 Normalization vs. Standardization . . . 18
6.3.3 Task-Specific Preprocessing . . . 19
6.3.4 Tensor and Batch Conversion . . . 20
Practice Tasks . . . 20

Chapter 6.4 — Image Augmentation **21**
6.4.1 Augmentation vs. Preprocessing . . . 21
6.4.2 Common Augmentations . . . 21
6.4.3 torchvision Implementation . . . 22
6.4.4 TensorFlow / Keras Implementation . . . 22
6.4.5 MixUp and CutMix From Scratch . . . 23
6.4.6 Training vs. Validation/Test: What Runs Where . . . 23
Practice Tasks . . . 24

Chapter 6.5 — CNN Architecture Deep Dive **25**
6.5.1 Convolution, Kernels, Filters, Feature Maps . . . 25
6.5.2 Stride, Padding, Dilation . . . 25
6.5.3 Pooling and Global Average Pooling . . . 26
6.5.4 Receptive Field . . . 26
6.5.5 Parameter Counting . . . 26
6.5.6 A Complete CNN From Scratch . . . 26

Practice Tasks . . . 27

Chapter 6.6 — CNN Architecture Families **29**
6.6.1 Overview . . . 29
6.6.2 When to Use What . . . 29
6.6.3 Loading Pretrained Models and Transfer Learning . . . 30
6.6.4 Inception Modules (GoogLeNet) . . . 30
6.6.5 DenseNet: Feature Reuse . . . 30
6.6.6 Comparison Table . . . 30
Practice Tasks . . . 31

Chapter 6.7 — ResNet & Skip Connections **32**
6.7.1 The Vanishing Gradient Problem . . . 32
6.7.2 Residual Connections . . . 32
6.7.3 ResNet Block Implementation . . . 32
6.7.4 Using Pretrained ResNet: Feature Extraction vs. Fine-Tuning . . . 33
Practice Tasks . . . 34

Chapter 6.8 — MobileNet & Efficient Models **35**
6.8.1 Depthwise Separable Convolution . . . 35
6.8.2 Inverted Residuals (MobileNetV2/V3) . . . 35
6.8.3 EfficientNet: Compound Scaling . . . 36
6.8.4 Model Size, Latency, and Mobile Inference . . . 36
6.8.5 Edge Deployment Considerations . . . 36
Practice Tasks . . . 37

Chapter 6.9 — Transfer Learning for Computer Vision **38**
6.9.1 The Standard Workflow . . . 38
6.9.2 Complete Workflow Implementation . . . 38
6.9.3 Discriminative Learning Rates . . . 39
6.9.4 Choosing Freeze Depth . . . 39
Practice Tasks . . . 40

Chapter 6.10 — Image Classification **41**
6.10.1 Classification Task Types . . . 41
6.10.2 Dataset and DataLoader . . . 41
6.10.3 Complete Training Loop . . . 42
6.10.4 Confusion Matrix and Inference . . . 43
6.10.5 Three Classifiers, in Increasing Production-Readiness . . . 43
Practice Tasks . . . 44

Chapter 6.11 — Classification Metrics **46**
6.11.1 The Core Metrics . . . 46
6.11.2 ROC-AUC and PR-AUC . . . 46
6.11.3 Metric Choice by Domain . . . 47
6.11.4 Confusion Matrix in Practice . . . 47
Practice Tasks . . . 48

Chapter 6.12 — Object Detection Fundamentals **49**
6.12.1 Bounding Box Representations . . . 49
6.12.2 Confidence, Objectness, and Class Labels . . . 49
6.12.3 Anchors vs. Anchor-Free Detection . . . 50
6.12.4 The Detection Pipeline . . . 50
Practice Tasks . . . 50

Chapter 6.13 — IoU & Non-Maximum Suppression **51**
6.13.1 Intersection over Union . . . 51
6.13.2 Non-Maximum Suppression From Scratch . . . 51
6.13.3 Confidence Threshold + IoU Threshold: Two Separate Knobs . . . 52
6.13.4 Class-Aware NMS and Soft-NMS . . . 52
6.13.5 Framework Implementations . . . 52
Practice Tasks . . . 53

Chapter 6.14 — YOLO Fundamentals **54**
6.14.1 One-Stage Detection . . . 54
6.14.2 The Grid Concept . . . 54
6.14.3 Practical Usage (Modern Python Workflow) . . . 55
6.14.4 Confidence and Class Prediction . . . 55
6.14.5 Validation . . . 55
Practice Tasks . . . 56

Chapter 6.15 — YOLO Training **57**
6.15.1 The Custom Training Workflow . . . 57
6.15.2 Dataset Directory Structure . . . 57
6.15.3 Dataset YAML Configuration . . . 58
6.15.4 Training . . . 58
6.15.5 Validation and Evaluation . . . 59
6.15.6 Inference and Threshold Tuning . . . 59
Practice Tasks . . . 60

Chapter 6.16 — YOLO Custom Dataset Project **61**
Project Specification . . . 61
Dataset Specification . . . 61
Environment Specification . . . 62
Environment Check . . . 63
Requirements . . . 64
Folder Structure . . . 64
Dataset Download . . . 65
Annotation Format . . . 65
Dataset Validation . . . 65
Training . . . 66
Evaluation . . . 66
Inference . . . 67
Error Analysis . . . 67
Model Export . . . 67
Deployment Options . . . 67
Improvement Tasks . . . 68
Interview Questions . . . 68

Chapter 6.17 — YOLO Deployment & Optimization **69**
6.17.1 ONNX Export . . . 69
6.17.2 Inference Optimization Levers . . . 69
6.17.3 Image-Size Tradeoffs . . . 70
6.17.4 GPU vs. CPU Inference . . . 70
6.17.5 Deployment Targets in Practice . . . 70
Practice Tasks . . . 70

Chapter 6.18 — SSD (Single Shot Detector) **72**
6.18.1 Architecture . . . 72
6.18.2 Anchor Boxes and Multiscale Prediction . . . 72

6.18.3 torchvision Implementation . . . 72
6.18.4 Strengths and Limitations . . . 73
6.18.5 SSD vs. YOLO . . . 73
Practice Tasks . . . 74

Chapter 6.19 — Faster R-CNN **75**
6.19.1 Two-Stage Detection . . . 75
6.19.2 Region Proposal Network (RPN) . . . 75
6.19.3 ROI Pooling / ROI Align . . . 75
6.19.4 Why Faster R-CNN Can Be More Accurate but Slower . . . 76
6.19.5 torchvision Implementation . . . 76
Practice Tasks . . . 77

Chapter 6.20 — RetinaNet **78**
6.20.1 The Class Imbalance Problem in One-Stage Detectors . . . 78
6.20.2 Focal Loss . . . 78
6.20.3 Feature Pyramid Networks (FPN) . . . 78
6.20.4 Classification and Regression Subnets . . . 79
6.20.5 Implementation . . . 79
6.20.6 Comparison: RetinaNet vs. YOLO vs. SSD vs. Faster R-CNN . . . 79
Practice Tasks . . . 80

Chapter 6.21 — Object Detector Comparison **81**
6.21.1 Comparison Table . . . 81
6.21.2 Which Detector Should I Choose? . . . 81
6.21.3 Decision Checklist . . . 82
Practice Tasks . . . 82

Chapter 6.22 — Semantic Segmentation **84**
6.22.1 Pixel-Level Classification . . . 84
6.22.2 Mask Encoding . . . 84
6.22.3 Architecture Families . . . 84
6.22.4 Complete U-Net Implementation . . . 85
Practice Tasks . . . 86

Chapter 6.23 — U-Net Deep Dive **87**
6.23.1 Encoder, Decoder, and Skip Connections . . . 87
6.23.2 Upsampling: Transposed Convolution vs. Interpolation . . . 87
6.23.3 Dataset Preparation and Mask Preprocessing . . . 87
6.23.4 Dice Loss and Training . . . 88
6.23.5 IoU for Segmentation . . . 88
6.23.6 Inference and Visualization . . . 89
Practice Tasks . . . 89

Chapter 6.24 — Instance Segmentation **90**
6.24.1 Semantic vs. Instance Segmentation . . . 90
6.24.2 Mask R-CNN: Faster R-CNN + a Mask Branch . . . 90
6.24.3 torchvision Implementation . . . 91
6.24.4 Visualizing Instance Masks . . . 91
Practice Tasks . . . 92

Chapter 6.25 — Segmentation Metrics **93**
6.25.1 Core Metrics . . . 93
6.25.2 Which Metric to Use When . . . 94
Practice Tasks . . . 94

Chapter 6.26 — OCR Pipeline **95**
6.26.1 The OCR Workflow . . . 95
6.26.2 OpenCV Preprocessing for OCR . . . 95
6.26.3 pytesseract: Basic Usage . . . 96
6.26.4 Handling Noisy Document Images . . . 96
Practice Tasks . . . 97

Chapter 6.27 — Tesseract OCR **98**
6.27.1 Installation and Setup . . . 98
6.27.2 Language Packs . . . 98
6.27.3 Page Segmentation Modes (PSM) . . . 98
6.27.4 OCR Engine Modes (OEM) . . . 99
6.27.5 Extracting Bounding Boxes and Confidence . . . 99
6.27.6 Preprocessing Before Tesseract . . . 99
6.27.7 Common Errors and Debugging . . . 99
Practice Tasks . . . 100

Chapter 6.28 — Computer Vision Datasets **101**
6.28.1 Classification Datasets . . . 101
6.28.2 Detection Datasets . . . 101
6.28.3 Segmentation Datasets . . . 101
6.28.4 OCR Datasets . . . 102
6.28.5 Recommended Projects by Dataset . . . 102
Practice Tasks . . . 103

Chapter 6.29 — Computer Vision Data Quality **104**
6.29.1 Corrupted and Duplicate Images . . . 104
6.29.2 Mislabeled Images and Noisy Labels . . . 105
6.29.3 Class Imbalance . . . 105
6.29.4 Inconsistent Resolution and Domain Shift . . . 106
Practice Tasks . . . 106

Chapter 6.30 — Computer Vision Data Leakage **107**
6.30.1 Duplicate Leakage . . . 107
6.30.2 Patient-Level Leakage (Medical/Person-Level Imaging) . . . 107
6.30.3 Video-Frame Leakage . . . 108
6.30.4 Augmentation Leakage . . . 108
6.30.5 Preprocessing Leakage . . . 108
6.30.6 Temporal Leakage . . . 108
6.30.7 Train/Test Contamination Summary . . . 108
Practice Tasks . . . 109

Chapter 6.31 — Computer Vision Debugging **110**
6.31.1 Model Predicts One Class for Everything . . . 110
6.31.2 Loss Doesn't Decrease . . . 110
6.31.3 NaN Loss . . . 110
6.31.4 CUDA Out-of-Memory . . . 111
6.31.5 Incorrect Image Channels / Tensor Shape . . . 111
6.31.6 Wrong Normalization . . . 111
6.31.7 Bounding Boxes Shifted / Masks Misaligned . . . 111
6.31.8 OCR Produces Garbage . . . 111
6.31.9 Detection Confidence Extremely Low . . . 112
6.31.10 Model Works on Training Data but Fails in Deployment . . . 112
Practice Tasks . . . 112

Chapter 6.32 — Computer Vision Performance Optimization **113**
6.32.1 Batch Size and Resolution . . . 113
6.32.2 DataLoader Workers and Pin Memory . . . 113
6.32.3 Mixed Precision Training . . . 113
6.32.4 Gradient Accumulation . . . 114
6.32.5 Caching and Augmentation Bottlenecks . . . 114
6.32.6 GPU Utilization Monitoring . . . 114
6.32.7 Model Pruning and Quantization (Concepts) . . . 115
6.32.8 ONNX and Inference Batching . . . 115
Practice Tasks . . . 115

Chapter 6.33 — Computer Vision Deployment **116**
6.33.1 The Deployment Pipeline . . . 116
6.33.2 Python Inference Script . . . 116
6.33.3 FastAPI Inference Endpoint . . . 117
6.33.4 Docker . . . 118
6.33.5 ONNX Runtime and TFLite/LiteRT Serving . . . 118
6.33.6 Edge and Mobile Deployment (Concepts) . . . 118
Practice Tasks . . . 119

Chapter 6.34 — Video Computer Vision **120**
6.34.1 Video Fundamentals . . . 120
6.34.2 Frame Skipping . . . 120
6.34.3 Real-Time Webcam Object Detection . . . 121
6.34.4 Tracking Preview . . . 121
6.34.5 Temporal Considerations . . . 122
6.34.6 Performance Bottlenecks . . . 122
Practice Tasks . . . 122

Chapter 6.35 — Computer Vision Projects **124**
Project 1 — Image Classification From Scratch (CIFAR-10) . . . 124
Project 2 — Transfer-Learning Image Classifier (Custom Categories) . . . 125
Project 4 — Real-Time Webcam Detection with Counting . . . 127
Project 5 — U-Net Medical/Object Segmentation . . . 128
Project 7 — Production-Style CV API . . . 130
Project 8 — Portfolio-Grade End-to-End CV System: Retail Shelf Analyzer . . . 132
Final Project Index . . . 133

Chapter 6.36 — Computer Vision Model Selection **135**
6.36.1 Decision Tree . . . 135
6.36.2 Choosing a Backbone (Classification and as a Detector/Segmenter Base) . . . 135
6.36.3 Choosing a Detector . . . 136
6.36.4 Choosing a Segmentation Approach . . . 136
6.36.5 A Practical Selection Checklist . . . 136
Practice Tasks . . . 136

Chapter 6.37 — Computer Vision Interview Preparation **138**
CNN Fundamentals . . . 138
Architectures & Transfer Learning . . . 139
Detection . . . 139
OCR . . . 141
OpenCV . . . 141
Deployment & GPU Optimization . . . 141
Debugging . . . 142
Data & Leakage . . . 143

System Design / Architecture Selection 143

Chapter 6.38 — Computer Vision Cheat Sheets **145**
One-Page CV Roadmap 148
Model-Selection Decision Tree 149
Environment Checklist 149
Project Checklist 150
Interview Question Bank 150

Chapter 6.1 — The Computer Vision Pipeline

Every computer vision system, from a hobby webcam filter to a production defect-detection line, moves data through the same sequence of stages. Understanding this pipeline before touching any framework saves you from the single most common beginner failure mode: writing model code before understanding what shape and type of data actually flows into it.

```
Image/Video Input
      |
      v
Loading
      |
      v
Inspection
      |
      v
Preprocessing
      |
      v
Augmentation (training only)
      |
      v
Feature Learning (the model)
      |
      v
Prediction
      |
      v
Post-processing
      |
      v
Evaluation
      |
      v
Deployment
```

6.1.1 How Images Are Represented

A digital image is a tensor. That single sentence explains most of the bugs you will spend your career debugging.

A grayscale image is a 2D array of shape `(H, W)`, where each value is a pixel intensity, typically `uint8` in the range `[0, 255]`. A color image adds a channel dimension: `(H, W, C)` where `C = 3` for RGB or BGR. Deep learning frameworks almost always want channels-first tensors, `(C, H, W)`, and a batch dimension on top of that, `(N, C, H, W)`.

Concept	Meaning
Pixel	A single sample of color/intensity at one spatial location
Resolution	Width x Height in pixels, e.g. `1920x1080`
Channel	One color plane — R, G, B (or a single grayscale plane)
Bit depth	Bits per channel per pixel — 8-bit (`0-255`) is standard for natural images
Tensor	The N-dimensional array representation the framework operates on
Batch dimension	The leading `N` axis that stacks multiple images for parallel processing

- **Common Mistake**

OpenCV loads and stores images as BGR, not RGB. Every other library you will touch — PIL, matplotlib, PyTorch, TensorFlow — expects RGB. If you load with `cv2.imread` and feed the array straight into a PyTorch model or display it with `matplotlib`, colors will look subtly, confusingly wrong (skin tones turn blue, skies turn orange). Always convert: `cv2.cvtColor(img, cv2.COLOR_BGR2RGB)`.

6.1.2 Image Tensors and Batch Dimensions

```
import numpy as np
import cv2

img = cv2.imread("photo.jpg")          # shape: (H, W, 3), dtype: uint8, order: BGR
print(img.shape, img.dtype)            # e.g. (480, 640, 3) uint8

# Convert to RGB and to a float tensor in CHW order, as PyTorch expects
rgb = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)          # (H, W, 3)
chw = np.transpose(rgb, (2, 0, 1)).astype(np.float32) / 255.0  # (3, H, W), [0,1]
batch = np.expand_dims(chw, axis=0)                 # (1, 3, H, W)
print(batch.shape, batch.dtype)
```

Keras/TensorFlow, by contrast, defaults to channels-last: `(N, H, W, C)`. Mixing the two conventions is one of the most common sources of silent shape bugs — the array often "works" (no crash) but produces garbage, because a `(3, 224, 224)` tensor accidentally gets interpreted as an image 3 pixels tall.

- **Best Practice**

When you write a preprocessing function, put the expected input and output shape in a comment or docstring immediately, e.g. `"in: (H, W, 3) uint8 BGR -> out: (3, 224, 224) float32 RGB normalized"`. This one habit eliminates the majority of channel-order and layout bugs before they happen.

6.1.3 Normalization and Resizing

Raw pixel values in `[0, 255]` are almost never fed directly into a network. Two common schemes:

- **Scale to `[0, 1]`**: divide by 255. Simple, used by many from-scratch CNNs.
- **Standardize with ImageNet statistics**: subtract the per-channel mean and divide by the per-channel standard deviation used to pretrain the backbone you're relying on. Required whenever you use a pretrained ImageNet model (ResNet, MobileNet, EfficientNet, etc.):

```
IMAGENET_MEAN = [0.485, 0.456, 0.406]
IMAGENET_STD = [0.229, 0.224, 0.225]
# x is float32 in [0, 1], shape (3, H, W)
normalized = (x - torch.tensor(IMAGENET_MEAN).view(3, 1, 1)) / torch.tensor(IMAGENET_STD).view(3, 1, 1)
```

Resizing has real consequences beyond "making the image smaller." `cv2.resize` and `torchvision.transforms.Resize` use different default interpolation methods (`INTER_LINEAR` vs. bilinear resampling — usually consistent, but antialiasing defaults differ), and resizing without preserving aspect ratio distorts object proportions, which matters for detection and segmentation tasks far more than for classification.

```
import cv2

img = cv2.imread("photo.jpg")
resized = cv2.resize(img, (224, 224), interpolation=cv2.INTER_LINEAR)  # (224, 224, 3)

# Aspect-ratio preserving resize + pad (common for detection pipelines)
def resize_with_padding(img, target_size=640, pad_value=114):
    h, w = img.shape[:2]
    scale = target_size / max(h, w)
    new_h, new_w = int(h * scale), int(w * scale)
    resized = cv2.resize(img, (new_w, new_h))
    canvas = np.full((target_size, target_size, 3), pad_value, dtype=np.uint8)
    canvas[:new_h, :new_w] = resized
    return canvas, scale  # keep `scale` to map predictions back to original coordinates
```

• Common Mistake

Resizing to a square without tracking the scale factor and padding offset, then forgetting to map bounding-box or mask coordinates back to the original image size at inference time. This produces boxes and masks that are visibly wrong in exactly the way that's hardest to spot in a quick eyeball check — they look "close" but are systematically off.

6.1.4 Color-Space Conversion, Cropping, Padding

```
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)        # (H, W) — single channel
hsv = cv2.cvtColor(img, cv2.COLOR_BGR2HSV)          # useful for color-based thresholding

# Center crop
def center_crop(img, crop_h, crop_w):
    h, w = img.shape[:2]
    top = (h - crop_h) // 2
    left = (w - crop_w) // 2
    return img[top:top + crop_h, left:left + crop_w]
```

```python
# Padding (e.g. to make an image square before resizing)
padded = cv2.copyMakeBorder(img, 10, 10, 10, 10, cv2.BORDER_CONSTANT, value=(0, 0, 0))
```

6.1.5 Loading Images, Saving Images, Video Input

```python
# Single image
img = cv2.imread("input.jpg")              # None if file missing/unreadable — always check
cv2.imwrite("output.png", img)

# Video / webcam
cap = cv2.VideoCapture(0)                  # 0 = default webcam; or pass a file path
while cap.isOpened():
    ret, frame = cap.read()
    if not ret:
        break
    cv2.imshow("frame", frame)
    if cv2.waitKey(1) & 0xFF == ord("q"):
        break
cap.release()
cv2.destroyAllWindows()
```

• Best Practice

`cv2.imread` returns `None` on failure instead of raising an exception — a bad path, a corrupted file, or an unsupported format all fail silently this way. Always check `if img is None: raise FileNotFoundError(path)` immediately after loading, especially inside a `Dataset.__getitem__`, where a silent `None` will crash many steps later with a confusing stack trace.

★ Quick Revision Sheet

CV Pipeline, in one screen:

- Images are tensors: `(H, W, C)` in OpenCV/NumPy, `(N, C, H, W)` in PyTorch, `(N, H, W, C)` in TensorFlow/Keras.
- OpenCV loads BGR. Convert to RGB before anything else touches the array.
- Normalize with ImageNet mean/std whenever using an ImageNet-pretrained backbone; otherwise scale to `[0, 1]`.
- Track scale and padding offsets through resizing so you can map predictions back to original image coordinates.
- Always check for `None` after `cv2.imread` — it fails silently.

Practice Tasks

- **Level 1 (Understand):** Load an image with OpenCV, print its shape and dtype, then convert it to RGB and display it correctly with matplotlib.
- **Level 2 (Modify):** Change `resize_with_padding` to accept a non-square `target_size` (target_h, target_w).
- **Level 3 (Implement):** Write a function that loads every image in a folder, reports any that fail to load, and returns a summary of resolutions found.
- **Level 4 (Debug):** Given a pipeline where colors look inverted after a model prediction is drawn back onto the original image, find and fix the BGR/RGB bug.

- **Level 5 (Optimize):** Batch-load and resize 10,000 images using `cv2.imread` in a loop vs. using multiple worker processes; measure the wall-clock difference.
- **Level 6 (Deploy):** Wrap the loading + preprocessing steps into a single function with full input validation, ready to be called from an inference API.

Chapter 6.2 — OpenCV Fundamentals

OpenCV is the workhorse library for image I/O, classical preprocessing, and anything that happens before a tensor reaches a neural network. This chapter is a practical reference, not an exhaustive API dump — it covers the functions you will actually call, weekly, in real projects.

6.2.1 Reading, Writing, Displaying

```
import cv2

img = cv2.imread("input.jpg", cv2.IMREAD_COLOR)   # BGR, uint8
if img is None:
    raise FileNotFoundError("input.jpg")

cv2.imwrite("output.jpg", img, [cv2.IMWRITE_JPEG_QUALITY, 95])

cv2.imshow("window", img)   # only works with a display attached (not headless servers/CI)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

Parameter	Function	Meaning	Default
`flags`	`cv2.imread`	`IMREAD_COLOR`, `IMREAD_GRAYSCALE`, `IMREAD_UNCHANGED` (keeps alpha)	`IMREAD_COLOR`
`params`	`cv2.imwrite`	Encoder-specific quality/compression flags	format default

• Common Mistake

Calling `cv2.imshow` on a headless server (a training VM, a Docker container, a CI pipeline) raises an obscure error about a missing display, or silently does nothing depending on the build. Use `cv2.imwrite` to a file for inspection instead, or display via matplotlib after a BGR->RGB conversion.

6.2.2 Resizing, Cropping, Rotation, Flipping

```
resized = cv2.resize(img, (300, 300), interpolation=cv2.INTER_AREA)   # INTER_AREA for shrinking
cropped = img[50:200, 100:300]                                        # NumPy slicing: [y1:y2, x1:x2]
```

```
flipped_h = cv2.flip(img, 1)     # 1 = horizontal, 0 = vertical, -1 = both

(h, w) = img.shape[:2]
center = (w // 2, h // 2)
M = cv2.getRotationMatrix2D(center, angle=15, scale=1.0)
rotated = cv2.warpAffine(img, M, (w, h))
```

6.2.3 Color Conversion and Thresholding

```
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
rgb = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
hsv = cv2.cvtColor(img, cv2.COLOR_BGR2HSV)

# Global threshold
_, binary = cv2.threshold(gray, 127, 255, cv2.THRESH_BINARY)

# Otsu's method — automatically picks the threshold value
_, otsu = cv2.threshold(gray, 0, 255, cv2.THRESH_BINARY + cv2.THRESH_OTSU)

# Adaptive threshold — handles uneven lighting across the image
adaptive = cv2.adaptiveThreshold(
    gray, 255, cv2.ADAPTIVE_THRESH_GAUSSIAN_C, cv2.THRESH_BINARY, 11, 2
)
```

Parameter	Meaning	Default
`blockSize` (adaptive)	Neighborhood size used to compute the local threshold	must be odd, e.g. `11`
`C` (adaptive)	Constant subtracted from the computed mean	`2`

6.2.4 Blurring and Edge Detection

```
gaussian = cv2.GaussianBlur(img, (5, 5), sigmaX=0)     # kernel must have odd dimensions
median = cv2.medianBlur(img, 5)                        # excellent for salt-and-pepper noise

edges = cv2.Canny(gray, threshold1=100, threshold2=200)
```

Parameter	Meaning
`threshold1` (Canny)	Lower hysteresis threshold — edges below this are discarded
`threshold2` (Canny)	Upper hysteresis threshold — edges above this are kept as strong edges

• Best Practice

Set Canny's upper threshold to roughly 2-3x the lower threshold as a starting point (e.g. `100, 200`), then tune on your actual data. Blur the image with a Gaussian kernel first — Canny is very sensitive to sensor noise, and skipping the blur step produces a haze of spurious short edges.

6.2.5 Contours and Morphology

```
contours, hierarchy = cv2.findContours(binary, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)
cv2.drawContours(img, contours, -1, (0, 255, 0), 2)

largest = max(contours, key=cv2.contourArea)
x, y, w, h = cv2.boundingRect(largest)

kernel = np.ones((5, 5), np.uint8)
eroded = cv2.erode(binary, kernel, iterations=1)
dilated = cv2.dilate(binary, kernel, iterations=1)
opened = cv2.morphologyEx(binary, cv2.MORPH_OPEN, kernel)    # erode then dilate — removes small noise
closed = cv2.morphologyEx(binary, cv2.MORPH_CLOSE, kernel)   # dilate then erode — fills small holes
```

6.2.6 Drawing and Text

```
cv2.rectangle(img, (50, 50), (200, 200), (0, 255, 0), thickness=2)
cv2.circle(img, (100, 100), radius=30, color=(255, 0, 0), thickness=-1)  # -1 = filled
cv2.line(img, (0, 0), (300, 300), (0, 0, 255), thickness=2)
cv2.putText(img, "Detected", (50, 40), cv2.FONT_HERSHEY_SIMPLEX, 0.8, (0, 255, 0), 2)
```

6.2.7 Common Mistakes and Debugging

• Common Mistake

NumPy slicing for cropping is `[y1:y2, x1:x2]` — row (height) first, column (width) second — the opposite order of most `(x, y)` coordinate conventions used elsewhere in OpenCV (e.g. `cv2.rectangle` takes `(x, y)` points). Mixing these up produces crops or boxes that are transposed relative to what you intended, and on non-square images the bug is immediately visible; on roughly-square images it can slip through review.

• Common Mistake

Kernel sizes for blur, erode, and dilate operations must be odd numbers (`(3,3)`, `(5,5)`, ...). An even kernel size raises an assertion error that is easy to trigger when a kernel size is computed dynamically (e.g. as a fraction of image size) without rounding to the nearest odd number.

6.2.8 Mini-Project: Document Edge Detector

A complete, runnable mini-project combining the operations above: detect the outline of a document (e.g. a receipt or page) photographed on a contrasting background.

```python
import cv2
import numpy as np

def find_document_contour(image_path):
    img = cv2.imread(image_path)
    if img is None:
        raise FileNotFoundError(image_path)

    gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
    blurred = cv2.GaussianBlur(gray, (5, 5), 0)
    edges = cv2.Canny(blurred, 75, 200)

    # Close small gaps in the edge map so the document outline forms one contour
    kernel = np.ones((5, 5), np.uint8)
    closed = cv2.morphologyEx(edges, cv2.MORPH_CLOSE, kernel)

    contours, _ = cv2.findContours(closed, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)
    if not contours:
        return None, img

    largest = max(contours, key=cv2.contourArea)
    peri = cv2.arcLength(largest, True)
    approx = cv2.approxPolyDP(largest, 0.02 * peri, True)  # simplify to a polygon

    output = img.copy()
    cv2.drawContours(output, [approx], -1, (0, 255, 0), 3)
    return approx, output

if __name__ == "__main__":
    contour, annotated = find_document_contour("receipt.jpg")
    cv2.imwrite("annotated_receipt.jpg", annotated)
    print(f"Found {0 if contour is None else len(contour)}-point contour")
```

★ Quick Revision Sheet

OpenCV Fundamentals, in one screen:

- Load: `cv2.imread` (returns `None` on failure, always check). Save: `cv2.imwrite`.
- Slicing is `[y1:y2, x1:x2]`; drawing functions take `(x, y)` points — don't confuse the two conventions.
- Blur before edge detection. `Canny(low, high)` with high ≈ 2-3x low as a starting point.
- Morphology: `OPEN` removes small noise, `CLOSE` fills small holes.
- Kernel sizes for blur/erode/dilate must be odd.
- `cv2.imshow` needs a real display — use `imwrite` on headless machines.

Practice Tasks

- **Level 1:** Load an image, convert to grayscale, and save the result.
- **Level 2:** Modify the document-contour project to work on a folder of images instead of one file.
- **Level 3:** Implement a webcam-based edge-detection viewer using `cv2.VideoCapture`.
- **Level 4:** Debug a script where `findContours` returns an empty list — inspect the intermediate `edges` and `closed`

images to find the cause.

- **Level 5:** Compare the runtime of `GaussianBlur` vs. `medianBlur` on a batch of 1,000 images.
- **Level 6:** Wrap the document-detector into a function with error handling suitable for a REST endpoint.

Chapter 6.3 — Image Preprocessing for Deep Learning

Preprocessing converts a raw image into exactly the tensor shape, value range, and statistical distribution a model was designed (or pretrained) to expect. Get this wrong and a model will still run — it just won't learn, or a pretrained backbone will perform far below its published accuracy, with no error message telling you why.

6.3.1 Resize Strategies

Technique	When to use
Direct resize to fixed size	Classification, when aspect-ratio distortion is acceptable
Resize + center crop	Classification with pretrained ImageNet models (standard `Resize(256) -> CenterCrop(224)`)
Resize + pad (letterbox)	Detection — preserves aspect ratio and object proportions
Random crop	Training-time augmentation for classification
Multi-scale resize	Detection/segmentation training to improve scale robustness

```
import torchvision.transforms as T

# Standard ImageNet-pretrained classifier preprocessing
val_transform = T.Compose([
    T.Resize(256),
    T.CenterCrop(224),
    T.ToTensor(),                                    # (H,W,C) uint8 [0,255] -> (C,H,W) float32 [0,1]
    T.Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225]),
])
```

6.3.2 Normalization vs. Standardization

- **Normalization** rescales values into a fixed range, typically `[0, 1]`: `x / 255.0`.
- **Standardization** shifts and scales using a mean and standard deviation so the resulting distribution has zero mean and unit variance per channel: `(x - mean) / std`.

Use ImageNet mean/std whenever fine-tuning or running inference on any ImageNet-pretrained backbone (ResNet, VGG, EfficientNet, MobileNet, etc.) — the pretrained weights were learned against that specific input distribution, and

skipping this step silently degrades accuracy without raising any error. If training entirely from scratch on your own dataset, compute the dataset's own per-channel mean/std instead.

```
import numpy as np

def compute_dataset_stats(image_paths):
    """Compute per-channel mean/std across a dataset — run once, cache the result."""
    pixel_sum = np.zeros(3)
    pixel_sq_sum = np.zeros(3)
    n_pixels = 0
    for path in image_paths:
        img = cv2.imread(path)
        img = cv2.cvtColor(img, cv2.COLOR_BGR2RGB).astype(np.float64) / 255.0
        pixel_sum += img.sum(axis=(0, 1))
        pixel_sq_sum += (img ** 2).sum(axis=(0, 1))
        n_pixels += img.shape[0] * img.shape[1]
    mean = pixel_sum / n_pixels
    std = np.sqrt(pixel_sq_sum / n_pixels - mean ** 2)
    return mean, std
```

- **Common Mistake**

Normalizing twice — once inside a custom `Dataset` and again inside a library's built-in transform pipeline — silently produces near-zero-magnitude inputs that a network technically accepts but cannot learn from effectively. If accuracy is mysteriously terrible from epoch one despite a correct architecture, print the min/max/mean of a batch right before it enters the model.

6.3.3 Task-Specific Preprocessing

Task	Key preprocessing consideration
Classification	Fixed-size resize/crop is fine; aspect distortion rarely matters
Detection	Must preserve aspect ratio (letterbox) and track the resize/pad transform to map boxes back
Segmentation	Any spatial transform (resize, crop, flip) applied to the image must be applied identically to the mask, using nearest-neighbor interpolation for the mask so label values stay integers
OCR	Preserve text sharpness — avoid aggressive blurring or lossy resizing; binarization and deskewing often help more than resizing

- **Common Mistake**

Applying bilinear interpolation to a segmentation mask when resizing. Bilinear interpolation blends pixel values, which turns integer class labels (`0, 1, 2, ...`) into meaningless fractional values at object boundaries. Always resize masks with `cv2.INTER_NEAREST` or `torchvision`'s nearest-mode resize.

6.3.4 Tensor and Batch Conversion

```
import torch

def preprocess_for_inference(img_bgr, size=224):
    img_rgb = cv2.cvtColor(img_bgr, cv2.COLOR_BGR2RGB)
    resized = cv2.resize(img_rgb, (size, size))
    tensor = torch.from_numpy(resized).permute(2, 0, 1).float() / 255.0   # (3, H, W)
    mean = torch.tensor([0.485, 0.456, 0.406]).view(3, 1, 1)
    std = torch.tensor([0.229, 0.224, 0.225]).view(3, 1, 1)
    tensor = (tensor - mean) / std
    return tensor.unsqueeze(0)   # (1, 3, H, W) — add batch dimension
```

★ Quick Revision Sheet

Preprocessing, in one screen:

- Match preprocessing to what the backbone was pretrained with — ImageNet mean/std for ImageNet backbones.
- Detection/segmentation: preserve aspect ratio; track resize scale and pad offset to invert at inference.
- Segmentation masks: always resize with nearest-neighbor, never bilinear.
- OCR: prioritize text sharpness over uniform resizing; consider deskew and binarization instead.
- Print min/max/mean of a batch when accuracy looks wrong from epoch one — it is almost always a preprocessing bug.

Practice Tasks

- **Level 1:** Explain why `T.Resize(256) -> T.CenterCrop(224)` is used instead of `T.Resize((224, 224))` for standard ImageNet evaluation.
- **Level 2:** Modify `preprocess_for_inference` to accept a target aspect-preserving letterbox size instead of a plain square resize.
- **Level 3:** Implement `compute_dataset_stats` and run it on a sample dataset to produce custom normalization values.
- **Level 4:** Given a segmentation model producing blurry, non-integer mask boundaries after resizing, find and fix the interpolation bug.
- **Level 5:** Profile the preprocessing step of a `Dataset.__getitem__` to see whether it is a training bottleneck.
- **Level 6:** Package `preprocess_for_inference` so its output is guaranteed byte-identical to the training-time preprocessing (a common source of train/serve skew).

Chapter 6.4 — Image Augmentation

Augmentation artificially expands the effective size and diversity of a training set by applying label-preserving transformations. It is a training-time-only technique — validation and test data must always see the same deterministic preprocessing the model will face in production, never randomized augmentation.

6.4.1 Augmentation vs. Preprocessing

	Preprocessing	Augmentation
Applied to	Train, validation, test, and production inference	Training only
Deterministic?	Yes	No — randomized per sample, per epoch
Purpose	Make input compatible with the model	Improve generalization, reduce overfitting
Example	Resize to 224x224, normalize	Random crop, random flip, color jitter

- **Common Mistake**

Applying random augmentation to the validation or test set. This makes validation metrics noisy and non-comparable across epochs, and it can mask overfitting — a model can appear to generalize because the ”test” images were also perturbed in ways that happen to be easy, not because it actually learned robust features.

6.4.2 Common Augmentations

Augmentation	Effect	Typical use
Horizontal flip	Mirrors left-right	Almost always safe for natural images; NOT safe for text/OCR or asymmetric objects (e.g. reading direction)
Vertical flip	Mirrors top-bottom	Safe for aerial/satellite imagery; usually unsafe for natural photos (upside-down objects are unrealistic)
Rotation	Small-angle rotation	Simulates camera tilt; keep angles modest (e.g. +/-15deg) for natural scenes

Augmentation	Effect	Typical use
Random crop	Crops a sub-region	Forces the model to not rely on objects always being centered
Color jitter	Random brightness/contrast/saturation shifts	Robustness to lighting and camera differences
Gaussian noise	Adds pixel-level noise	Robustness to sensor noise, compression artifacts
Cutout / Random Erasing	Masks out a random rectangular region	Forces reliance on multiple object parts, not just one distinctive patch
MixUp	Blends two images and their labels linearly	Regularization, improves calibration
CutMix	Pastes a patch from one image onto another, mixes labels proportionally	Regularization; often outperforms MixUp on classification

6.4.3 torchvision Implementation

```
import torchvision.transforms as T

train_transform = T.Compose([
    T.RandomResizedCrop(224, scale=(0.7, 1.0)),
    T.RandomHorizontalFlip(p=0.5),
    T.ColorJitter(brightness=0.2, contrast=0.2, saturation=0.2),
    T.RandomRotation(degrees=10),
    T.ToTensor(),
    T.Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225]),
    T.RandomErasing(p=0.25),   # applied after ToTensor — operates on the tensor
])

val_transform = T.Compose([
    T.Resize(256),
    T.CenterCrop(224),
    T.ToTensor(),
    T.Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225]),
])
```

6.4.4 TensorFlow / Keras Implementation

```
import tensorflow as tf

data_augmentation = tf.keras.Sequential([
    tf.keras.layers.RandomFlip("horizontal"),
    tf.keras.layers.RandomRotation(0.05),
    tf.keras.layers.RandomZoom(0.1),
    tf.keras.layers.RandomContrast(0.2),
])

# Applied as part of the model graph — active only during training (model.fit),
```

```
# automatically disabled during model.evaluate/predict.
inputs = tf.keras.Input(shape=(224, 224, 3))
x = data_augmentation(inputs)
x = tf.keras.applications.efficientnet.preprocess_input(x)
```

6.4.5 MixUp and CutMix From Scratch

```
import numpy as np
import torch

def mixup(x, y, alpha=0.2):
    lam = np.random.beta(alpha, alpha)
    idx = torch.randperm(x.size(0))
    mixed_x = lam * x + (1 - lam) * x[idx]
    return mixed_x, y, y[idx], lam
    # loss = lam * criterion(pred, y) + (1 - lam) * criterion(pred, y[idx])

def cutmix(x, y, alpha=1.0):
    lam = np.random.beta(alpha, alpha)
    idx = torch.randperm(x.size(0))
    H, W = x.shape[2], x.shape[3]
    cut_ratio = np.sqrt(1.0 - lam)
    cut_h, cut_w = int(H * cut_ratio), int(W * cut_ratio)
    cy, cx = np.random.randint(H), np.random.randint(W)
    y1, y2 = np.clip(cy - cut_h // 2, 0, H), np.clip(cy + cut_h // 2, 0, H)
    x1, x2 = np.clip(cx - cut_w // 2, 0, W), np.clip(cx + cut_w // 2, 0, W)
    x[:, :, y1:y2, x1:x2] = x[idx, :, y1:y2, x1:x2]
    lam_adjusted = 1 - ((x2 - x1) * (y2 - y1) / (H * W))
    return x, y, y[idx], lam_adjusted
```

6.4.6 Training vs. Validation/Test: What Runs Where

```
Training set    -> Preprocessing + Augmentation (randomized every epoch)
Validation set  -> Preprocessing only (deterministic)
Test set        -> Preprocessing only (deterministic)
Production input -> Preprocessing only, IDENTICAL to validation/test preprocessing
```

- **Best Practice**

Keep exactly one canonical ”eval preprocessing” function and reuse it for validation, test, and production inference. Never hand-write it three times in three places — that duplication is where train/serve skew silently creeps in.

• Interview Trap

"Why not just augment the validation set too, to get a more robust accuracy estimate?" Because validation accuracy needs to be a stable, comparable number across epochs and experiments so you can make model-selection decisions (early stopping, hyperparameter choice) — random noise from augmentation defeats that purpose. If you want to measure robustness, that's a separate, deliberately-designed robustness evaluation, not the epoch-to-epoch validation metric.

★ Quick Revision Sheet

Augmentation, in one screen:

- Augmentation is training-only and randomized; preprocessing is applied everywhere and deterministic.
- Flip/rotate/crop/color-jitter are the default safe set for natural images; check flip validity for text and asymmetric classes.
- MixUp/CutMix blend two samples and their labels — strong regularizers, add complexity to the loss computation.
- Use one shared preprocessing function across validation, test, and production to prevent train/serve skew.

Practice Tasks

- **Level 1:** List three augmentations that would be inappropriate for a digit-recognition (MNIST-style) task and explain why.
- **Level 2:** Add `RandAugment` to the torchvision training pipeline above.
- **Level 3:** Implement CutMix training for a small classifier and confirm the loss computation uses the mixed labels correctly.
- **Level 4:** Debug a pipeline where validation accuracy jumps around wildly between epochs — determine whether augmentation is leaking into the validation transform.
- **Level 5:** Measure the throughput cost of adding `ColorJitter` and `RandomErasing` to a DataLoader with 8 workers.
- **Level 6:** Ensure the deployed inference preprocessing function is unit-tested against the validation transform for byte-identical output on a fixed sample image.

Chapter 6.5 — CNN Architecture Deep Dive

Part 5 introduced convolutional networks conceptually. This chapter is the engineering deep dive: what each layer actually computes, what shape it produces, and how to build and reason about a CNN from first principles in both PyTorch and Keras.

6.5.1 Convolution, Kernels, Filters, Feature Maps

A convolutional layer slides a small learnable kernel (e.g. `3x3`) across the input, computing a weighted sum at each position. Each kernel produces one output channel, called a feature map. A layer with `out_channels=64` learns 64 independent `3x3` kernels (each spanning all input channels), producing 64 feature maps.

```
Input:  (C_in, H, W)
Kernel: (C_out, C_in, kH, kW)
Output: (C_out, H_out, W_out)
```

6.5.2 Stride, Padding, Dilation

Parameter	Effect
Stride	Step size the kernel moves each iteration. Stride 2 halves spatial resolution.
Padding	Zeros added around the border. `"same"` padding keeps output spatial size equal to input.
Dilation	Spacing inserted between kernel elements, expanding receptive field without adding parameters.

Output spatial size formula:

```
H_out = floor((H_in + 2*padding - dilation*(kernel_size-1) - 1) / stride) + 1
```

```
import torch.nn as nn

conv = nn.Conv2d(in_channels=3, out_channels=64, kernel_size=3, stride=1, padding=1)
# input (N, 3, 224, 224) -> output (N, 64, 224, 224)  — padding=1 preserves spatial size for a 3x3 kernel

conv_stride2 = nn.Conv2d(64, 128, kernel_size=3, stride=2, padding=1)
# input (N, 64, 224, 224) -> output (N, 128, 112, 112) — stride 2 halves H and W
```

6.5.3 Pooling and Global Average Pooling

```
maxpool = nn.MaxPool2d(kernel_size=2, stride=2)   # (N, C, H, W) -> (N, C, H/2, W/2)
gap = nn.AdaptiveAvgPool2d(1)                     # (N, C, H, W) -> (N, C, 1, 1) — any input size to fixed out
```

- **Best Practice**

Global average pooling before the final classifier (instead of flattening a large feature map into a huge fully-connected layer) drastically reduces parameter count and lets the same network architecture accept variable input resolutions. This is standard in every modern CNN backbone since GoogLeNet.

6.5.4 Receptive Field

The receptive field is the region of the original input that influences a single output unit. It grows with depth, kernel size, and stride/dilation. A stack of small `3x3` convolutions (as in VGG/ResNet) achieves a large receptive field with fewer parameters than one large kernel — two stacked `3x3` convs have the same receptive field as one `5x5` conv, but with fewer weights and an extra nonlinearity in between.

6.5.5 Parameter Counting

```
Conv2d params = (kernel_h * kernel_w * in_channels + 1) * out_channels   # +1 per filter for bias
```

```
conv = nn.Conv2d(64, 128, kernel_size=3)
# params = (3*3*64 + 1) * 128 = 73,856
print(sum(p.numel() for p in conv.parameters()))  # confirms: 73856
```

6.5.6 A Complete CNN From Scratch

PyTorch:

```
import torch
import torch.nn as nn

class SimpleCNN(nn.Module):
    def __init__(self, num_classes=10):
        super().__init__()
        self.features = nn.Sequential(
            nn.Conv2d(3, 32, 3, padding=1), nn.BatchNorm2d(32), nn.ReLU(),
            nn.Conv2d(32, 32, 3, padding=1), nn.BatchNorm2d(32), nn.ReLU(),
            nn.MaxPool2d(2),                                        # 224 -> 112

            nn.Conv2d(32, 64, 3, padding=1), nn.BatchNorm2d(64), nn.ReLU(),
            nn.Conv2d(64, 64, 3, padding=1), nn.BatchNorm2d(64), nn.ReLU(),
            nn.MaxPool2d(2),                                        # 112 -> 56

            nn.Conv2d(64, 128, 3, padding=1), nn.BatchNorm2d(128), nn.ReLU(),
            nn.AdaptiveAvgPool2d(1),                                # -> (N, 128, 1, 1)
        )
        self.classifier = nn.Linear(128, num_classes)
```

```
    def forward(self, x):
        x = self.features(x)          # (N, 128, 1, 1)
        x = torch.flatten(x, 1)       # (N, 128)
        return self.classifier(x)     # (N, num_classes)

model = SimpleCNN(num_classes=10)
dummy = torch.randn(4, 3, 224, 224)
out = model(dummy)
print(out.shape)  # torch.Size([4, 10])
```

Keras:

```
from tensorflow.keras import layers, Model

def build_simple_cnn(num_classes=10):
    inputs = layers.Input(shape=(224, 224, 3))
    x = layers.Conv2D(32, 3, padding="same", activation="relu")(inputs)
    x = layers.BatchNormalization()(x)
    x = layers.MaxPooling2D(2)(x)

    x = layers.Conv2D(64, 3, padding="same", activation="relu")(x)
    x = layers.BatchNormalization()(x)
    x = layers.MaxPooling2D(2)(x)

    x = layers.Conv2D(128, 3, padding="same", activation="relu")(x)
    x = layers.GlobalAveragePooling2D()(x)
    outputs = layers.Dense(num_classes)(x)
    return Model(inputs, outputs)
```

- **Common Mistake**

Forgetting `model.eval()` (PyTorch) before validation/inference. `BatchNorm` and `Dropout` behave differently in train vs. eval mode — `BatchNorm` uses running statistics instead of batch statistics, and `Dropout` is disabled. Leaving a model in train mode during evaluation produces noisy, non-reproducible validation metrics that don't match what a saved/exported model will actually do at inference.

★ Quick Revision Sheet

CNN Deep Dive, in one screen:

- Convolution: kernel slides over input, one kernel per output channel, output shape depends on kernel size, stride, padding, dilation.
- Stacked small kernels (3x3) give large receptive fields with fewer parameters than one big kernel.
- Global average pooling replaces flattening — fewer parameters, resolution-independent.
- Always `model.eval()` before validation/inference in PyTorch; Keras handles this automatically via `fit` vs. `predict`.

Practice Tasks

- **Level 1:** Compute by hand the output shape of a `Conv2d(3, 16, kernel_size=5, stride=2, padding=2)` applied to a `(1, 3, 128, 128)` input, then verify in code.

- **Level 2:** Add a third convolutional block to `SimpleCNN` and confirm the output shape is still correct.
- **Level 3:** Implement the equivalent `SimpleCNN` in Keras and confirm both frameworks produce the same total parameter count.
- **Level 4:** Debug a model that trains fine but produces wildly different validation accuracy each run — check for a missing `model.eval()`.
- **Level 5:** Compare training speed of `MaxPool2d` vs. strided convolution for downsampling.
- **Level 6:** Export `SimpleCNN` to `TorchScript` and confirm inference output matches the eager-mode model.

Chapter 6.6 — CNN Architecture Families

6.6.1 Overview

Modern computer vision rarely starts from a from-scratch architecture. Practical engineering work means choosing the right pretrained backbone. This chapter covers each major family's core idea, when to reach for it, and how to load and fine-tune it.

Architecture	Core Idea	Major Improvement	Pretrained?
LeNet	First practical CNN	Conv + pooling stacking	No (historical/educational only)
AlexNet	Deep CNN at scale	ReLU, dropout, GPU training	Yes (`torchvision`)
VGG	Uniform small kernels	Depth via stacked 3x3 convs	Yes
GoogLeNet/Inception	Multi-scale filters per layer	Inception module, 1x1 bottlenecks	Yes
ResNet	Residual (skip) connections	Trains very deep networks reliably	Yes
DenseNet	Dense skip connections	Feature reuse, fewer parameters	Yes
MobileNet	Depthwise-separable convolution	Efficient mobile/edge inference	Yes
EfficientNet	Compound scaling	Balanced depth/width/resolution scaling	Yes
ConvNeXt	Modernized ConvNet	Transformer-inspired design choices in a pure CNN	Yes

6.6.2 When to Use What

• Best Practice

Default choice for most classification tasks in 2026: a ResNet50 or EfficientNet-B0 for a strong general baseline, MobileNetV3 when you need CPU or mobile inference, and ConvNeXt-Tiny when you want the best accuracy-per-parameter and can afford a modern training recipe. Reach for AlexNet, VGG, or LeNet only for teaching or reproducing historical baselines — they are outclassed on every axis by newer architectures.

6.6.3 Loading Pretrained Models and Transfer Learning

```
import torchvision.models as models
import torch.nn as nn

# ResNet50, pretrained on ImageNet
resnet = models.resnet50(weights=models.ResNet50_Weights.IMAGENET1K_V2)
resnet.fc = nn.Linear(resnet.fc.in_features, num_classes := 10)  # replace classifier head

# MobileNetV3
mobilenet = models.mobilenet_v3_small(weights=models.MobileNet_V3_Small_Weights.IMAGENET1K_V1)
mobilenet.classifier[-1] = nn.Linear(mobilenet.classifier[-1].in_features, 10)

# EfficientNet
efficientnet = models.efficientnet_b0(weights=models.EfficientNet_B0_Weights.IMAGENET1K_V1)
efficientnet.classifier[-1] = nn.Linear(efficientnet.classifier[-1].in_features, 10)

# ConvNeXt
convnext = models.convnext_tiny(weights=models.ConvNeXt_Tiny_Weights.IMAGENET1K_V1)
convnext.classifier[-1] = nn.Linear(convnext.classifier[-1].in_features, 10)
```

- **Common Mistake**

Every architecture family names its final classifier layer differently — `.fc` for ResNet, `.classifier[-1]` for MobileNet/EfficientNet/ConvNeXt, `.heads.head` for some ViT-style models. Copy-pasting a transfer-learning snippet from one architecture to another without checking `print(model)` first silently leaves the old 1000-class ImageNet head in place, or raises an attribute error.

6.6.4 Inception Modules (GoogLeNet)

The Inception module runs multiple kernel sizes (`1x1`, `3x3`, `5x5`) and a pooling branch in parallel on the same input, then concatenates the results — letting the network learn which spatial scale matters at each layer rather than committing to one kernel size upfront. `1x1` convolutions before the larger kernels reduce channel depth first, cutting compute cost — this "bottleneck" pattern reappears in ResNet and nearly every architecture since.

6.6.5 DenseNet: Feature Reuse

Where ResNet adds the input to the output (`F(x) + x`), DenseNet concatenates every preceding layer's output as input to each new layer. This maximizes feature reuse and gradient flow, at the cost of higher memory usage during training due to the growing concatenated feature maps.

6.6.6 Comparison Table

Model	Params (approx.)	Top-1 ImageNet (approx.)	Relative Inference Speed	Typical Use
ResNet50	25M	~80% (v2 weights)	Medium	General-purpose baseline

Model	Params (approx.)	Top-1 ImageNet (approx.)	Relative Inference Speed	Typical Use
MobileNetV3-Small	2.5M	~67%	Fast	Mobile / edge
EfficientNet-B0	5.3M	~77%	Medium-fast	Accuracy/efficiency balance
DenseNet121	8M	~75%	Medium	Feature reuse, medical imaging
ConvNeXt-Tiny	28M	~82%	Medium	Modern high-accuracy baseline
VGG16	138M	~72%	Slow	Legacy/teaching

• Interview Trap

"Which architecture is best?" has no single answer — interviewers are testing whether you reach for latency, accuracy, and deployment-target tradeoffs instead of naming one architecture. Always answer with the constraint first: "for real-time mobile inference I'd start with MobileNetV3; for a server-side accuracy-critical pipeline I'd start with ConvNeXt or EfficientNet and only reach for something heavier if the metric demands it."

★ Quick Revision Sheet

CNN Families, in one screen:

- ResNet: skip connections, trains very deep nets reliably. Default general baseline.
- MobileNet: depthwise-separable convs, built for mobile/edge.
- EfficientNet: compound scaling of depth/width/resolution, strong accuracy-per-FLOP.
- DenseNet: concatenated skip connections, strong feature reuse, higher training memory.
- ConvNeXt: modernized pure-CNN design, competitive with vision transformers.
- Each family names its classifier head differently — always `print(model)` before replacing it.

Practice Tasks

- **Level 1:** List which architectures above use skip/residual connections and which do not.
- **Level 2:** Modify the ResNet50 snippet to freeze all layers except the final block and the new classifier head.
- **Level 3:** Load three different backbones and print their total parameter counts and classifier layer names.
- **Level 4:** Debug a fine-tuning run where accuracy stays at random-chance level — check whether the old 1000-class head was actually replaced.
- **Level 5:** Benchmark inference latency (CPU) for ResNet50 vs. MobileNetV3-Small on the same batch size.
- **Level 6:** Export both models to ONNX and compare file size and inference latency after export.

Chapter 6.7 — ResNet & Skip Connections

6.7.1 The Vanishing Gradient Problem

As networks get deeper, gradients backpropagated through many stacked layers can shrink toward zero (vanish) or grow uncontrollably (explode), making very deep plain CNNs difficult or impossible to train effectively — adding more layers to a plain stack often makes training accuracy *worse*, not better, which is a degradation problem distinct from overfitting.

6.7.2 Residual Connections

ResNet's core idea: instead of a block learning a direct mapping `H(x)`, it learns a residual `F(x) = H(x) - x`, and the block outputs `F(x) + x`. The `+x` identity shortcut gives gradients a direct path back to earlier layers, and gives the network an easy path to learn "do nothing" (`F(x) = 0`) for blocks that aren't useful, rather than having to learn an identity mapping through nonlinear layers, which is much harder.

```
Input x
   |
   +---------------------+
   |                     |
Conv -> BN -> ReLU       |
   |                     |
Conv -> BN               |
   |                     |
   + <-------------------+   (element-wise addition)
   |
 ReLU
   |
Output
```

6.7.3 ResNet Block Implementation

```
import torch
import torch.nn as nn

class BasicBlock(nn.Module):
    """Used in ResNet18/34."""
    def __init__(self, in_channels, out_channels, stride=1):
        super().__init__()
```

```
        self.conv1 = nn.Conv2d(in_channels, out_channels, 3, stride, 1, bias=False)
        self.bn1 = nn.BatchNorm2d(out_channels)
        self.conv2 = nn.Conv2d(out_channels, out_channels, 3, 1, 1, bias=False)
        self.bn2 = nn.BatchNorm2d(out_channels)
        self.relu = nn.ReLU(inplace=True)

        self.shortcut = nn.Sequential()
        if stride != 1 or in_channels != out_channels:
            # Projection shortcut when spatial size or channel count changes
            self.shortcut = nn.Sequential(
                nn.Conv2d(in_channels, out_channels, 1, stride, bias=False),
                nn.BatchNorm2d(out_channels),
            )

    def forward(self, x):
        identity = self.shortcut(x)
        out = self.relu(self.bn1(self.conv1(x)))
        out = self.bn2(self.conv2(out))
        out = out + identity        # the residual addition
        return self.relu(out)
```

Variant	Block Type	Depth	Typical Use
ResNet18	BasicBlock	18 layers	Fast baseline, edge-friendly
ResNet34	BasicBlock	34 layers	Slightly stronger baseline
ResNet50	Bottleneck (1x1-3x3-1x1)	50 layers	Standard production baseline
ResNet101	Bottleneck	101 layers	Higher accuracy, more compute

6.7.4 Using Pretrained ResNet: Feature Extraction vs. Fine-Tuning

```
import torchvision.models as models
import torch.nn as nn

# --- Feature extraction: freeze backbone, train only the new head ---
model = models.resnet50(weights=models.ResNet50_Weights.IMAGENET1K_V2)
for param in model.parameters():
    param.requires_grad = False
model.fc = nn.Linear(model.fc.in_features, 10)   # new head is trainable by default

# --- Full fine-tuning: unfreeze everything, train with a small learning rate ---
model_ft = models.resnet50(weights=models.ResNet50_Weights.IMAGENET1K_V2)
model_ft.fc = nn.Linear(model_ft.fc.in_features, 10)
optimizer = torch.optim.Adam(model_ft.parameters(), lr=1e-4)   # small LR to avoid destroying pretrained featu

# --- Partial fine-tuning: unfreeze only the last residual stage ---
model_partial = models.resnet50(weights=models.ResNet50_Weights.IMAGENET1K_V2)
```

```python
for param in model_partial.parameters():
    param.requires_grad = False
for param in model_partial.layer4.parameters():   # last stage
    param.requires_grad = True
model_partial.fc = nn.Linear(model_partial.fc.in_features, 10)
```

Strategy	When to use
Feature extraction (frozen backbone)	Small dataset, similar domain to ImageNet
Partial fine-tuning (unfreeze last stage)	Medium dataset, somewhat different domain
Full fine-tuning	Large dataset, or domain very different from ImageNet (e.g. medical, satellite)

• Best Practice

When fine-tuning a pretrained backbone, use a learning rate 10-100x smaller than you would use training from scratch (e.g. `1e-4` instead of `1e-2`). The pretrained weights already encode useful features; a large learning rate destroys them within the first few batches, a failure mode sometimes called "catastrophic forgetting."

• Common Mistake

Forgetting to set `requires_grad = True` on the newly-created classifier head after freezing all other parameters. New layers created after a loop that sets `requires_grad = False` default to trainable, but if you replace the head *before* the freezing loop, the loop will freeze the new head too, and the model will train nothing.

★ Quick Revision Sheet

ResNet, in one screen:

- Residual connection: block output is `F(x) + x`; gives gradients a direct path and an easy identity fallback.
- BasicBlock (ResNet18/34) vs. Bottleneck (ResNet50/101) — bottleneck uses 1x1-3x3-1x1 to cut compute at depth.
- Freeze backbone for small/similar datasets; partially or fully fine-tune for larger/different datasets.
- Use a small learning rate (10-100x smaller) when fine-tuning to avoid destroying pretrained features.

Practice Tasks

- **Level 1:** Explain in one sentence why the shortcut path in `BasicBlock` needs a projection when `stride != 1`.
- **Level 2:** Modify `BasicBlock` into a `Bottleneck` block (1x1 reduce, 3x3, 1x1 expand).
- **Level 3:** Build a tiny 4-block ResNet-style network from scratch and train it on CIFAR-10.
- **Level 4:** Debug a fine-tuning run that diverges (loss goes to NaN) within the first epoch — check the learning rate.
- **Level 5:** Compare convergence speed of feature-extraction vs. full fine-tuning on a small (500-image) dataset.
- **Level 6:** Export a fine-tuned ResNet50 to TorchScript and confirm parity with the eager-mode model's predictions.

Chapter 6.8 — MobileNet & Efficient Models

6.8.1 Depthwise Separable Convolution

A standard convolution mixes spatial and cross-channel information in one operation, which is expensive. MobileNet splits this into two cheaper steps:

1. **Depthwise convolution**: one filter per input channel, spatial-only (no cross-channel mixing).
2. **Pointwise convolution**: a `1x1` convolution that mixes channels.

```
Standard conv cost:    kH * kW * C_in * C_out * H_out * W_out
Depthwise-sep cost:    (kH * kW * C_in * H_out * W_out) + (C_in * C_out * H_out * W_out)
```

For a typical `3x3` kernel this reduces compute by roughly 8-9x with a small accuracy cost — the central tradeoff that makes MobileNet viable on phones and edge devices.

```
import torch.nn as nn

class DepthwiseSeparableConv(nn.Module):
    def __init__(self, in_channels, out_channels, stride=1):
        super().__init__()
        self.depthwise = nn.Conv2d(
            in_channels, in_channels, kernel_size=3, stride=stride,
            padding=1, groups=in_channels, bias=False   # groups=in_channels -> depthwise
        )
        self.pointwise = nn.Conv2d(in_channels, out_channels, kernel_size=1, bias=False)
        self.bn1 = nn.BatchNorm2d(in_channels)
        self.bn2 = nn.BatchNorm2d(out_channels)
        self.relu = nn.ReLU6(inplace=True)   # ReLU6 used throughout MobileNet for quantization-friendliness

    def forward(self, x):
        x = self.relu(self.bn1(self.depthwise(x)))
        x = self.relu(self.bn2(self.pointwise(x)))
        return x
```

6.8.2 Inverted Residuals (MobileNetV2/V3)

MobileNetV2 introduced the inverted residual block: expand channels with a `1x1` conv, apply a depthwise `3x3` conv, then project back down with another `1x1` conv — the opposite of ResNet's bottleneck (which narrows then widens). A

skip connection links the narrow input/output when shapes match, keeping activation memory low while still allowing rich intermediate representations.

MobileNetV3 adds squeeze-and-excitation attention blocks and a hardware-aware neural architecture search to further optimize the latency/accuracy tradeoff.

6.8.3 EfficientNet: Compound Scaling

Rather than arbitrarily scaling depth, width, or input resolution independently, EfficientNet scales all three together using a fixed compound coefficient, found to produce consistently better accuracy-per-FLOP than scaling any single dimension alone.

```
import torchvision.models as models

efficientnet_b0 = models.efficientnet_b0(weights=models.EfficientNet_B0_Weights.IMAGENET1K_V1)
efficientnet_b4 = models.efficientnet_b4(weights=models.EfficientNet_B4_Weights.IMAGENET1K_V1)
# B0 -> B7: each step scales depth, width, and input resolution together
```

6.8.4 Model Size, Latency, and Mobile Inference

Model	Params	Approx. mobile CPU latency (224px)	Notes
MobileNetV3-Small	2.5M	Fastest	Best for real-time mobile
MobileNetV3-Large	5.4M	Fast	Better accuracy, still mobile-friendly
EfficientNet-B0	5.3M	Medium	Good accuracy/latency balance
ResNet50	25M	Slow on mobile CPU	Server-side default, not mobile-first

• Best Practice

When targeting mobile or edge deployment, benchmark actual on-device latency early — don't rely on parameter count or FLOPs alone as a proxy. Depthwise convolutions have poor FLOP-to-latency efficiency on some mobile GPU/NPU backends despite low theoretical FLOP counts, so the "efficient" architecture on paper isn't always the fastest on a specific target chip.

6.8.5 Edge Deployment Considerations

```
Trained Model (PyTorch/Keras)
      |
      v
Export (ONNX / TFLite / Core ML)
      |
      v
Quantization (float32 -> int8, optional)
      |
```

```
      v
On-device Runtime (ONNX Runtime Mobile / LiteRT / Core ML)
```

Quantization (converting weights from 32-bit floats to 8-bit integers) typically shrinks model size 4x and speeds up inference on supporting hardware, at a small, usually acceptable, accuracy cost.

★ Quick Revision Sheet

MobileNet & Efficient Models, in one screen:

- Depthwise-separable convolution: depthwise (spatial) + pointwise (channel mixing) instead of one expensive standard conv.
- Inverted residuals (MobileNetV2/V3): expand -> depthwise -> project, opposite of ResNet's bottleneck shape.
- EfficientNet: compound scaling of depth, width, and resolution together.
- Always benchmark real on-device latency for edge targets — theoretical FLOPs don't always predict it.
- Quantization (int8) typically gives 4x size reduction with modest accuracy cost.

Practice Tasks

- **Level 1:** Compute the FLOP reduction factor of a depthwise-separable conv vs. a standard conv for `C_in=64`, `C_out=128`, `k=3`.
- **Level 2:** Modify `DepthwiseSeparableConv` to include a skip connection when `stride == 1` and `in_channels == out_channels`.
- **Level 3:** Fine-tune MobileNetV3-Small on a custom dataset and measure CPU inference latency before/after quantization.
- **Level 4:** Debug a quantized model whose accuracy drops sharply — check whether batch norm folding was applied correctly during export.
- **Level 5:** Compare EfficientNet-B0 vs. MobileNetV3-Large on the same dataset for both accuracy and latency.
- **Level 6:** Export a fine-tuned MobileNetV3 model to TFLite with int8 quantization and validate output parity within an acceptable tolerance.

Chapter 6.9 — Transfer Learning for Computer Vision

Transfer learning is the default starting point for almost every practical CV project. Training a competitive CNN from random initialization requires enormous labeled datasets and compute; starting from ImageNet-pretrained weights gets most projects to strong accuracy with a few hundred to a few thousand labeled images.

6.9.1 The Standard Workflow

```
Pretrained Model
      |
      v
Freeze Backbone
      |
      v
Replace Head
      |
      v
Train Head
      |
      v
Unfreeze Selected Layers
      |
      v
Fine-tune (small learning rate)
```

6.9.2 Complete Workflow Implementation

```
import torch
import torch.nn as nn
import torchvision.models as models

def build_transfer_model(num_classes, freeze_backbone=True):
    model = models.resnet50(weights=models.ResNet50_Weights.IMAGENET1K_V2)
    if freeze_backbone:
        for param in model.parameters():
            param.requires_grad = False
```

```
    model.fc = nn.Linear(model.fc.in_features, num_classes)  # created after freezing -> trainable
    return model

model = build_transfer_model(num_classes=5)

# Stage 1: train only the head
optimizer = torch.optim.Adam(model.fc.parameters(), lr=1e-3)
# ... train for a few epochs ...

# Stage 2: unfreeze the last residual stage and fine-tune with a smaller LR
for param in model.layer4.parameters():
    param.requires_grad = True

optimizer = torch.optim.Adam([
    {"params": model.layer4.parameters(), "lr": 1e-5},   # discriminative LR: smaller for pretrained layers
    {"params": model.fc.parameters(), "lr": 1e-4},        # larger for the new head
])
```

6.9.3 Discriminative Learning Rates

Earlier layers in a pretrained CNN learn generic features (edges, textures, colors) that transfer well across almost any visual domain; later layers learn task-specific, higher-level features. Discriminative learning rates apply smaller learning rates to earlier layers (change them less) and larger learning rates to later layers and the new head (let them adapt more), which tends to produce more stable fine-tuning than a single global learning rate.

```
def get_discriminative_param_groups(model, base_lr=1e-4):
    return [
        {"params": model.layer1.parameters(), "lr": base_lr * 0.01},
        {"params": model.layer2.parameters(), "lr": base_lr * 0.05},
        {"params": model.layer3.parameters(), "lr": base_lr * 0.1},
        {"params": model.layer4.parameters(), "lr": base_lr * 0.5},
        {"params": model.fc.parameters(), "lr": base_lr},
    ]
```

6.9.4 Choosing Freeze Depth

Dataset size	Domain similarity to ImageNet	Recommended strategy
Small (< 1,000 images/class)	High (natural photos)	Freeze entire backbone, train head only
Small	Low (medical, satellite, X-ray)	Freeze most, unfreeze last 1-2 stages
Medium (1,000-10,000/class)	High	Unfreeze last 1-2 stages
Medium	Low	Full fine-tuning with discriminative LR
Large (10,000+/class)	Any	Full fine-tuning, pretrained weights mainly help initialization/convergence speed

• Common Mistake

Full fine-tuning on a very small dataset (a few hundred images) with a single global learning rate. Every parameter in the network — including the parts that learned excellent, general-purpose ImageNet features — becomes free to move, and a small dataset provides too weak a gradient signal to keep them from drifting into an overfit, worse state than the frozen-backbone approach. Start frozen, and only unfreeze if there's a clear validation-accuracy reason to.

• Best Practice

Always train the new classifier head alone for a few epochs *before* unfreezing anything. Freshly-initialized head weights produce large, somewhat random gradients early in training; if the backbone is unfrozen from step one, those large gradients propagate back and can meaningfully perturb the pretrained backbone weights before the head has learned anything useful.

★ Quick Revision Sheet

Transfer Learning, in one screen:

- Standard flow: freeze backbone -> replace head -> train head -> selectively unfreeze -> fine-tune with small LR.
- Discriminative LRs: smaller for early (generic) layers, larger for late (task-specific) layers and the new head.
- Freeze depth depends on dataset size and domain similarity to ImageNet — smaller/more-different datasets favor freezing more.
- Train the head alone first before unfreezing anything, to avoid perturbing the backbone with noisy early gradients.

Practice Tasks

- **Level 1:** Explain why later CNN layers are considered more "task-specific" than earlier layers.
- **Level 2:** Modify `get_discriminative_param_groups` to work with an EfficientNet backbone instead of ResNet.
- **Level 3:** Implement the full two-stage workflow (train head, then fine-tune) end-to-end on a small custom image dataset.
- **Level 4:** Debug a fine-tuning run where validation accuracy is worse than the frozen-backbone baseline — check learning rate and unfreeze depth.
- **Level 5:** Compare final accuracy across three freeze-depth strategies on the same small dataset.
- **Level 6:** Package the two-stage training workflow into a reusable function that accepts any torchvision backbone.

Chapter 6.10 — Image Classification

6.10.1 Classification Task Types

Type	Description	Output layer / loss
Binary	Two mutually exclusive classes	1 output unit, sigmoid, `BCEWithLogitsLoss`
Multiclass	Exactly one label from N classes	N output units, softmax, `CrossEntropyLoss`
Multilabel	Zero or more labels from N classes, independently	N output units, sigmoid per unit, `BCEWithLogitsLoss`

6.10.2 Dataset and DataLoader

```
import torch
from torch.utils.data import Dataset, DataLoader
from PIL import Image
import os

class ImageFolderDataset(Dataset):
    def __init__(self, root_dir, transform=None):
        self.samples = []
        self.classes = sorted(os.listdir(root_dir))
        self.class_to_idx = {c: i for i, c in enumerate(self.classes)}
        for cls in self.classes:
            cls_dir = os.path.join(root_dir, cls)
            for fname in os.listdir(cls_dir):
                self.samples.append((os.path.join(cls_dir, fname), self.class_to_idx[cls]))
        self.transform = transform

    def __len__(self):
        return len(self.samples)

    def __getitem__(self, idx):
        path, label = self.samples[idx]
        img = Image.open(path).convert("RGB")
        if img is None:
            raise FileNotFoundError(path)
```

```
        if self.transform:
            img = self.transform(img)
        return img, label

train_loader = DataLoader(
    ImageFolderDataset("data/train", transform=train_transform),
    batch_size=32, shuffle=True, num_workers=4, pin_memory=True,
)
val_loader = DataLoader(
    ImageFolderDataset("data/val", transform=val_transform),
    batch_size=32, shuffle=False, num_workers=4, pin_memory=True,
)
```

6.10.3 Complete Training Loop

```
import torch
import torch.nn as nn

def train_one_epoch(model, loader, optimizer, criterion, device):
    model.train()
    total_loss, correct, total = 0.0, 0, 0
    for images, labels in loader:
        images, labels = images.to(device), labels.to(device)
        optimizer.zero_grad()
        outputs = model(images)
        loss = criterion(outputs, labels)
        loss.backward()
        optimizer.step()

        total_loss += loss.item() * images.size(0)
        correct += (outputs.argmax(1) == labels).sum().item()
        total += images.size(0)
    return total_loss / total, correct / total

@torch.no_grad()
def validate(model, loader, criterion, device):
    model.eval()                          # disables dropout, freezes BN running stats
    total_loss, correct, total = 0.0, 0, 0
    for images, labels in loader:
        images, labels = images.to(device), labels.to(device)
        outputs = model(images)
        loss = criterion(outputs, labels)
        total_loss += loss.item() * images.size(0)
        correct += (outputs.argmax(1) == labels).sum().item()
        total += images.size(0)
    return total_loss / total, correct / total

device = "cuda" if torch.cuda.is_available() else "cpu"
```

```
model = build_transfer_model(num_classes=5).to(device)
criterion = nn.CrossEntropyLoss()
optimizer = torch.optim.Adam(model.parameters(), lr=1e-4)

best_val_acc = 0.0
for epoch in range(20):
    train_loss, train_acc = train_one_epoch(model, train_loader, optimizer, criterion, device)
    val_loss, val_acc = validate(model, val_loader, criterion, device)
    print(f"Epoch {epoch+1}: train_loss={train_loss:.4f} train_acc={train_acc:.4f} "
          f"val_loss={val_loss:.4f} val_acc={val_acc:.4f}")
    if val_acc > best_val_acc:
        best_val_acc = val_acc
        torch.save(model.state_dict(), "best_model.pt")   # checkpoint the best model only
```

6.10.4 Confusion Matrix and Inference

```
from sklearn.metrics import confusion_matrix, classification_report
import numpy as np

@torch.no_grad()
def evaluate_full(model, loader, device, class_names):
    model.eval()
    all_preds, all_labels = [], []
    for images, labels in loader:
        images = images.to(device)
        preds = model(images).argmax(1).cpu().numpy()
        all_preds.extend(preds)
        all_labels.extend(labels.numpy())
    cm = confusion_matrix(all_labels, all_preds)
    report = classification_report(all_labels, all_preds, target_names=class_names)
    return cm, report

def predict_single_image(model, image_path, transform, device, class_names):
    model.eval()
    img = Image.open(image_path).convert("RGB")
    tensor = transform(img).unsqueeze(0).to(device)
    with torch.no_grad():
        logits = model(tensor)
        probs = torch.softmax(logits, dim=1)
        pred_idx = probs.argmax(1).item()
    return class_names[pred_idx], probs[0, pred_idx].item()
```

6.10.5 Three Classifiers, in Increasing Production-Readiness

1. Simple CNN classifier — use `SimpleCNN` from Chapter 6.5, trained from scratch. Appropriate for large, simple datasets (e.g. digit or shape recognition) where transfer learning offers little benefit.

2. Transfer-learning classifier — `build_transfer_model` from Chapter 6.9. The default choice for almost every real

project with a moderate-sized labeled dataset.

3. Production-style classifier — adds checkpointing, early stopping, logging, and a clean inference interface:

```python
class ProductionClassifier:
    def __init__(self, model, transform, class_names, device):
        self.model = model.to(device).eval()
        self.transform = transform
        self.class_names = class_names
        self.device = device

    @classmethod
    def from_checkpoint(cls, checkpoint_path, num_classes, class_names, transform, device):
        model = build_transfer_model(num_classes=num_classes, freeze_backbone=False)
        model.load_state_dict(torch.load(checkpoint_path, map_location=device))
        return cls(model, transform, class_names, device)

    @torch.no_grad()
    def predict(self, image_path, top_k=1):
        img = Image.open(image_path).convert("RGB")
        tensor = self.transform(img).unsqueeze(0).to(self.device)
        probs = torch.softmax(self.model(tensor), dim=1)[0]
        top_probs, top_idx = probs.topk(top_k)
        return [(self.class_names[i], p.item()) for i, p in zip(top_idx, top_probs)]
```

- **Common Mistake**

Checkpointing every epoch's model instead of only the best-validation-accuracy model. Aside from wasting disk space, it's easy to accidentally load the final (possibly overfit) epoch instead of the best one at deployment time. Track and save only the best checkpoint by a chosen validation metric.

★ Quick Revision Sheet

Image Classification, in one screen:

- Binary: sigmoid + BCE. Multiclass: softmax + cross-entropy. Multilabel: per-class sigmoid + BCE.
- Always call `model.eval()` and wrap inference in `torch.no_grad()` for validation and prediction.
- Save only the best checkpoint by validation metric, not every epoch.
- A production classifier wraps preprocessing, model, and class names into one deterministic, testable interface.

Practice Tasks

- **Level 1:** Convert the multiclass training loop above to a multilabel setup (sigmoid + BCE, threshold-based prediction).
- **Level 2:** Add early stopping (stop if validation accuracy hasn't improved in 5 epochs) to the training loop.
- **Level 3:** Implement `ProductionClassifier` end-to-end and confirm it produces identical predictions to the raw model.
- **Level 4:** Debug a classifier stuck at exactly `1/num_classes` accuracy — check for a frozen-and-unreplaced head or a learning rate of zero.
- **Level 5:** Profile the training loop to determine whether the DataLoader or the GPU forward/backward pass is the bottleneck.
- **Level 6:** Wrap `ProductionClassifier` in a minimal FastAPI endpoint that accepts an uploaded image and

returns top-3 predictions.

Chapter 6.11 — Classification Metrics

6.11.1 The Core Metrics

Metric	Formula	Question it answers
Accuracy	`(TP+TN)/(TP+TN+FP+FN)`	What fraction of predictions were correct?
Precision	`TP/(TP+FP)`	Of predicted positives, how many were actually positive?
Recall (Sensitivity)	`TP/(TP+FN)`	Of actual positives, how many did we catch?
Specificity	`TN/(TN+FP)`	Of actual negatives, how many did we correctly reject?
F1	`2 * (Precision * Recall) / (Precision + Recall)`	Harmonic mean balancing precision and recall
Top-k accuracy	Correct if true label is in top-k predictions	Common for large-N multiclass problems (e.g. ImageNet)

```
from sklearn.metrics import (
    accuracy_score, precision_score, recall_score, f1_score,
    roc_auc_score, average_precision_score, confusion_matrix,
)

acc = accuracy_score(y_true, y_pred)
precision = precision_score(y_true, y_pred, average="macro")
recall = recall_score(y_true, y_pred, average="macro")
f1 = f1_score(y_true, y_pred, average="macro")
```

6.11.2 ROC-AUC and PR-AUC

- **ROC-AUC** plots true positive rate vs. false positive rate across thresholds. Good general-purpose ranking metric.
- **PR-AUC** (precision-recall AUC) plots precision vs. recall across thresholds. More informative than ROC-AUC on imbalanced datasets, because it doesn't count the (often huge) number of true negatives.

```
roc_auc = roc_auc_score(y_true, y_scores)          # y_scores = predicted probability of positive class
pr_auc = average_precision_score(y_true, y_scores)
```

• Interview Trap

"When is accuracy a bad metric?" — on imbalanced datasets. A dataset with 99% negative and 1% positive samples gets 99% accuracy by predicting "negative" for everything, while catching zero true positives. Always report precision, recall, and F1 (or PR-AUC) alongside accuracy whenever class balance isn't guaranteed to be roughly even.

6.11.3 Metric Choice by Domain

Domain	Preferred metric(s)	Why
Medical imaging (e.g. tumor detection)	Recall (sensitivity), PR-AUC	Missing a positive case (false negative) is far more costly than a false alarm
Fraud-like image classification (e.g. counterfeit detection)	Precision, PR-AUC	False positives create costly manual review or false accusations
Safety systems (e.g. defect detection on a production line)	Recall, with a human-review step for positives	Missing a defect is much costlier than flagging a false one
Balanced datasets, general classification	Accuracy, F1	No strong asymmetry between error types
Highly imbalanced datasets	PR-AUC, F1, per-class recall	ROC-AUC and raw accuracy can look deceptively good

6.11.4 Confusion Matrix in Practice

```
import matplotlib.pyplot as plt
from sklearn.metrics import ConfusionMatrixDisplay

cm = confusion_matrix(y_true, y_pred)
disp = ConfusionMatrixDisplay(confusion_matrix=cm, display_labels=class_names)
disp.plot(cmap="Blues", xticks_rotation=45)
plt.tight_layout()
plt.savefig("confusion_matrix.png")
```

• Best Practice

Look at the confusion matrix, not just the aggregate metric. A model with 90% overall accuracy might be making all its errors between two visually similar classes (e.g. "wolf" vs. "husky") — a completely different, more fixable problem than errors spread randomly across every class pair.

★ Quick Revision Sheet

Classification Metrics, in one screen:

- Accuracy is misleading on imbalanced data — always pair it with precision/recall/F1.
- Medical/safety domains: prioritize recall (don't miss positives).
- Fraud-like/high-false-positive-cost domains: prioritize precision.
- PR-AUC beats ROC-AUC as a summary metric on imbalanced datasets.
- Always inspect the confusion matrix, not just the scalar metric.

Practice Tasks

- **Level 1:** Given TP=80, FP=20, FN=10, TN=890, compute accuracy, precision, recall, and F1 by hand.
- **Level 2:** Modify the metric-computation snippet to report per-class precision/recall instead of macro-averaged.
- **Level 3:** Implement a full evaluation report (confusion matrix + classification report + PR-AUC) for a trained classifier.
- **Level 4:** Debug a model reporting 98% accuracy but 0.1 recall on the minority class — diagnose class imbalance from the confusion matrix.
- **Level 5:** Compare macro vs. weighted vs. micro averaging on a 10-class dataset with strong class imbalance.
- **Level 6:** Build a threshold-tuning routine that selects the classification threshold maximizing F1 on a held-out validation set.

Chapter 6.12 — Object Detection Fundamentals

Object detection extends classification with localization: for every object in an image, predict both *what* it is and *where* it is, as a bounding box.

6.12.1 Bounding Box Representations

Format	Meaning	Used by
`xyxy`	`(x_min, y_min, x_max, y_max)`	torchvision, COCO evaluation
`xywh` (corner)	`(x_min, y_min, width, height)`	COCO annotation format
`cxcywh`	`(center_x, center_y, width, height)`, often normalized `[0,1]`	YOLO

```python
def xyxy_to_cxcywh(box):
    x1, y1, x2, y2 = box
    return [(x1 + x2) / 2, (y1 + y2) / 2, x2 - x1, y2 - y1]

def cxcywh_to_xyxy(box):
    cx, cy, w, h = box
    return [cx - w / 2, cy - h / 2, cx + w / 2, cy + h / 2]
```

- **Common Mistake**

Mixing normalized (`[0,1]`, relative to image size — YOLO's format) and absolute pixel coordinate boxes without converting. This produces boxes that are either invisibly tiny or wildly oversized, and the bug is easy to miss because the code runs without error — it just draws garbage.

6.12.2 Confidence, Objectness, and Class Labels

Every detection carries: box coordinates, a class label, and a confidence score. Many detectors (YOLO, SSD, RetinaNet) additionally predict an "objectness" score — the probability that *any* object exists at that location — separately from the per-class probability, then multiply the two together for the final reported confidence.

6.12.3 Anchors vs. Anchor-Free Detection

- **Anchor-based**: the model predicts offsets relative to a fixed set of predefined box shapes (anchors) tiled across the image at each spatial location. Requires choosing anchor sizes/ratios in advance (often via k-means on the training set's box sizes).
- **Anchor-free**: the model directly predicts box coordinates or distances from a center point, with no predefined shapes. Simpler to configure, avoids the anchor-tuning step, used by modern YOLO variants and detectors like FCOS.

6.12.4 The Detection Pipeline

```
Image
 |
 v
Feature Extraction (backbone CNN)
 |
 v
Detection Head (per-location box + class + confidence predictions)
 |
 v
Raw Predictions (many overlapping, low-confidence boxes)
 |
 v
Post-processing (confidence threshold + NMS)
 |
 v
Final Boxes + Classes + Confidence
```

★ Quick Revision Sheet

Detection Fundamentals, in one screen:

- Boxes come in `xyxy`, `xywh`, and `cxcywh` formats — always know which one a given API expects.
- Confidence is often objectness $\times$ class probability.
- Anchor-based detectors need tuned anchor boxes; anchor-free detectors predict coordinates directly.
- Raw model output is noisy and overlapping — post-processing (thresholding + NMS) is mandatory, not optional.

Practice Tasks

- **Level 1:** Convert a box `(50, 50, 150, 200)` in `xyxy` to `cxcywh` and back, confirming round-trip correctness.
- **Level 2:** Write a function that converts a full COCO-format annotation list to normalized YOLO `cxcywh` format.
- **Level 3:** Implement a k-means anchor generator over a set of training-set box dimensions.
- **Level 4:** Debug a detector drawing boxes that are all clustered in the top-left corner — check for a normalized/absolute coordinate mismatch.
- **Level 5:** Compare inference latency of an anchor-based vs. anchor-free head on the same backbone.
- **Level 6:** Wrap box-format conversion utilities into a single tested module reused across the training and inference code paths.

Chapter 6.13 — IoU & Non-Maximum Suppression

6.13.1 Intersection over Union

IoU measures how well two boxes overlap: the area of their intersection divided by the area of their union. It ranges from 0 (no overlap) to 1 (identical boxes), and is the standard way to decide whether a predicted box "matches" a ground-truth box.

```
def compute_iou(box1, box2):
    """boxes in xyxy format: [x1, y1, x2, y2]"""
    x1 = max(box1[0], box2[0])
    y1 = max(box1[1], box2[1])
    x2 = min(box1[2], box2[2])
    y2 = min(box1[3], box2[3])

    inter_area = max(0, x2 - x1) * max(0, y2 - y1)
    area1 = (box1[2] - box1[0]) * (box1[3] - box1[1])
    area2 = (box2[2] - box2[0]) * (box2[3] - box2[1])
    union_area = area1 + area2 - inter_area

    return inter_area / union_area if union_area > 0 else 0.0
```

6.13.2 Non-Maximum Suppression From Scratch

A detector's raw output typically has many overlapping boxes around the same object. NMS keeps only the highest-confidence box in each cluster of overlapping boxes and discards the rest.

```
import numpy as np

def nms(boxes, scores, iou_threshold=0.5):
    """boxes: (N, 4) xyxy, scores: (N,) confidence. Returns indices to keep."""
    if len(boxes) == 0:
        return []
    boxes = np.array(boxes)
    scores = np.array(scores)
    order = scores.argsort()[::-1]   # sort by confidence, descending

    keep = []
```

```
    while len(order) > 0:
        current = order[0]
        keep.append(current)
        if len(order) == 1:
            break
        rest = order[1:]
        ious = np.array([compute_iou(boxes[current], boxes[i]) for i in rest])
        order = rest[ious < iou_threshold]   # discard boxes that overlap too much with the kept box
    return keep
```

6.13.3 Confidence Threshold + IoU Threshold: Two Separate Knobs

```
def postprocess_detections(boxes, scores, classes, conf_threshold=0.25, iou_threshold=0.45):
    keep_conf = scores > conf_threshold
    boxes, scores, classes = boxes[keep_conf], scores[keep_conf], classes[keep_conf]

    final_boxes, final_scores, final_classes = [], [], []
    for c in np.unique(classes):                    # class-aware NMS: run separately per class
        cls_mask = classes == c
        cls_boxes, cls_scores = boxes[cls_mask], scores[cls_mask]
        keep = nms(cls_boxes, cls_scores, iou_threshold)
        final_boxes.extend(cls_boxes[keep])
        final_scores.extend(cls_scores[keep])
        final_classes.extend([c] * len(keep))
    return final_boxes, final_scores, final_classes
```

Threshold	Effect of raising it	Effect of lowering it
Confidence threshold	Fewer, higher-confidence detections; more missed objects	More detections; more false positives
IoU threshold (NMS)	More overlapping boxes survive (useful for dense/crowded scenes)	More aggressive suppression, risk of removing valid nearby objects

6.13.4 Class-Aware NMS and Soft-NMS

Standard NMS run globally (ignoring class) can incorrectly suppress a correct detection of one class because it overlaps with a detection of a different class — class-aware NMS (grouping by class before running NMS, as above) avoids this. **Soft-NMS** is a refinement that decays the score of overlapping boxes proportionally to their IoU with the kept box instead of discarding them outright, which helps in dense scenes with legitimately overlapping objects (e.g. a crowd of people).

6.13.5 Framework Implementations

```
import torchvision.ops as ops
import torch

boxes = torch.tensor([[10, 10, 100, 100], [12, 12, 98, 98], [200, 200, 300, 300]], dtype=torch.float32)
```

```
scores = torch.tensor([0.9, 0.75, 0.8])
keep_idx = ops.nms(boxes, scores, iou_threshold=0.5)   # torchvision's optimized NMS

# class-aware batched NMS
class_ids = torch.tensor([0, 0, 1])
keep_idx_batched = ops.batched_nms(boxes, scores, class_ids, iou_threshold=0.5)
```

• Best Practice

Use `torchvision.ops.nms` / `batched_nms` in production code rather than a from-scratch Python implementation — they're implemented in C++/CUDA and are dramatically faster, especially with hundreds or thousands of raw candidate boxes. Implement NMS from scratch only to build understanding, as above.

★ Quick Revision Sheet

IoU & NMS, in one screen:

- IoU = intersection area / union area; standard measure of box overlap.
- NMS keeps the highest-confidence box per overlapping cluster, discards the rest above an IoU threshold.
- Confidence threshold and IoU threshold are independent knobs with opposite-direction tradeoffs.
- Run NMS per class (class-aware) to avoid cross-class suppression errors.
- Use the framework's optimized NMS in production; understand the from-scratch version conceptually.

Practice Tasks

- **Level 1:** Compute IoU by hand for boxes `[0,0,10,10]` and `[5,5,15,15]`.
- **Level 2:** Modify `nms` to implement soft-NMS (score decay instead of hard removal).
- **Level 3:** Implement class-aware NMS from scratch and verify it matches `torchvision.ops.batched_nms` on a test case.
- **Level 4:** Debug a detector producing 5 overlapping boxes around one object — confirm whether NMS is being applied at all, and at what threshold.
- **Level 5:** Benchmark the from-scratch NMS vs. `torchvision.ops.nms` on 10,000 candidate boxes.
- **Level 6:** Tune confidence and IoU thresholds on a validation set to maximize mAP for a specific deployment scenario.

Chapter 6.14 — YOLO Fundamentals

YOLO ("You Only Look Once") is the dominant practical choice for real-time object detection. This chapter covers the concepts shared across the YOLO family and the modern Python workflow for using it.

6.14.1 One-Stage Detection

YOLO is a one-stage detector: a single forward pass through the network directly produces boxes, classes, and confidences — no separate region-proposal step (contrast with Faster R-CNN in Chapter 6.19). This makes YOLO fast enough for real-time video, at a historically small accuracy cost versus two-stage detectors that has mostly closed in modern versions.

6.14.2 The Grid Concept

Conceptually, YOLO divides the image into a grid; each grid cell is responsible for detecting objects whose center falls inside it. Each cell predicts a fixed number of boxes (via anchors, in older versions, or directly in modern anchor-free versions), each with a confidence score and class probabilities. Modern YOLO architectures predict at multiple grid resolutions simultaneously (a feature pyramid), so both large and small objects are detected well — a single grid resolution struggles to represent both.

```
Image
 |
 v
Backbone (feature extraction, multiple resolutions)
 |
 v
Neck (feature pyramid — fuses multi-scale features)
 |
 v
Detection Heads (one per scale: box + class + confidence)
 |
 v
NMS
 |
 v
Final Detections
```

6.14.3 Practical Usage (Modern Python Workflow)

```
from ultralytics import YOLO

# Load a pretrained model (COCO-trained, 80 classes)
model = YOLO("yolo11n.pt")   # "n" = nano, smallest/fastest variant

# Inference on a single image
results = model("street_scene.jpg")
for result in results:
    boxes = result.boxes.xyxy       # (N, 4) tensor
    confidences = result.boxes.conf  # (N,) tensor
    class_ids = result.boxes.cls     # (N,) tensor
    result.save(filename="annotated.jpg")

# Inference on a video file / webcam
results = model("video.mp4", stream=True)   # stream=True for memory-efficient frame iteration
for result in results:
    frame = result.plot()   # BGR numpy array with boxes drawn
```

- **Common Mistake**

Iterating over `model(video_path)` without `stream=True` on a long video loads every frame's results into memory at once, which can exhaust RAM on anything longer than a short clip. Always use `stream=True` for video or large batch inference and process results one frame at a time.

6.14.4 Confidence and Class Prediction

```
results = model("image.jpg", conf=0.25, iou=0.45)   # confidence and NMS IoU thresholds, tunable at inference

for result in results:
    for box in result.boxes:
        cls_name = model.names[int(box.cls)]
        conf = float(box.conf)
        x1, y1, x2, y2 = box.xyxy[0].tolist()
        print(f"{cls_name}: {conf:.2f} at [{x1:.0f}, {y1:.0f}, {x2:.0f}, {y2:.0f}]")
```

6.14.5 Validation

```
metrics = model.val(data="coco8.yaml")   # runs full validation, returns mAP metrics
print(metrics.box.map)       # mAP@0.5:0.95
print(metrics.box.map50)     # mAP@0.5
```

• Best Practice

Start every custom YOLO project from a pretrained checkpoint (e.g. `yolo11n.pt`), never from random weights, even if your target classes don't overlap with COCO's 80 classes. The pretrained backbone's general-purpose visual features transfer well and converge far faster than training detection from scratch, exactly as with classification transfer learning.

• Interview Trap

"Is YOLO always the right choice for detection?" No — it's the right choice when real-time or near-real-time inference speed matters and a small accuracy gap versus two-stage detectors is acceptable. For offline batch processing where maximum accuracy is the only priority (e.g. analyzing a fixed dataset of satellite images with no latency constraint), a two-stage detector like Faster R-CNN can still be the better choice.

★ Quick Revision Sheet

YOLO Fundamentals, in one screen:

- One-stage detector: single forward pass produces boxes + classes + confidence directly.
- Multi-scale detection heads (feature pyramid) handle both large and small objects.
- Always start from a pretrained checkpoint, never random weights.
- Use `stream=True` for video/large-batch inference to avoid loading all results into memory.
- `conf` and `iou` are tunable at inference time without retraining.

Practice Tasks

- **Level 1:** Run inference with a pretrained YOLO model on a sample image and print the detected class names and confidences.
- **Level 2:** Modify the inference call to only keep detections above `conf=0.5` for a specific class of interest.
- **Level 3:** Implement a script that runs YOLO on a folder of images and saves annotated outputs for all of them.
- **Level 4:** Debug a video-inference script that runs out of memory — confirm `stream=True` is set and results are processed one frame at a time.
- **Level 5:** Compare inference latency across the nano/small/medium model size variants on the same hardware.
- **Level 6:** Wrap YOLO inference in a function with full error handling for corrupted or missing image inputs.

Chapter 6.15 — YOLO Training

6.15.1 The Custom Training Workflow

```
Collect Images
      |
      v
Annotate
      |
      v
Create Dataset
      |
      v
Train/Val Split
      |
      v
Dataset YAML/Configuration
      |
      v
Train
      |
      v
Validate
      |
      v
Evaluate
      |
      v
Inference
      |
      v
Export
```

6.15.2 Dataset Directory Structure

```
my_dataset/
├── images/
│   ├── train/
│   │   ├── img001.jpg
```

```
│   │   └── img002.jpg
│   └── val/
│       ├── img101.jpg
│       └── img102.jpg
└── labels/
    ├── train/
    │   ├── img001.txt
    │   └── img002.txt
    └── val/
        ├── img101.txt
        └── img102.txt
```

Each image has a matching `.txt` label file of the same name, one line per object:

```
<class_id> <x_center> <y_center> <width> <height>
```

All four numeric values are normalized to `[0, 1]` relative to image width/height (i.e. `cxcywh` format, normalized). Class IDs are zero-indexed integers matching the order defined in the dataset YAML.

6.15.3 Dataset YAML Configuration

```yaml
# my_dataset.yaml
path: /data/my_dataset
train: images/train
val: images/val

names:
  0: cat
  1: dog
  2: bird
```

6.15.4 Training

```python
from ultralytics import YOLO

model = YOLO("yolo11n.pt")   # start from pretrained weights

results = model.train(
    data="my_dataset.yaml",
    epochs=100,
    imgsz=640,
    batch=16,
    lr0=0.01,              # initial learning rate
    patience=20,           # early stopping: stop if no val improvement for this many epochs
    device=0,              # GPU index, or "cpu"
    project="runs/detect",
    name="my_experiment",
)
```

Hyperparameter	Meaning	Typical starting value
`epochs`	Training passes over the full dataset	100-300 for small custom datasets
`batch`	Images per gradient step	16-64, limited by GPU memory
`imgsz`	Training/inference resolution	640 (default); smaller for speed, larger for small-object accuracy
`lr0`	Initial learning rate	0.01 (SGD default), 0.001-ish for Adam
`patience`	Early-stopping epochs	20-50

6.15.5 Validation and Evaluation

```
metrics = model.val()
print(f"mAP50-95: {metrics.box.map:.3f}")
print(f"mAP50:    {metrics.box.map50:.3f}")
print(f"Precision: {metrics.box.mp:.3f}")
print(f"Recall:    {metrics.box.mr:.3f}")
```

6.15.6 Inference and Threshold Tuning

```
results = model.predict(
    source="test_images/",
    conf=0.25,
    iou=0.45,
    save=True,
)
```

• Common Mistake

Class IDs in label files that don't zero-index consistently, or that don't match the order in the dataset YAML's `names` dictionary. This produces a model that trains without error but confidently predicts the wrong class names — one of the more insidious detection bugs because loss curves look completely normal.

• Best Practice

Before starting a long training run, visualize a handful of annotated images (draw the label boxes back onto the images and inspect them) to confirm the annotation format, normalization, and class IDs are all correct. Catching an annotation bug after a 4-hour training run is far more expensive than catching it in 30 seconds beforehand.

★ Quick Revision Sheet

YOLO Training, in one screen:

- Directory structure: parallel `images/` and `labels/` trees, split into `train`/`val`.
- Label format: one line per object, `class_id cx cy w h`, all normalized to `[0,1]`.
- Dataset YAML defines paths and the `names` class-id-to-label mapping.
- Always start from pretrained weights; tune `epochs`, `batch`, `imgsz`, `lr0`, `patience`.
- Visualize annotations before a long training run — annotation bugs don't show up in the loss curve.

Practice Tasks

- **Level 1:** Write a valid YOLO label file for an image containing two objects of different classes.
- **Level 2:** Modify the training call to use a larger input resolution (`imgsz=1280`) for small-object detection.
- **Level 3:** Implement an annotation-visualization script that draws label boxes back onto training images for a manual sanity check.
- **Level 4:** Debug a training run where mAP stays near zero — check class ID consistency between labels and YAML.
- **Level 5:** Compare training time and final mAP between `imgsz=416` and `imgsz=640` on the same dataset.
- **Level 6:** Set up a full training run with early stopping and confirm the best checkpoint (not the last epoch) is what gets exported.

Chapter 6.16 — YOLO Custom Dataset Project

Project Specification

Project name: Personal Protective Equipment (PPE) Detector

Problem statement: Detect hard hats, safety vests, and their absence on people in warehouse/construction camera footage, to support automated safety-compliance monitoring.

Real-world use case: Manufacturing and construction site safety monitoring — flagging frames where a worker is not wearing required PPE for human review.

Task: Multi-class object detection.

Input: RGB images (from fixed CCTV-style cameras or handheld photos).

Output: Bounding boxes with class labels (`person`, `hardhat`, `safety_vest`, `no_hardhat`, `no_vest`) and confidence scores.

Architecture: YOLO11 (nano or small variant), fine-tuned from COCO-pretrained weights.

Why this architecture: Real-time inference requirement (live camera feeds), moderate object count per frame, and a dataset too small to train from scratch competitively — transfer learning from a pretrained YOLO checkpoint is the clear right default.

Alternatives considered: Faster R-CNN (more accurate but too slow for live monitoring at typical camera frame rates on edge hardware); SSD (comparable speed, generally lower accuracy on small objects like hard hats at a distance).

Difficulty: Intermediate.

Estimated time: 4-8 hours end-to-end (annotation reuse from an existing public dataset, several training runs, evaluation, and a basic inference script).

Portfolio value: High — combines a real-world safety use case, a full custom-dataset training workflow, and a deployment-ready inference script; a strong resume/interview talking point that demonstrates the complete pipeline rather than just "I fine-tuned a model on a tutorial dataset."

Dataset Specification

Field	Value
Dataset	Hard Hat Workers Dataset (or equivalent public PPE detection dataset)

Field	Value
Official source	Roboflow Universe
URL	https://universe.roboflow.com/
Task	Object detection
Samples	~5,000 images (varies by specific dataset version — verify count on the source page before committing to a training plan)
Classes	`person`, `head`, `helmet`, `no_helmet` (exact class set varies by dataset version — confirm against the downloaded `data.yaml`)
Input size	Variable; resized to 640x640 during training
Format	YOLO `.txt` labels (Roboflow can export directly in YOLO format)
Train split	~80%
Validation split	~15%
Test split	~5%
Class distribution	Typically imbalanced toward `helmet`/`person`; verify and note under-represented classes before training
License	Varies by specific dataset on Roboflow Universe — check the license on the dataset's page before any commercial use
Download method	Roboflow export (YOLO format) via their web UI or API
Storage	~500MB-2GB depending on image resolution and count
Preprocessing	Resize to 640x640, normalize per YOLO defaults (handled internally by the training framework)
Augmentation	Default YOLO augmentation (mosaic, HSV jitter, flip) — appropriate here since PPE items don't have a fixed canonical orientation
Evaluation	mAP@0.5, mAP@0.5:0.95, per-class recall (recall is the priority metric — missing a safety violation is costlier than a false alarm)

- **Common Mistake**

Treating a publicly-listed dataset's class names and counts as fixed facts without checking the actual downloaded `data.yaml`. Public dataset versions on aggregator sites change over time — always verify class list, counts, and license directly from the downloaded files before writing them into a report or committing to a training plan.

Environment Specification

Field	Value
OS	Ubuntu 22.04 / 24.04 (or equivalent Linux; Windows/macOS also supported by the framework)
Python version	3.10 or 3.11
Framework	PyTorch (via the `ultralytics` package)

Field	Value
Library versions	`ultralytics>=8.3`, `torch>=2.1` — pin exact versions in `requirements.txt` at project start and record what was actually used
CPU	Any modern multi-core CPU for data loading; training itself is GPU-bound
RAM	16GB recommended
GPU	NVIDIA GPU with 6GB+ VRAM recommended for `imgsz=640`, `batch=16` on the nano/small model
VRAM	6-8GB comfortable for nano/small variants at batch 16; reduce batch size on smaller GPUs
Storage	5-10GB free (dataset + checkpoints + logs)
CUDA requirements	CUDA 11.8+ or 12.x matching the installed PyTorch build
Estimated training time	1-3 hours for 100 epochs on a single mid-range GPU (e.g. RTX 3060-class), dataset-size dependent
CPU-only feasibility	Feasible for inference; training on CPU alone is possible but substantially slower (potentially 10-20x) — acceptable only for small-scale experimentation

Environment Check

```python
# environment_check.py
import sys
import subprocess
import importlib

def check_python():
    v = sys.version_info
    ok = v.major == 3 and v.minor in (10, 11)
    print(f"Python {v.major}.{v.minor}.{v.micro} — {'OK' if ok else 'WARNING: expected 3.10 or 3.11'}")

def check_package(name, min_version=None):
    try:
        mod = importlib.import_module(name)
        version = getattr(mod, "__version__", "unknown")
        print(f"{name} {version} — installed")
    except ImportError:
        print(f"{name} — NOT INSTALLED")

def check_gpu():
    try:
        import torch
        if torch.cuda.is_available():
            print(f"GPU: {torch.cuda.get_device_name(0)}, "
                  f"VRAM: {torch.cuda.get_device_properties(0).total_memory / 1e9:.1f} GB")
            print(f"CUDA version: {torch.version.cuda}")
        else:
```

```
            print("No GPU detected — training will run on CPU (slow)")
    except ImportError:
        print("PyTorch not installed — cannot check GPU")

def check_dataset(path="my_dataset"):
    import os
    required = ["images/train", "images/val", "labels/train", "labels/val"]
    for r in required:
        full = os.path.join(path, r)
        exists = os.path.isdir(full)
        count = len(os.listdir(full)) if exists else 0
        print(f"{full}: {'OK' if exists else 'MISSING'} ({count} files)")

def check_storage(min_gb=5):
    import shutil
    free_gb = shutil.disk_usage(".").free / 1e9
    print(f"Free disk space: {free_gb:.1f} GB — {'OK' if free_gb >= min_gb else 'WARNING: low'}")

if __name__ == "__main__":
    check_python()
    check_package("torch")
    check_package("ultralytics")
    check_gpu()
    check_dataset()
    check_storage()
```

Requirements

```
# requirements.txt
ultralytics>=8.3.0
torch>=2.1.0
opencv-python>=4.9.0
numpy>=1.26.0
pyyaml>=6.0
```

Folder Structure

```
ppe_detector/
├── requirements.txt
├── environment_check.py
├── data.yaml
├── dataset/
│   ├── images/{train,val,test}/
│   └── labels/{train,val,test}/
├── scripts/
│   ├── visualize_annotations.py
│   ├── train.py
```

```
│   ├── evaluate.py
│   └── infer.py
├── runs/                # created automatically by training
└── deployment/
    ├── inference_api.py
    └── Dockerfile
```

Dataset Download

```
# Via Roboflow's Python SDK (requires a free API key from roboflow.com)
pip install roboflow --break-system-packages
python3 -c "
from roboflow import Roboflow
rf = Roboflow(api_key='YOUR_API_KEY')
project = rf.workspace('WORKSPACE_NAME').project('PROJECT_NAME')
dataset = project.version(1).download('yolov8')
"
```

Annotation Format

Already covered in Chapter 6.15: one `.txt` file per image, one line per object, `class_id cx cy w h`, all normalized to `[0,1]`.

Dataset Validation

```
# scripts/visualize_annotations.py
import cv2
import os
import yaml

def visualize(image_dir, label_dir, data_yaml_path, num_samples=10):
    with open(data_yaml_path) as f:
        class_names = yaml.safe_load(f)["names"]

    images = sorted(os.listdir(image_dir))[:num_samples]
    for fname in images:
        img_path = os.path.join(image_dir, fname)
        label_path = os.path.join(label_dir, os.path.splitext(fname)[0] + ".txt")
        img = cv2.imread(img_path)
        if img is None:
            print(f"WARNING: failed to load {img_path}")
            continue
        h, w = img.shape[:2]

        if not os.path.exists(label_path):
            print(f"WARNING: no label file for {fname}")
            continue
```

```
        with open(label_path) as f:
            for line in f:
                cls_id, cx, cy, bw, bh = map(float, line.split())
                x1 = int((cx - bw / 2) * w)
                y1 = int((cy - bh / 2) * h)
                x2 = int((cx + bw / 2) * w)
                y2 = int((cy + bh / 2) * h)
                cv2.rectangle(img, (x1, y1), (x2, y2), (0, 255, 0), 2)
                cv2.putText(img, class_names[int(cls_id)], (x1, y1 - 5),
                            cv2.FONT_HERSHEY_SIMPLEX, 0.5, (0, 255, 0), 1)

        out_path = f"annotated_{fname}"
        cv2.imwrite(out_path, img)
        print(f"Saved {out_path}")

if __name__ == "__main__":
    visualize("dataset/images/train", "dataset/labels/train", "data.yaml")
```

Training

```
# scripts/train.py
from ultralytics import YOLO

model = YOLO("yolo11n.pt")
model.train(
    data="data.yaml",
    epochs=100,
    imgsz=640,
    batch=16,
    patience=20,
    device=0,
    project="runs/detect",
    name="ppe_detector_v1",
)
```

Evaluation

```
# scripts/evaluate.py
from ultralytics import YOLO

model = YOLO("runs/detect/ppe_detector_v1/weights/best.pt")
metrics = model.val(data="data.yaml", split="test")
print(f"mAP50-95: {metrics.box.map:.3f}")
print(f"mAP50:    {metrics.box.map50:.3f}")
print(f"Per-class recall: {metrics.box.r}")
```

Inference

```
# scripts/infer.py
from ultralytics import YOLO

model = YOLO("runs/detect/ppe_detector_v1/weights/best.pt")
results = model.predict(source="dataset/images/test", conf=0.4, save=True)
```

Error Analysis

Industry Case Study: PPE Detector Failure Modes

Common failure modes for a first-pass PPE detector: (1) helmets confused with hair color/hats in low light — usually fixed by adding more low-light training examples rather than architecture changes; (2) partial occlusion (a worker turned away from camera) causing missed `no_helmet` detections — recall-oriented threshold tuning and a wider camera angle in data collection both help; (3) small, distant workers below the model's effective detection resolution — increasing `imgsz` at inference time (with a matching latency cost) or adding a tiling/crop-based inference pass for distant regions of the frame are the standard fixes.

Steps for systematic error analysis:

```
# Inspect false negatives specifically for the safety-critical classes
import numpy as np

def find_missed_detections(model, image_dir, label_dir, class_of_interest_id, conf=0.25):
    """Flag images where a ground-truth box for the class of interest has no matching prediction."""
    missed = []
    for fname in os.listdir(image_dir):
        label_path = os.path.join(label_dir, os.path.splitext(fname)[0] + ".txt")
        if not os.path.exists(label_path):
            continue
        gt_boxes = [line.split() for line in open(label_path) if int(line.split()[0]) == class_of_interest_id]
        if not gt_boxes:
            continue
        results = model.predict(os.path.join(image_dir, fname), conf=conf, verbose=False)
        pred_classes = results[0].boxes.cls.tolist()
        if class_of_interest_id not in pred_classes:
            missed.append(fname)
    return missed
```

Model Export

```
model.export(format="onnx", imgsz=640, dynamic=False)      # for ONNX Runtime deployment
model.export(format="engine", imgsz=640)                    # TensorRT, for NVIDIA edge/server deployment
```

Deployment Options

Target	Approach
Server (GPU)	ONNX Runtime GPU or TensorRT behind a FastAPI endpoint
Server (CPU)	ONNX Runtime CPU; reduce `imgsz` if latency-constrained
Edge device (e.g. Jetson)	TensorRT export, optimized for the specific device
Live camera pipeline	`stream=True` inference loop reading directly from RTSP/webcam input

Improvement Tasks

- Collect additional low-light and occluded-worker examples to address the failure modes identified above.
- Add temporal smoothing across video frames (e.g. require 3 consecutive frames of a violation before alerting) to reduce false-positive alert spam.
- Fine-tune confidence thresholds per class, since a missed `no_helmet` detection and a missed `person` detection have different real-world costs.

Interview Questions

IQ1. Why fine-tune from a COCO-pretrained checkpoint instead of training from scratch on the PPE dataset alone? **DIFFICULTY: EASY** *Companies: General* The pretrained backbone already encodes general-purpose visual features (edges, textures, common object parts) learned from a much larger and more diverse dataset than a typical custom PPE dataset. Fine-tuning converges faster and reaches higher accuracy with far less labeled data than training from random initialization.

IQ2. Why prioritize recall over precision for the safety-violation classes in this project? **DIFFICULTY: MEDIUM** *Companies: General* A missed safety violation (false negative) can lead to a real workplace injury going unflagged, while a false positive only costs a few seconds of unnecessary human review. The asymmetric real-world cost of the two error types justifies tuning the confidence threshold to favor recall, even at some precision cost.

★ Quick Revision Sheet

PPE Detector Project, in one screen:

- Full pipeline: dataset download -> annotation validation -> training -> evaluation -> error analysis -> export -> deployment.
- Always visualize a sample of annotations before committing to a long training run.
- Recall is the priority metric here — missing a safety violation is costlier than a false alarm.
- Export to ONNX/TensorRT depending on the deployment target's hardware.

Chapter 6.17 — YOLO Deployment & Optimization

6.17.1 ONNX Export

```
from ultralytics import YOLO

model = YOLO("best.pt")
model.export(format="onnx", imgsz=640, dynamic=False, simplify=True)
```

`dynamic=False` locks the input shape, which is faster and simpler for most deployment targets. Use `dynamic=True` only if inference-time image size genuinely varies and you can't standardize it before the model call.

6.17.2 Inference Optimization Levers

Lever	Effect	Tradeoff
Smaller `imgsz`	Faster inference	Reduced accuracy, especially on small objects
Smaller model variant (n vs. s vs. m)	Faster inference, smaller memory footprint	Reduced accuracy ceiling
Batch inference	Higher throughput	Higher latency per individual result, more memory
FP16 / half precision	~2x speedup on supporting GPUs	Negligible accuracy loss on most models
INT8 quantization	~2-4x speedup, smaller model	Some accuracy loss; needs calibration data
Higher confidence threshold	Fewer boxes to run NMS over	Risk of missing valid low-confidence detections

```
# Batch inference — higher throughput for offline processing
results = model.predict(source="images/", batch=32, imgsz=640)

# FP16 inference on GPU
results = model.predict(source="image.jpg", half=True, device=0)
```

6.17.3 Image-Size Tradeoffs

Reducing `imgsz` from 640 to 416 roughly halves compute cost, but disproportionately hurts detection of small, distant objects, since fewer pixels remain to represent them. For applications dominated by small objects (aerial imagery, distant security cameras), it's often better to keep `imgsz` high and instead choose a smaller model variant to control latency.

6.17.4 GPU vs. CPU Inference

```
# GPU
results = model.predict(source="image.jpg", device=0)

# CPU (e.g. for edge deployment without a GPU)
results = model.predict(source="image.jpg", device="cpu")
```

Target	Typical choice
Cloud server, high throughput	GPU with batched inference, FP16 or TensorRT
Cloud server, cost-sensitive, low volume	CPU with ONNX Runtime, smaller model variant
Desktop application	CPU or consumer GPU, ONNX Runtime
Mobile	TFLite / Core ML export, INT8 quantization
Edge device (Jetson, Coral)	TensorRT / Edge TPU-specific export, INT8

6.17.5 Deployment Targets in Practice

```
Desktop:  ONNX Runtime + a local GUI or CLI tool
Server:   FastAPI + ONNX Runtime GPU or TensorRT, behind a load balancer
Mobile:   TFLite (Android) / Core ML (iOS), quantized
Edge:     TensorRT (NVIDIA Jetson) or platform-specific compiler (Coral Edge TPU)
```

• Best Practice

Benchmark end-to-end latency (preprocessing + inference + postprocessing/NMS), not just raw model forward-pass time. NMS cost scales with the number of raw candidate boxes and can become a meaningful fraction of total latency in cluttered scenes, and preprocessing (especially resize + letterbox) is easy to overlook when reporting "inference speed."

★ Quick Revision Sheet

YOLO Deployment & Optimization, in one screen:

- Export to ONNX/TensorRT/TFLite depending on target hardware.
- Levers: image size, model variant, batching, FP16/INT8 — each trades some accuracy for speed.
- Small objects suffer disproportionately from reduced `imgsz` — prefer a smaller model over smaller images when small-object accuracy matters.
- Always benchmark full end-to-end latency, including preprocessing and NMS, not just the model forward pass.

Practice Tasks

- **Level 1:** Export a trained YOLO model to ONNX and confirm it loads correctly with ONNX Runtime.

- **Level 2:** Benchmark FP32 vs. FP16 inference latency on the same GPU.
- **Level 3:** Implement a batched offline inference script that processes a folder of 1,000 images and reports total throughput.
- **Level 4:** Debug a deployment where ONNX inference results don't match the original PyTorch model — check export settings (dynamic shapes, opset version).
- **Level 5:** Compare end-to-end latency (including preprocessing and NMS) across `imgsz` 416/640/1280.
- **Level 6:** Set up an INT8-quantized TensorRT export and validate accuracy parity against the FP32 baseline on a held-out test set.

Chapter 6.18 — SSD (Single Shot Detector)

6.18.1 Architecture

SSD is a one-stage detector, like YOLO, but generates predictions from multiple feature map layers at different resolutions directly within the backbone's forward pass, rather than through a separate fusion "neck." Earlier, higher-resolution feature maps detect small objects; later, lower-resolution feature maps (with larger receptive fields) detect large objects.

```
Input Image
     |
     v
Backbone (e.g. VGG16, truncated)
     |
     +---> Feature map 1 (high-res)  -> Detection head -> small-object boxes
     |
     +---> Feature map 2 (mid-res)   -> Detection head -> medium-object boxes
     |
     +---> Feature map 3 (low-res)   -> Detection head -> large-object boxes
```

6.18.2 Anchor Boxes and Multiscale Prediction

At every spatial location on every selected feature map, SSD tiles a fixed set of anchor boxes at multiple aspect ratios and scales, and the detection head predicts a class score and box offset for each anchor. Because anchors are tiled at multiple feature-map resolutions, SSD naturally covers a wide range of object sizes without needing an explicit multi-scale training trick.

6.18.3 torchvision Implementation

```
import torchvision.models.detection as detection

model = detection.ssd300_vgg16(weights=detection.SSD300_VGG16_Weights.COCO_V1)
model.eval()

import torch
from torchvision.transforms.functional import to_tensor
from PIL import Image

img = to_tensor(Image.open("image.jpg").convert("RGB"))
```

```
with torch.no_grad():
    predictions = model([img])   # torchvision detection models take a list of tensors

boxes = predictions[0]["boxes"]      # (N, 4) xyxy, absolute pixel coordinates
labels = predictions[0]["labels"]    # (N,)
scores = predictions[0]["scores"]    # (N,)
```

6.18.4 Strengths and Limitations

Strengths	Limitations
Single forward pass, real-time capable	Generally lower accuracy than modern YOLO on small objects
Simple, well-understood architecture	Anchor design is sensitive and needs tuning per dataset
Good balance of speed and accuracy for its era	Largely superseded by modern YOLO variants in most practical use cases today

6.18.5 SSD vs. YOLO

	SSD	Modern YOLO
Detection approach	Multi-scale feature maps, anchor-based	Multi-scale feature pyramid, often anchor-free
Small-object accuracy	Weaker	Stronger (better multi-scale fusion via the neck)
Ecosystem/tooling	Mature but less actively developed	Actively developed, extensive tooling (`ultralytics`)
Typical choice today	Legacy systems, teaching, specific embedded toolchains that already support it	Default choice for new detection projects

• Interview Trap

”Why would you ever choose SSD over YOLO today?” A fair answer isn’t ”you wouldn’t” — it’s about existing deployment constraints: some embedded/legacy toolchains (certain older mobile inference SDKs, some fixed-function hardware accelerators) have mature, well-tested SSD support and immature or nonexistent support for the latest YOLO architectures. In a genuinely constrained deployment environment, that operational reality can outweigh a modest accuracy difference.

★ Quick Revision Sheet

SSD, in one screen:

- One-stage detector predicting directly from multiple backbone feature-map resolutions.
- Anchor-based, multiple aspect ratios/scales tiled per feature map location.
- Largely superseded by modern YOLO for new projects; still relevant for specific legacy/embedded toolchains.

Practice Tasks

- **Level 1:** Explain why SSD predicts from multiple feature map resolutions instead of just the final layer.
- **Level 2:** Run `ssd300_vgg16` on a sample image and visualize the predicted boxes.
- **Level 3:** Compare SSD's raw prediction count (before NMS) to YOLO's for the same image.
- **Level 4:** Debug an SSD model performing poorly on small objects — check which feature map resolutions are actually being used for detection heads.
- **Level 5:** Benchmark SSD vs. a comparable-size YOLO model on the same hardware.
- **Level 6:** Export an SSD model to ONNX and confirm output parity with the PyTorch model.

Chapter 6.19 — Faster R-CNN

6.19.1 Two-Stage Detection

Faster R-CNN splits detection into two stages: first propose candidate object regions, then classify and refine each one. This division of labor — "where might an object be" followed by "what is it, exactly" — is what separates two-stage from one-stage detectors like YOLO/SSD.

```
Image
 |
 v
Backbone (feature extraction)
 |
 v
Region Proposal Network (RPN) --> candidate object regions ("proposals")
 |
 v
ROI Pooling/Align --> fixed-size feature per proposal
 |
 v
Classification Head --> class probabilities
 |
 v
Bounding-Box Regression Head --> refined box coordinates
```

6.19.2 Region Proposal Network (RPN)

The RPN slides a small network over the backbone's feature map and, at each location, predicts an "objectness" score and a box refinement for a set of anchor boxes — producing a ranked list of a few hundred to a couple thousand candidate regions likely to contain an object, regardless of class. This replaces the much slower, non-learned region-proposal methods (like Selective Search) used in the original R-CNN and Fast R-CNN.

6.19.3 ROI Pooling / ROI Align

Each proposed region has a different size and shape; the classification and regression heads need a fixed-size input. ROI Pooling crops and resizes each proposal's features to a fixed grid via coarse quantization; ROI Align (used in Mask R-CNN and modern Faster R-CNN implementations) avoids that quantization using bilinear interpolation, producing more precisely aligned features — important for accurate localization and essential for pixel-precise instance segmentation.

6.19.4 Why Faster R-CNN Can Be More Accurate but Slower

The two-stage design lets the classification head focus entirely on a small, curated set of promising regions rather than densely evaluating the whole image, and the RPN's learned proposals are typically higher-quality than a one-stage detector's dense anchor grid — both factors that tend to improve accuracy, especially on small or crowded objects. The cost is architectural: two networks must run sequentially (RPN, then per-region classification/regression) instead of one shared forward pass, and a variable, potentially large number of ROI-level operations must run per image, meaning inference time scales with the number of proposals in a way a one-stage detector's fixed-cost forward pass does not.

6.19.5 torchvision Implementation

```
import torchvision.models.detection as detection
import torch

model = detection.fasterrcnn_resnet50_fpn_v2(
    weights=detection.FasterRCNN_ResNet50_FPN_V2_Weights.COCO_V1
)
model.eval()

from torchvision.transforms.functional import to_tensor
from PIL import Image

img = to_tensor(Image.open("image.jpg").convert("RGB"))
with torch.no_grad():
    predictions = model([img])

boxes = predictions[0]["boxes"]
labels = predictions[0]["labels"]
scores = predictions[0]["scores"]
```

Fine-tuning on a custom dataset (replacing the classifier head):

```
from torchvision.models.detection.faster_rcnn import FastRCNNPredictor

num_classes = 5 + 1   # +1 for background class, required by torchvision's Faster R-CNN
model = detection.fasterrcnn_resnet50_fpn_v2(weights="DEFAULT")
in_features = model.roi_heads.box_predictor.cls_score.in_features
model.roi_heads.box_predictor = FastRCNNPredictor(in_features, num_classes)
```

- **Common Mistake**

Forgetting that torchvision's Faster R-CNN implementation requires a background class in addition to your actual object classes — `num_classes` must be your class count plus one. Omitting the background class produces a shape mismatch error deep inside the ROI head that is confusing to trace back to its actual cause.

★ Quick Revision Sheet

Faster R-CNN, in one screen:

- Two-stage: Region Proposal Network generates candidates, then a head classifies + refines each one.
- ROI Align > ROI Pooling for precise localization (no quantization).
- More accurate on small/crowded objects than typical one-stage detectors, at a real latency cost.
- torchvision's implementation needs `num_classes` = actual classes + 1 (background).

Practice Tasks

- **Level 1:** Explain the difference between the RPN's "objectness" score and the classification head's class score.
- **Level 2:** Fine-tune a Faster R-CNN model's classifier head on a small custom dataset with 3 object classes.
- **Level 3:** Implement an inference script that filters Faster R-CNN's raw output by a confidence threshold and visualizes the results.
- **Level 4:** Debug a shape-mismatch error during fine-tuning — check whether the background class was correctly accounted for.
- **Level 5:** Benchmark Faster R-CNN vs. YOLO on the same dataset for both mAP and latency.
- **Level 6:** Export a fine-tuned Faster R-CNN model to ONNX and evaluate export compatibility issues (dynamic proposal counts are a common obstacle).

Chapter 6.20 — RetinaNet

6.20.1 The Class Imbalance Problem in One-Stage Detectors

Dense one-stage detectors (SSD, and RetinaNet itself) evaluate tens of thousands of candidate anchor locations per image, the vast majority of which are background. Standard cross-entropy loss, applied uniformly, lets this huge volume of easy background examples dominate the total loss and gradient signal, drowning out the comparatively rare, harder foreground examples that actually matter for learning. This is the central problem RetinaNet's authors identified as the reason one-stage detectors historically trailed two-stage detectors in accuracy.

6.20.2 Focal Loss

Focal loss modifies cross-entropy to down-weight well-classified (easy) examples, letting training focus on hard, misclassified examples:

```
FL(p_t) = -alpha * (1 - p_t)^gamma * log(p_t)
```

Where `p_t` is the model's predicted probability for the correct class. When `p_t` is high (an easy, correctly-classified example), `(1 - p_t)^gamma` shrinks toward zero, down-weighting its contribution to the loss. `gamma` (typically 2) controls how aggressively easy examples are down-weighted; `alpha` balances the overall foreground/background class weighting.

```
import torch
import torch.nn.functional as F

def focal_loss(logits, targets, alpha=0.25, gamma=2.0):
    bce = F.binary_cross_entropy_with_logits(logits, targets, reduction="none")
    p_t = torch.exp(-bce)                       # recovers p_t from the BCE value
    loss = alpha * (1 - p_t) ** gamma * bce
    return loss.mean()
```

6.20.3 Feature Pyramid Networks (FPN)

RetinaNet builds on a Feature Pyramid Network: a top-down pathway with lateral connections that combines high-resolution, low-semantic features from early backbone layers with low-resolution, high-semantic features from late layers, producing a pyramid of feature maps that are all both spatially precise and semantically rich. Detection heads run on every pyramid level, giving good multi-scale coverage.

```
Backbone (bottom-up)          FPN (top-down + lateral)
   C5 (low-res, rich) --------> P5
                                 |  (upsample + lateral connection from C4)
```

```
   C4 ----------------------> P4
                               |
   C3 ----------------------> P3
```

6.20.4 Classification and Regression Subnets

At every FPN level, two small fully-convolutional subnets run in parallel: a classification subnet (predicts per-anchor class probability) and a box-regression subnet (predicts per-anchor box offsets). Both subnets share the same architecture across all pyramid levels but not their weights across the two subnet types.

6.20.5 Implementation

```
import torchvision.models.detection as detection

model = detection.retinanet_resnet50_fpn_v2(
    weights=detection.RetinaNet_ResNet50_FPN_V2_Weights.COCO_V1
)
model.eval()

# Fine-tuning: replace the classification head for a custom number of classes
from torchvision.models.detection.retinanet import RetinaNetClassificationHead

num_classes = 5   # RetinaNet's torchvision implementation does NOT need a +1 background class
in_channels = model.head.classification_head.conv[0][0].in_channels
num_anchors = model.head.classification_head.num_anchors
model.head.classification_head = RetinaNetClassificationHead(
    in_channels, num_anchors, num_classes
)
```

6.20.6 Comparison: RetinaNet vs. YOLO vs. SSD vs. Faster R-CNN

	RetinaNet	YOLO	SSD	Faster R-CNN
Stages	One	One	One	Two
Key innovation	Focal loss + FPN	Real-time single-pass, multi-scale head	Multi-scale anchors from backbone layers	RPN + ROI Align
Small-object accuracy	Strong (via FPN)	Strong (modern versions)	Weaker	Strong
Speed	Medium	Fast	Fast	Slower
Typical use today	Accuracy-focused one-stage detection, dense/small-object scenes	Real-time detection, the default general choice	Legacy/embedded toolchains	Offline high-accuracy pipelines

• Best Practice

Reach for RetinaNet specifically when the practical problem has severe foreground/background imbalance and small objects — e.g. detecting small defects across a large industrial image, or dense small-object aerial imagery — where focal loss's core benefit (down-weighting the flood of easy background) directly addresses the failure mode you'd otherwise see with a standard-loss one-stage detector.

★ Quick Revision Sheet

RetinaNet, in one screen:

- Focal loss down-weights easy (well-classified) examples, letting hard examples dominate the gradient signal.
- Built on a Feature Pyramid Network — combines spatial precision (early layers) with semantic richness (late layers).
- Parallel classification and box-regression subnets run at every pyramid level.
- Best fit: severe class imbalance + small-object-heavy one-stage detection tasks.

Practice Tasks

- **Level 1:** Explain in one sentence what `gamma` controls in focal loss.
- **Level 2:** Implement focal loss from scratch and confirm it produces a smaller loss for an easy (high-confidence-correct) example than standard BCE.
- **Level 3:** Fine-tune RetinaNet's classification head on a custom small-object dataset.
- **Level 4:** Debug a RetinaNet training run where loss stays flat — check the `alpha`/`gamma` values and anchor configuration.
- **Level 5:** Compare RetinaNet vs. Faster R-CNN mAP and latency on a dataset with many small, dense objects.
- **Level 6:** Package a full RetinaNet fine-tuning + evaluation workflow for a custom small-object detection dataset.

Chapter 6.21 — Object Detector Comparison

6.21.1 Comparison Table

Model	Type	Speed	Accuracy	Complexity	Small Objects	Training Difficulty	Deployment
YOLO (modern)	One-stage	Fastest	High	Low-medium	Strong (multi-scale head)	Easy (mature tooling)	Excellent (ONNX/TensorRT/TFLite well-supported)
SSD	One-stage	Fast	Medium	Low	Weak-medium	Easy	Good, but tooling less actively maintained
RetinaNet	One-stage	Medium	High	Medium	Strong (FPN)	Medium	Good
Faster R-CNN	Two-stage	Slow	Highest (typically)	High	Strong	Medium-hard	Harder (variable proposal count complicates export)

6.21.2 Which Detector Should I Choose?

- **Best Practice**

Real-time video, live camera feeds, edge/mobile deployment: YOLO. It is fast, has the most mature deployment tooling (ONNX/TensorRT/TFLite/Core ML export all well-supported), and modern versions have closed most of the historical accuracy gap with two-stage detectors.

• Best Practice

Offline batch processing where accuracy matters more than latency (e.g. auditing a fixed archive of satellite images, medical imaging pipelines with no real-time constraint): Faster R-CNN, or RetinaNet if small objects and class imbalance are dominant challenges.

• Best Practice

Severe foreground/background imbalance with small, dense objects (e.g. small defect detection across large industrial images, dense aerial object counting): RetinaNet — focal loss directly targets this exact failure mode.

• Best Practice

Legacy or fixed-function embedded hardware with existing, mature SSD support and no support for newer architectures: SSD, as a pragmatic constraint-driven choice rather than a first-choice recommendation.

6.21.3 Decision Checklist

```
Is real-time / low-latency inference required?
   YES -> Is a mature deployment toolchain (ONNX/TensorRT/mobile export) needed?
       YES -> YOLO
       NO (fixed legacy hardware only supports SSD) -> SSD
   NO -> Is the dataset dominated by small objects / severe class imbalance?
       YES -> RetinaNet
       NO -> Is maximum achievable accuracy the only priority?
           YES -> Faster R-CNN
           NO -> YOLO (still a strong, simpler default)
```

• Interview Trap

A system-design interview question like ”design an object detection system for X” is testing whether you ask about latency budget, deployment hardware, object size distribution, and accuracy requirements *before* naming a model. Naming ”YOLO” immediately, without first surfacing those constraints, signals pattern-matching rather than engineering judgment — even when YOLO ends up being the right answer.

★ Quick Revision Sheet

Detector Comparison, in one screen:

- YOLO: default choice for real-time and edge/mobile deployment, mature tooling.
- Faster R-CNN: highest typical accuracy, offline/batch use cases, harder to deploy.
- RetinaNet: best for severe class imbalance + small/dense objects.
- SSD: legacy/embedded-constrained scenarios only, for new projects.
- Always establish latency budget, deployment target, and object-size distribution before recommending a model.

Practice Tasks

- **Level 1:** For a phone-app real-time object counter, justify a detector choice using the decision checklist.
- **Level 2:** For an offline satellite-imagery audit with no latency constraint, justify a different choice.
- **Level 3:** Benchmark YOLO, RetinaNet, and Faster R-CNN on the same small custom dataset and compare mAP.
- **Level 4:** Given a scenario description with conflicting requirements (real-time AND maximum accuracy), iden-

tify the actual tradeoff and propose a resolution (e.g. a smaller high-accuracy model, or a two-tier detect-then-verify pipeline).

- **Level 5:** Measure export/deployment friction (time and issues encountered) across all four detector types for the same target format (ONNX).
- **Level 6:** Write a one-page model-selection memo for a hypothetical stakeholder, justifying a detector choice against stated latency and accuracy requirements.

Chapter 6.22 — Semantic Segmentation

6.22.1 Pixel-Level Classification

Where classification labels an entire image and detection labels bounding boxes, segmentation labels every individual pixel with a class. The output is a mask — an array the same spatial size as the input image, where each pixel value is a class ID.

```
Input image:  (3, H, W)
Output mask:  (H, W), each value in {0, 1, ..., num_classes - 1}
```

6.22.2 Mask Encoding

```
import numpy as np
import cv2

mask = cv2.imread("mask.png", cv2.IMREAD_GRAYSCALE)   # each pixel is a class ID: 0, 1, 2, ...
print(np.unique(mask))   # confirm the mask actually contains the expected class IDs, nothing else

# Visualize by mapping class IDs to colors
palette = np.array([[0,0,0],[255,0,0],[0,255,0],[0,0,255]], dtype=np.uint8)  # background + 3 classes
color_mask = palette[mask]   # (H, W, 3)
```

- **Common Mistake**

Saving a mask as a JPEG. JPEG's lossy compression blurs pixel values at edges, silently turning a clean integer label (e.g. `2`) into a nearby non-integer-equivalent value (e.g. `1` or `3` after re-quantization) right at object boundaries — exactly where segmentation accuracy matters most. Always save masks as a lossless format: PNG.

6.22.3 Architecture Families

Architecture	Core Idea
FCN (Fully Convolutional Network)	Replaces a classifier's dense layers with convolutions, producing a per-pixel output map directly
U-Net	Symmetric encoder-decoder with skip connections between matching-resolution encoder/decoder stages
DeepLab	Atrous (dilated) convolutions to expand receptive field without losing resolution, plus atrous spatial pyramid pooling for multi-scale context

6.22.4 Complete U-Net Implementation

```
import torch
import torch.nn as nn

class DoubleConv(nn.Module):
    def __init__(self, in_ch, out_ch):
        super().__init__()
        self.block = nn.Sequential(
            nn.Conv2d(in_ch, out_ch, 3, padding=1), nn.BatchNorm2d(out_ch), nn.ReLU(inplace=True),
            nn.Conv2d(out_ch, out_ch, 3, padding=1), nn.BatchNorm2d(out_ch), nn.ReLU(inplace=True),
        )
    def forward(self, x):
        return self.block(x)

class UNet(nn.Module):
    def __init__(self, in_channels=3, num_classes=1, base=64):
        super().__init__()
        self.enc1 = DoubleConv(in_channels, base)
        self.enc2 = DoubleConv(base, base * 2)
        self.enc3 = DoubleConv(base * 2, base * 4)
        self.enc4 = DoubleConv(base * 4, base * 8)
        self.pool = nn.MaxPool2d(2)

        self.bottleneck = DoubleConv(base * 8, base * 16)

        self.up4 = nn.ConvTranspose2d(base * 16, base * 8, 2, stride=2)
        self.dec4 = DoubleConv(base * 16, base * 8)   # base*16 because of concatenated skip connection
        self.up3 = nn.ConvTranspose2d(base * 8, base * 4, 2, stride=2)
        self.dec3 = DoubleConv(base * 8, base * 4)
        self.up2 = nn.ConvTranspose2d(base * 4, base * 2, 2, stride=2)
        self.dec2 = DoubleConv(base * 4, base * 2)
        self.up1 = nn.ConvTranspose2d(base * 2, base, 2, stride=2)
        self.dec1 = DoubleConv(base * 2, base)

        self.out_conv = nn.Conv2d(base, num_classes, kernel_size=1)

    def forward(self, x):
        e1 = self.enc1(x)                    # (N, base, H, W)
        e2 = self.enc2(self.pool(e1))         # (N, base*2, H/2, W/2)
        e3 = self.enc3(self.pool(e2))         # (N, base*4, H/4, W/4)
        e4 = self.enc4(self.pool(e3))         # (N, base*8, H/8, W/8)
        b = self.bottleneck(self.pool(e4))    # (N, base*16, H/16, W/16)

        d4 = self.up4(b)
        d4 = self.dec4(torch.cat([d4, e4], dim=1))   # skip connection: concat, not add
        d3 = self.up3(d4)
        d3 = self.dec3(torch.cat([d3, e3], dim=1))
        d2 = self.up2(d3)
```

```
        d2 = self.dec2(torch.cat([d2, e2], dim=1))
        d1 = self.up1(d2)
        d1 = self.dec1(torch.cat([d1, e1], dim=1))

        return self.out_conv(d1)   # (N, num_classes, H, W)

model = UNet(in_channels=3, num_classes=1)   # binary segmentation
dummy = torch.randn(2, 3, 256, 256)
out = model(dummy)
print(out.shape)  # torch.Size([2, 1, 256, 256])
```

★ Quick Revision Sheet

Semantic Segmentation, in one screen:

- Output is a per-pixel class-ID mask, same spatial size as the input.
- Always save masks as PNG (lossless), never JPEG.
- U-Net: encoder-decoder with concatenated skip connections between matching resolutions.
- DeepLab: dilated convolutions + multi-scale pooling for large receptive field without losing resolution.

Practice Tasks

- **Level 1:** Load a segmentation mask PNG and print the unique class IDs present.
- **Level 2:** Modify `UNet` to accept a variable number of encoder/decoder stages instead of a fixed 4.
- **Level 3:** Implement a training loop for `UNet` on a small binary segmentation dataset.
- **Level 4:** Debug a segmentation model producing blocky, low-resolution mask boundaries — check whether masks were resized with bilinear instead of nearest-neighbor interpolation.
- **Level 5:** Compare inference speed of `UNet` at different `base` channel widths.
- **Level 6:** Export `UNet` to ONNX and confirm mask output parity with the PyTorch model.

Chapter 6.23 — U-Net Deep Dive

6.23.1 Encoder, Decoder, and Skip Connections

The encoder progressively downsamples the input, extracting increasingly abstract, low-resolution features (much like a classification backbone). The decoder progressively upsamples back to the original resolution, recovering spatial detail. Skip connections concatenate each encoder stage's feature map directly into the matching-resolution decoder stage — without them, the decoder would have to reconstruct fine spatial detail (exact object edges) purely from the bottleneck's heavily compressed representation, which loses precisely that detail.

6.23.2 Upsampling: Transposed Convolution vs. Interpolation

```
# Learnable upsampling (used in the UNet implementation above)
up = nn.ConvTranspose2d(in_channels=128, out_channels=64, kernel_size=2, stride=2)

# Non-learnable upsampling (cheaper, sometimes preferred to avoid checkerboard artifacts)
up_interp = nn.Sequential(
    nn.Upsample(scale_factor=2, mode="bilinear", align_corners=False),
    nn.Conv2d(128, 64, kernel_size=1),
)
```

- **Common Mistake**

Transposed convolutions can produce checkerboard artifacts in the output — a regular grid pattern from uneven kernel overlap. If this appears in your predicted masks, switching to bilinear upsampling followed by a regular convolution (as above) usually resolves it, at a small cost in representational flexibility.

6.23.3 Dataset Preparation and Mask Preprocessing

```
from torch.utils.data import Dataset
import cv2
import numpy as np

class SegmentationDataset(Dataset):
    def __init__(self, image_paths, mask_paths, transform=None):
        self.image_paths = image_paths
        self.mask_paths = mask_paths
        self.transform = transform
```

```
    def __len__(self):
        return len(self.image_paths)

    def __getitem__(self, idx):
        img = cv2.imread(self.image_paths[idx])
        img = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
        mask = cv2.imread(self.mask_paths[idx], cv2.IMREAD_GRAYSCALE)   # single-channel class-ID mask
        if img is None or mask is None:
            raise FileNotFoundError(self.image_paths[idx])

        if self.transform:
            augmented = self.transform(image=img, mask=mask)   # e.g. albumentations — applies identically to
            img, mask = augmented["image"], augmented["mask"]

        return img, mask.long()   # mask stays integer class IDs, not float
```

• Best Practice

Use a library that applies the exact same random transform to both image and mask simultaneously (e.g. `albumentations`), rather than calling two separate transform pipelines. Calling `RandomCrop` independently on the image and the mask crops two *different* random regions, silently misaligning every pixel label with its image — a bug that trains without error but produces a model that has learned nothing real.

6.23.4 Dice Loss and Training

Cross-entropy alone often underperforms on segmentation with class imbalance (e.g. a small foreground object against a large background). Dice loss directly optimizes for mask overlap and handles imbalance better:

```
def dice_loss(pred_logits, target, eps=1e-6):
    pred = torch.sigmoid(pred_logits)          # for binary segmentation
    pred_flat = pred.view(pred.size(0), -1)
    target_flat = target.view(target.size(0), -1).float()
    intersection = (pred_flat * target_flat).sum(dim=1)
    dice = (2 * intersection + eps) / (pred_flat.sum(dim=1) + target_flat.sum(dim=1) + eps)
    return 1 - dice.mean()

def combined_loss(pred_logits, target):
    bce = F.binary_cross_entropy_with_logits(pred_logits, target.float())
    dice = dice_loss(pred_logits, target)
    return bce + dice   # combining both often trains more stably than either alone
```

6.23.5 IoU for Segmentation

```
def segmentation_iou(pred_mask, gt_mask, num_classes):
    ious = []
    for cls in range(num_classes):
        pred_cls = pred_mask == cls
        gt_cls = gt_mask == cls
        intersection = (pred_cls & gt_cls).sum()
```

```
        union = (pred_cls | gt_cls).sum()
        ious.append(intersection / union if union > 0 else float("nan"))
    return ious
```

6.23.6 Inference and Visualization

```
@torch.no_grad()
def predict_mask(model, image, device):
    model.eval()
    tensor = preprocess(image).unsqueeze(0).to(device)
    logits = model(tensor)
    probs = torch.sigmoid(logits)                # binary; use softmax for multiclass
    mask = (probs > 0.5).squeeze(0).squeeze(0).cpu().numpy().astype(np.uint8)
    return mask

def overlay_mask(image, mask, color=(0, 255, 0), alpha=0.5):
    overlay = image.copy()
    overlay[mask == 1] = color
    return cv2.addWeighted(overlay, alpha, image, 1 - alpha, 0)
```

★ Quick Revision Sheet

U-Net Deep Dive, in one screen:

- Skip connections carry fine spatial detail from encoder to decoder — essential for sharp mask boundaries.
- Transposed convolutions can cause checkerboard artifacts; bilinear upsample + conv is a common fix.
- Apply identical random transforms to image and mask together (e.g. via `albumentations`) to prevent misalignment.
- Combine Dice loss with BCE/cross-entropy for more stable training under class imbalance.

Practice Tasks

- **Level 1:** Explain why mask class IDs must stay as integers (`long`) rather than being cast to `float` before the loss computation.
- **Level 2:** Modify `combined_loss` to support multiclass segmentation (softmax + categorical Dice).
- **Level 3:** Implement a full U-Net training loop with combined BCE+Dice loss and periodic mask visualization.
- **Level 4:** Debug a segmentation model with near-zero IoU despite reasonable-looking loss curves — check for image/mask misalignment from independently-applied augmentation.
- **Level 5:** Compare training convergence with Dice loss alone vs. BCE+Dice combined.
- **Level 6:** Build an inference pipeline that overlays predicted masks on original images for visual QA before deployment.

Chapter 6.24 — Instance Segmentation

6.24.1 Semantic vs. Instance Segmentation

Semantic segmentation labels every pixel with a class, but does not distinguish between separate objects of the same class — three overlapping people are all just "person" pixels. Instance segmentation adds per-object identity: each individual object instance gets its own mask, box, and class label.

	Semantic Segmentation	Instance Segmentation
Output	One class-ID mask	Per-object: box + class + individual mask
Distinguishes objects of the same class?	No	Yes
Typical architecture	U-Net, DeepLab, FCN	Mask R-CNN

6.24.2 Mask R-CNN: Faster R-CNN + a Mask Branch

Mask R-CNN extends Faster R-CNN (Chapter 6.19) with a third parallel head: alongside classification and box regression, a small fully-convolutional network predicts a binary mask for each detected object, within its predicted bounding box.

```
Image
 |
 v
Backbone + FPN
 |
 v
Region Proposal Network
 |
 v
ROI Align
 |
 +---> Classification Head --> class label
 |
 +---> Box Regression Head --> refined box
 |
 +---> Mask Head (small FCN) --> per-instance binary mask
```

6.24.3 torchvision Implementation

```
import torchvision.models.detection as detection
import torch
from torchvision.transforms.functional import to_tensor
from PIL import Image

model = detection.maskrcnn_resnet50_fpn_v2(
    weights=detection.MaskRCNN_ResNet50_FPN_V2_Weights.COCO_V1
)
model.eval()

img = to_tensor(Image.open("image.jpg").convert("RGB"))
with torch.no_grad():
    predictions = model([img])

boxes = predictions[0]["boxes"]        # (N, 4)
labels = predictions[0]["labels"]      # (N,)
scores = predictions[0]["scores"]      # (N,)
masks = predictions[0]["masks"]        # (N, 1, H, W), values in [0, 1] — threshold to get a binary mask
```

Fine-tuning on a custom dataset:

```
from torchvision.models.detection.faster_rcnn import FastRCNNPredictor
from torchvision.models.detection.mask_rcnn import MaskRCNNPredictor

num_classes = 3 + 1   # +1 background, same convention as Faster R-CNN
model = detection.maskrcnn_resnet50_fpn_v2(weights="DEFAULT")

in_features = model.roi_heads.box_predictor.cls_score.in_features
model.roi_heads.box_predictor = FastRCNNPredictor(in_features, num_classes)

in_features_mask = model.roi_heads.mask_predictor.conv5_mask.in_channels
model.roi_heads.mask_predictor = MaskRCNNPredictor(in_features_mask, 256, num_classes)
```

6.24.4 Visualizing Instance Masks

```
import numpy as np
import cv2

def draw_instances(image, boxes, masks, labels, scores, class_names, score_threshold=0.5):
    output = image.copy()
    colors = np.random.randint(0, 255, (len(class_names), 3))
    for box, mask, label, score in zip(boxes, masks, labels, scores):
        if score < score_threshold:
            continue
        binary_mask = (mask[0] > 0.5).numpy().astype(np.uint8)
        color = colors[label].tolist()
        output[binary_mask == 1] = (output[binary_mask == 1] * 0.5 + np.array(color) * 0.5).astype(np.uint8)
```

```
        x1, y1, x2, y2 = box.int().tolist()
        cv2.rectangle(output, (x1, y1), (x2, y2), color, 2)
        cv2.putText(output, f"{class_names[label]} {score:.2f}", (x1, y1 - 5),
                    cv2.FONT_HERSHEY_SIMPLEX, 0.5, color, 1)
    return output
```

- **Common Mistake**

Mask R-CNN's raw mask output is a per-pixel probability map in `[0, 1]`, not a binary mask. Forgetting to threshold it (typically at 0.5) before using it for area calculations, overlays, or downstream processing produces subtly wrong results — soft edges get counted partially, throwing off pixel-area-based metrics.

★ Quick Revision Sheet

Instance Segmentation, in one screen:

- Instance segmentation distinguishes individual object instances; semantic segmentation does not.
- Mask R-CNN = Faster R-CNN + a parallel mask-prediction head running on ROI-Align'd features.
- Mask R-CNN's torchvision output masks are probabilities in `[0,1]` — threshold before using them as binary masks.
- Fine-tuning requires replacing both the box predictor and the mask predictor heads.

Practice Tasks

- **Level 1:** Explain in one sentence why instance segmentation needs per-object masks rather than one global class mask.
- **Level 2:** Modify `draw_instances` to also report the pixel area of each detected instance.
- **Level 3:** Fine-tune Mask R-CNN's box and mask predictors on a small custom instance-segmentation dataset.
- **Level 4:** Debug an instance-segmentation pipeline reporting incorrect object areas — check whether raw probability masks were thresholded before area calculation.
- **Level 5:** Benchmark Mask R-CNN inference latency vs. a U-Net semantic segmentation model on the same hardware.
- **Level 6:** Export a fine-tuned Mask R-CNN model and validate mask output parity after export.

Chapter 6.25 — Segmentation Metrics

6.25.1 Core Metrics

Metric	Formula	Notes
IoU (Jaccard Index)	`intersection / union`	Standard per-class overlap metric
Dice coefficient	`2 * intersection / (area1 + area2)`	Related to IoU (`Dice = 2*IoU / (1+IoU)`); more sensitive to small objects
Pixel accuracy	`correct pixels / total pixels`	Easy to compute, misleading under class imbalance (large background dominates)
Precision	`TP / (TP + FP)` (pixel-level)	Fraction of predicted foreground pixels that are correct
Recall	`TP / (TP + FN)` (pixel-level)	Fraction of actual foreground pixels correctly predicted
Mean IoU (mIoU)	Average of per-class IoU	Standard summary metric for multiclass segmentation benchmarks

```python
import numpy as np

def compute_segmentation_metrics(pred_mask, gt_mask, num_classes):
    ious, dices = [], []
    for cls in range(num_classes):
        pred_cls = pred_mask == cls
        gt_cls = gt_mask == cls
        intersection = np.logical_and(pred_cls, gt_cls).sum()
        union = np.logical_or(pred_cls, gt_cls).sum()
        iou = intersection / union if union > 0 else float("nan")
        dice = 2 * intersection / (pred_cls.sum() + gt_cls.sum()) if (pred_cls.sum() + gt_cls.sum()) > 0 else
        ious.append(iou)
        dices.append(dice)
    return {"per_class_iou": ious, "mean_iou": np.nanmean(ious), "per_class_dice": dices}
```

- **Common Mistake**

Reporting only pixel accuracy for a segmentation task with a small foreground object against a large background. A model that predicts ”background” for every pixel can score 95

6.25.2 Which Metric to Use When

Scenario	Preferred metric
Balanced multiclass segmentation (e.g. street scene parsing)	Mean IoU
Small object against large background (e.g. tumor segmentation)	Dice coefficient, per-class recall
Boundary precision matters (e.g. document/text region segmentation)	IoU at the pixel level, plus visual boundary inspection
Quick sanity check during early training	Pixel accuracy is acceptable, but confirm with IoU/Dice before trusting a model

★ Quick Revision Sheet

Segmentation Metrics, in one screen:

- IoU and Dice are the standard overlap metrics; Dice is more forgiving/sensitive on small objects.
- Pixel accuracy is misleading under class imbalance — pair it with IoU/Dice, never report it alone.
- Mean IoU (mIoU) is the standard multiclass summary metric.
- Medical/small-object segmentation: prioritize Dice and per-class recall over raw pixel accuracy.

Practice Tasks

- **Level 1:** Given a predicted and ground-truth binary mask as small arrays, compute IoU and Dice by hand.
- **Level 2:** Modify `compute_segmentation_metrics` to also return pixel-level precision and recall per class.
- **Level 3:** Implement a full evaluation script that computes mIoU across an entire validation set.
- **Level 4:** Debug a segmentation model reporting 96% pixel accuracy but visibly wrong masks — compute per-class IoU to diagnose.
- **Level 5:** Compare mIoU trends across training epochs to determine the best checkpoint (rather than relying on loss alone).
- **Level 6:** Build an automated evaluation report combining mIoU, Dice, and confusion-style per-class breakdown for a segmentation model.

Chapter 6.26 — OCR Pipeline

6.26.1 The OCR Workflow

```
Image
 |
 v
Preprocessing (deskew, denoise, binarize)
 |
 v
Text Detection (locate text regions)
 |
 v
Text Recognition (convert region pixels to characters)
 |
 v
Post-processing (spell-check, structure extraction)
 |
 v
Structured Output (plain text, or key-value fields)
```

Text detection and text recognition are conceptually separate steps, even when a single library call appears to do both — detection finds *where* text is (bounding boxes/regions), recognition converts each region's pixels into characters.

6.26.2 OpenCV Preprocessing for OCR

```
import cv2
import numpy as np

def preprocess_for_ocr(image_path):
    img = cv2.imread(image_path)
    gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

    # Denoise
    denoised = cv2.fastNlMeansDenoising(gray, h=10)

    # Binarize — adaptive threshold handles uneven document lighting well
    binary = cv2.adaptiveThreshold(
        denoised, 255, cv2.ADAPTIVE_THRESH_GAUSSIAN_C, cv2.THRESH_BINARY, 31, 15
```

```
    )

    # Deskew: detect rotation angle from text line orientation and correct it
    coords = np.column_stack(np.where(binary == 0))
    angle = cv2.minAreaRect(coords)[-1]
    if angle < -45:
        angle = -(90 + angle)
    else:
        angle = -angle
    (h, w) = binary.shape
    M = cv2.getRotationMatrix2D((w // 2, h // 2), angle, 1.0)
    deskewed = cv2.warpAffine(binary, M, (w, h), flags=cv2.INTER_CUBIC, borderMode=cv2.BORDER_REPLICATE)

    return deskewed
```

- **Best Practice**

Deskewing and binarizing before OCR often improves accuracy more than any OCR-engine configuration tuning. OCR engines are trained on relatively clean, upright text; a document photographed at even a 5-10 degree tilt or under uneven lighting can produce dramatically worse recognition than the same document after deskew and adaptive thresholding.

6.26.3 pytesseract: Basic Usage

```
import pytesseract
from PIL import Image

text = pytesseract.image_to_string(Image.open("document.png"))
print(text)

# With bounding boxes and confidence scores
data = pytesseract.image_to_data(Image.open("document.png"), output_type=pytesseract.Output.DICT)
for i in range(len(data["text"])):
    if int(data["conf"][i]) > 60 and data["text"][i].strip():
        x, y, w, h = data["left"][i], data["top"][i], data["width"][i], data["height"][i]
        print(f"'{data['text'][i]}' at ({x},{y},{w},{h}) confidence={data['conf'][i]}")
```

6.26.4 Handling Noisy Document Images

```
def full_ocr_pipeline(image_path, min_confidence=60):
    processed = preprocess_for_ocr(image_path)
    data = pytesseract.image_to_data(processed, output_type=pytesseract.Output.DICT)

    results = []
    for i in range(len(data["text"])):
        text = data["text"][i].strip()
        conf = int(data["conf"][i])
        if text and conf >= min_confidence:
```

```
            results.append({
                "text": text,
                "confidence": conf,
                "box": (data["left"][i], data["top"][i], data["width"][i], data["height"][i]),
            })
    return results
```

- **Common Mistake**

Running OCR directly on a raw, unpreprocessed photo of a document (as opposed to a clean scan) and treating low-confidence, garbled output as an OCR-engine limitation rather than a preprocessing problem. In practice the single highest-leverage fix for noisy real-world document images is almost always better preprocessing — denoising, binarization, deskewing, and cropping to the actual document region — not switching OCR engines.

★ Quick Revision Sheet

OCR Pipeline, in one screen:

- Pipeline: preprocess -> detect text regions -> recognize characters -> post-process -> structured output.
- Preprocessing (deskew, denoise, binarize) is usually the highest-leverage accuracy lever, not the OCR engine choice.
- `pytesseract.image_to_data` gives per-word bounding boxes and confidence scores — always filter by a minimum confidence.
- Treat garbled OCR output on real-world photos as a preprocessing problem first.

Practice Tasks

- **Level 1:** Run `pytesseract.image_to_string` on a clean scanned document and a noisy phone photo of the same text; compare output quality.
- **Level 2:** Modify `preprocess_for_ocr` to also crop to the largest detected contour (isolating the document from its background).
- **Level 3:** Implement `full_ocr_pipeline` end-to-end and test it on a folder of varied document photos.
- **Level 4:** Debug a pipeline producing garbled output on a tilted photo — confirm the deskew step is actually correcting the rotation angle.
- **Level 5:** Compare OCR accuracy (character error rate) with and without the preprocessing step on a small labeled test set.
- **Level 6:** Wrap `full_ocr_pipeline` into a function suitable for a document-upload API endpoint, with error handling for unreadable images.

Chapter 6.27 — Tesseract OCR

6.27.1 Installation and Setup

```
# Ubuntu/Debian
sudo apt-get install tesseract-ocr
sudo apt-get install tesseract-ocr-eng tesseract-ocr-fra   # language packs, one per language

pip install pytesseract --break-system-packages
```

```
import pytesseract
pytesseract.pytesseract.tesseract_cmd = "/usr/bin/tesseract"   # set explicitly if not on PATH
print(pytesseract.get_tesseract_version())
```

6.27.2 Language Packs

```
text = pytesseract.image_to_string(img, lang="eng")            # single language
text_multi = pytesseract.image_to_string(img, lang="eng+fra")  # multiple languages in one pass
print(pytesseract.get_languages())   # list installed language packs
```

- **Common Mistake**

Running OCR with the default English language pack on a document containing another language (or a mix of languages) produces confident-looking but wrong output — Tesseract doesn't fail loudly when the wrong language model is used, it just produces its best (bad) guess. Always confirm the document's language(s) and install/specify the matching language pack(s).

6.27.3 Page Segmentation Modes (PSM)

PSM	Meaning	When to use
3 (default)	Fully automatic page segmentation, no OSD	General documents with mixed text blocks
6	Assume a single uniform block of text	Simple paragraphs, receipts
7	Treat the image as a single text line	Cropped single-line regions (e.g. a form field)
11	Sparse text, find as much text as possible in no particular order	Scattered text, screenshots, signage

PSM	Meaning	When to use
13	Raw line, bypass Tesseract-specific processing	Single line, unusual fonts

```
config = "--psm 6"
text = pytesseract.image_to_string(img, config=config)
```

6.27.4 OCR Engine Modes (OEM)

```
config = "--oem 1 --psm 6"   # oem 1 = LSTM neural engine (default and recommended in modern Tesseract)
text = pytesseract.image_to_string(img, config=config)
```

6.27.5 Extracting Bounding Boxes and Confidence

```
data = pytesseract.image_to_data(img, output_type=pytesseract.Output.DICT, config="--psm 6")

words = []
for i in range(len(data["text"])):
    if data["text"][i].strip():
        words.append({
            "text": data["text"][i],
            "confidence": int(data["conf"][i]),
            "bbox": (data["left"][i], data["top"][i], data["width"][i], data["height"][i]),
        })
```

6.27.6 Preprocessing Before Tesseract

Reuse the `preprocess_for_ocr` function from Chapter 6.26 — Tesseract's accuracy is highly sensitive to image quality, and the standard advice (300 DPI minimum, high contrast, deskewed, minimal noise) applies directly.

```
def ocr_with_confidence_filter(image_path, psm=6, min_conf=60):
    processed = preprocess_for_ocr(image_path)
    config = f"--oem 1 --psm {psm}"
    data = pytesseract.image_to_data(processed, output_type=pytesseract.Output.DICT, config=config)
    return [
        {"text": data["text"][i], "conf": int(data["conf"][i])}
        for i in range(len(data["text"]))
        if data["text"][i].strip() and int(data["conf"][i]) >= min_conf
    ]
```

6.27.7 Common Errors and Debugging

Symptom	Likely cause	Fix
Garbled/random characters	Wrong PSM for the layout, or poor preprocessing	Try different PSM values; improve preprocessing
Missing whole sections of text	Image resolution too low, or dense/small font	Upscale image before OCR; increase source scan DPI
Wrong language characters	Wrong or missing language pack	Install and specify the correct `lang` parameter
`TesseractNotFoundError`	Tesseract binary not installed or not on PATH	Install via package manager; set `tesseract_cmd` explicitly

• Best Practice

When OCR accuracy is unsatisfactory, work through fixes in this order: (1) improve preprocessing — deskew, denoise, binarize, upscale if resolution is low; (2) try different PSM values matching the actual layout; (3) confirm the correct language pack is installed and specified. Only after exhausting these should you consider replacing Tesseract with a different OCR engine — most ”OCR engine is bad” complaints are actually preprocessing or configuration problems.

★ Quick Revision Sheet

Tesseract OCR, in one screen:

- Always install and specify the correct language pack(s) — wrong language fails silently, not loudly.
- PSM controls page-layout assumptions; match it to the actual document structure (single block, single line, sparse text, etc.).
- OEM 1 (LSTM) is the modern recommended engine mode.
- Debug in order: preprocessing -> PSM -> language pack -> only then consider a different engine.

Practice Tasks

- **Level 1:** Run Tesseract on a sample image with three different PSM values and compare output quality.
- **Level 2:** Modify `ocr_with_confidence_filter` to also return the average confidence across all detected words.
- **Level 3:** Implement a multi-language OCR pipeline that first detects likely language, then re-runs OCR with the matching language pack.
- **Level 4:** Debug an OCR run producing empty output on a valid document image — check PSM, language pack, and Tesseract installation.
- **Level 5:** Benchmark OCR accuracy (word-level match rate against ground truth) with and without the preprocessing pipeline from Chapter 6.26.
- **Level 6:** Wrap the full OCR pipeline (preprocessing + Tesseract + confidence filtering) into a production-ready function with structured error handling.

Chapter 6.28 — Computer Vision Datasets

6.28.1 Classification Datasets

Dataset	Official Source	Task	Classes	Format	Difficulty	License
MNIST	http://yann.lecun.com/exdb/mnist/	Digit classification	10	IDX binary	Beginner	Public/CC
Fashion-MNIST	https://github.com/zalandoresearch/fashion-mnist	Clothing classification	10	IDX binary (MNIST-compatible)	Beginner	MIT
CIFAR-10	https://www.cs.toronto.edu/~kriz/cifar.html	Object classification	10	Python pickle / binary	Beginner-intermediate	Research use, see site
CIFAR-100	https://www.cs.toronto.edu/~kriz/cifar.html	Fine-grained object classification	100	Python pickle / binary	Intermediate	Research use, see site
ImageNet (ILSVRC)	https://www.image-net.org/	Large-scale object classification	1,000 (ILSVRC subset)	JPEG + XML/text labels	Advanced	Non-commercial research, registration required

6.28.2 Detection Datasets

Dataset	Official Source	Task	Format	Difficulty	License
COCO	https://cocodataset.org/	Detection, segmentation, keypoints	COCO JSON	Intermediate-advanced	CC BY 4.0
Pascal VOC	http://host.robots.ox.ac.uk/pascal/VOC/	Detection, segmentation	XML (VOC format)	Beginner-intermediate	Research use, see site
Open Images	https://storage.googleapis.com/openimages/web/index.html	Detection, segmentation, visual relationships	CSV + image-level/box-level annotations	Advanced	CC BY 4.0 (varies by asset)

6.28.3 Segmentation Datasets

Dataset	Official Source	Task	Format	Difficulty	License
Cityscapes	https://www.cityscapes-dataset.com/	Urban scene semantic/instance segmentation	PNG masks + JSON	Advanced	Research use, registration required
Pascal VOC (segmentation)	http://host.robots.ox.ac.uk/pascal/VOC/	Semantic segmentation	PNG masks	Intermediate	Research use, see site
COCO (segmentation)	https://cocodataset.org/	Instance segmentation	COCO JSON (polygon/RLE masks)	Advanced	CC BY 4.0

6.28.4 OCR Datasets

Dataset	Official Source	Task	Format	Difficulty	License
ICDAR (various years/challenges)	https://rrc.cvc.uab.es/	Text detection and recognition	Varies by challenge (XML/text + images)	Intermediate-advanced	Competition-specific, see site
SVT (Street View Text)	http://vision.ucsd.edu/kai/svt/	Scene text recognition	XML + images	Intermediate	Research use, see site

- **Common Mistake**

Assuming a dataset's exact sample count, class count, or license from memory or a secondary source instead of the official page. Datasets get updated, re-versioned, and sometimes re-licensed over time — always confirm the current numbers and license terms directly from the official source linked above before including them in a report, a training plan, or (especially) anything commercial.

6.28.5 Recommended Projects by Dataset

Dataset	Recommended project
MNIST / Fashion-MNIST	First CNN-from-scratch classifier (Chapter 6.35, Project 1)
CIFAR-10	Transfer-learning classifier comparison (frozen vs. fine-tuned backbones)
COCO (subset)	Custom YOLO detector on a filtered subset of relevant classes
Cityscapes	U-Net or DeepLab semantic segmentation for urban scene understanding
ICDAR	End-to-end OCR pipeline benchmark

★ Quick Revision Sheet

CV Datasets, in one screen:

- MNIST/Fashion-MNIST/CIFAR: classification, beginner-friendly, small.
- COCO/Pascal VOC/Open Images: detection and segmentation, the standard benchmarks.
- Cityscapes: the standard urban-scene segmentation benchmark.
- ICDAR: the standard OCR benchmark family.
- Always verify current counts/format/license from the official source before relying on remembered numbers.

Practice Tasks

- **Level 1:** Download CIFAR-10 and print the shape and class distribution of the training set.
- **Level 2:** Write a loader that converts a Pascal VOC XML annotation into YOLO-format labels.
- **Level 3:** Build a small COCO-subset dataset (2-3 classes) filtered from the full COCO annotations for a custom detection project.
- **Level 4:** Debug a COCO-format loader producing empty annotations — check for a mismatch between image IDs and annotation `image_id` fields.
- **Level 5:** Compare class distribution across train/val splits for a downloaded dataset to check for imbalance.
- **Level 6:** Set up a reproducible dataset-download script with checksums to verify integrity after download.

Chapter 6.29 — Computer Vision Data Quality

6.29.1 Corrupted and Duplicate Images

```
import os
import hashlib
from PIL import Image

def find_corrupted_images(image_dir):
    corrupted = []
    for fname in os.listdir(image_dir):
        path = os.path.join(image_dir, fname)
        try:
            with Image.open(path) as img:
                img.verify()   # checks file integrity without fully decoding
        except Exception as e:
            corrupted.append((fname, str(e)))
    return corrupted

def find_duplicate_images(image_dir):
    hashes = {}
    duplicates = []
    for fname in os.listdir(image_dir):
        path = os.path.join(image_dir, fname)
        with open(path, "rb") as f:
            file_hash = hashlib.md5(f.read()).hexdigest()
        if file_hash in hashes:
            duplicates.append((fname, hashes[file_hash]))
        else:
            hashes[file_hash] = fname
    return duplicates
```

- **Common Mistake**

Exact-hash duplicate detection (as above) only catches byte-identical files. Near-duplicates — the same photo re-saved at a different quality/resolution, or two frames one second apart from the same video — require perceptual hashing (e.g. `imagehash`'s pHash) instead, since their raw bytes differ even though their visual content is nearly identical.

```
import imagehash
from PIL import Image

def find_near_duplicates(image_dir, threshold=5):
    hashes = {}
    near_dupes = []
    for fname in os.listdir(image_dir):
        h = imagehash.phash(Image.open(os.path.join(image_dir, fname)))
        for existing_fname, existing_hash in hashes.items():
            if h - existing_hash <= threshold:   # Hamming distance
                near_dupes.append((fname, existing_fname))
        hashes[fname] = h
    return near_dupes
```

6.29.2 Mislabeled Images and Noisy Labels

```
@torch.no_grad()
def find_likely_mislabeled(model, dataset, device, confidence_threshold=0.95):
    """Flag examples where the model is highly confident in a DIFFERENT class than the label —
    a strong signal (not proof) of a mislabeled example, especially after the model has already
    learned reasonably well on the rest of the dataset."""
    model.eval()
    suspects = []
    for idx in range(len(dataset)):
        img, label = dataset[idx]
        logits = model(img.unsqueeze(0).to(device))
        probs = torch.softmax(logits, dim=1)[0]
        pred = probs.argmax().item()
        if pred != label and probs[pred].item() > confidence_threshold:
            suspects.append((idx, label, pred, probs[pred].item()))
    return suspects
```

6.29.3 Class Imbalance

```
from collections import Counter

def check_class_balance(dataset):
    labels = [label for _, label in dataset.samples]
    counts = Counter(labels)
    total = len(labels)
    for cls, count in sorted(counts.items()):
        print(f"Class {cls}: {count} ({100*count/total:.1f}%)")
```

Imbalance handling technique	Approach
Class-weighted loss	Weight `CrossEntropyLoss` inversely proportional to class frequency

Imbalance handling technique	Approach
Oversampling	Repeat minority-class examples (with augmentation to avoid exact duplication)
Undersampling	Reduce majority-class examples — only when the majority class is still large enough afterward
Focal loss	Down-weight easy/majority examples automatically (see Chapter 6.20)

6.29.4 Inconsistent Resolution and Domain Shift

```python
def audit_resolutions(image_dir):
    resolutions = Counter()
    for fname in os.listdir(image_dir):
        with Image.open(os.path.join(image_dir, fname)) as img:
            resolutions[img.size] += 1
    return resolutions.most_common(10)
```

Domain shift — a mismatch between the training data's visual distribution and the deployment environment's (different cameras, lighting, geography, demographics) — is often invisible in offline metrics and only shows up as unexpectedly poor production performance. Auditing a sample of real production inputs against the training distribution, even informally, is worth doing before deployment, not after a performance complaint arrives.

• Best Practice

Run a data-quality audit (corrupted files, duplicates/near-duplicates, class balance, resolution consistency) as a standard first step on every new dataset, before writing any training code. Catching these issues at this stage is orders of magnitude cheaper than discovering them after a confusing training run or a production incident.

★ Quick Revision Sheet

CV Data Quality, in one screen:

- Check for corrupted files with `Image.verify()` before any training run.
- Exact-hash duplicates vs. perceptual-hash near-duplicates are different problems needing different tools.
- Use model disagreement (high-confidence wrong prediction) as a signal, not proof, of mislabeled data.
- Audit class balance and resolution consistency early — both are cheap to check and expensive to discover late.

Practice Tasks

- **Level 1:** Run `find_corrupted_images` on a sample dataset folder and report any failures.
- **Level 2:** Modify `find_near_duplicates` to use a configurable Hamming distance threshold and report a summary count.
- **Level 3:** Implement `find_likely_mislabeled` on a partially-trained classifier and manually review the top 20 suspects.
- **Level 4:** Debug a dataset where `check_class_balance` reveals one class has 100x more samples than another — propose and implement an imbalance-handling strategy.
- **Level 5:** Profile the runtime cost of perceptual-hash near-duplicate detection on a 50,000-image dataset.
- **Level 6:** Build a standard "dataset audit" script combining corruption, duplicate, balance, and resolution checks into one report, reusable across projects.

Chapter 6.30 — Computer Vision Data Leakage

Data leakage — information from outside the training set improperly influencing model evaluation — produces validation and test metrics that look excellent but don't reflect real-world performance. It is one of the most common and most damaging mistakes in applied computer vision, precisely because a leaked pipeline still "works" and still reports good numbers.

6.30.1 Duplicate Leakage

The same image (or a near-duplicate) appearing in both the training and validation/test sets. The model effectively memorizes the exact image rather than learning generalizable features, and validation accuracy overstates true generalization.

Prevention: Run near-duplicate detection (Chapter 6.29) *before* splitting the dataset, and split by unique underlying source, not by individual file.

6.30.2 Patient-Level Leakage (Medical/Person-Level Imaging)

If a dataset contains multiple images per patient (e.g. several X-rays from the same person, or several photos of the same individual), splitting by *image* instead of by *patient* lets images of the same patient appear in both train and test. The model can learn patient-specific, non-diagnostic features (equipment artifacts, positioning quirks, image metadata) that happen to correlate with the label for that specific patient, inflating test performance in a way that won't generalize to new patients.

```
from sklearn.model_selection import GroupShuffleSplit

def patient_level_split(image_paths, patient_ids, labels, test_size=0.2):
    splitter = GroupShuffleSplit(n_splits=1, test_size=test_size, random_state=42)
    train_idx, test_idx = next(splitter.split(image_paths, labels, groups=patient_ids))
    return train_idx, test_idx   # guarantees no patient appears in both splits
```

- **Common Mistake**

Using a standard random `train_test_split` on a dataset where samples are grouped (multiple images per patient, multiple frames per video, multiple crops per source photo) without accounting for the grouping. This is one of the single most common causes of a model that reports 95%+ validation accuracy but performs dramatically worse on genuinely new patients/subjects/sources after deployment.

6.30.3 Video-Frame Leakage

Consecutive video frames are highly similar. Splitting frames randomly (rather than by video/clip) lets near-identical frames land in both train and test, again inflating apparent performance.

Prevention: Split by video/clip ID, the same pattern as patient-level splitting above.

6.30.4 Augmentation Leakage

Applying augmentation *before* the train/val split, then splitting the augmented dataset — augmented copies of a validation image can end up in the training set, or vice versa, giving the model indirect exposure to validation examples.

Prevention: Always split first, on the original (unaugmented) data, then apply augmentation only within the training subset's data loading pipeline.

6.30.5 Preprocessing Leakage

Computing preprocessing statistics (e.g. normalization mean/std, or a PCA transform) across the *entire* dataset, including validation/test data, before splitting. This lets test-set statistics subtly influence how training data is transformed.

Prevention: Compute any data-dependent preprocessing statistics from the training split only, then apply those same fixed statistics to validation/test data.

6.30.6 Temporal Leakage

Using future information to predict the past — e.g. training a model with features derived from data that wouldn't have been available at the actual time of the historical prediction being modeled. In CV this shows up in time-series-adjacent tasks (e.g. predicting equipment failure from a sequence of inspection images) when a random split allows a model to "see" a later inspection image chronologically before an earlier one it's meant to predict from.

Prevention: Split strictly by time — all training examples must chronologically precede all validation/test examples.

6.30.7 Train/Test Contamination Summary

```
Leakage type          | Split by, instead of by individual sample
----------------------|-------------------------------------------
Duplicate leakage       | Unique source (after near-duplicate detection)
Patient-level leakage   | Patient/subject ID
Video-frame leakage     | Video/clip ID
Augmentation leakage    | (split before augmenting, not a grouping issue)
Preprocessing leakage   | (compute stats from train split only)
Temporal leakage        | Time — strict chronological split
```

- **Best Practice**

Before trusting any validation metric, ask explicitly: "could any information about this specific validation example have influenced the training set?" — through a duplicate, a same-patient image, a nearby video frame, shared preprocessing statistics, or future information. This one question, asked deliberately for every new dataset, catches the majority of leakage before it produces a misleadingly optimistic result.

★ Quick Revision Sheet

Data Leakage, in one screen:

- Leaked pipelines still run and still report good numbers — leakage doesn't announce itself.
- Grouped data (patients, videos, sources) needs grouped splitting, not random per-sample splitting.
- Split before augmenting; compute preprocessing statistics from the training split only.
- Temporal problems need strict chronological splits.
- Always ask "could this validation example have been influenced by training data?" for every new dataset.

Practice Tasks

- **Level 1:** Explain in one sentence why patient-level leakage inflates test accuracy without the model actually generalizing better.
- **Level 2:** Modify `patient_level_split` to also stratify by label within each group split.
- **Level 3:** Implement a full leakage-safe pipeline for a multi-image-per-subject dataset, from split to preprocessing to augmentation.
- **Level 4:** Debug a model reporting 97% validation accuracy but ~70% on a genuinely new held-out batch — audit the dataset for grouping-based leakage.
- **Level 5:** Compare validation metrics from a properly-grouped split vs. a naive random split on the same grouped dataset, to quantify the inflation.
- **Level 6:** Build a leakage-audit checklist script that flags potential duplicate, group, and temporal leakage risks in a new dataset automatically.

Chapter 6.31 — Computer Vision Debugging

A practical, symptom-first debugging checklist for the failures you will actually encounter.

6.31.1 Model Predicts One Class for Everything

Likely causes: severe class imbalance without correction; learning rate too high (collapsed to a trivial solution); frozen backbone with an unreplaced or incorrectly-replaced head; label encoding bug (all labels accidentally mapped to the same value).

```
# Diagnostic: check label distribution actually fed to the model
from collections import Counter
labels_seen = Counter()
for _, label in train_loader:
    labels_seen.update(label.tolist())
print(labels_seen)   # if this is imbalanced or wrong, that's very likely the root cause
```

6.31.2 Loss Doesn't Decrease

Likely causes: learning rate too low or too high; frozen layers that should be trainable; a data pipeline bug feeding constant or corrupted input; loss function mismatched to the output layer (e.g. `CrossEntropyLoss` on already-softmaxed output).

```
# Diagnostic: confirm gradients are actually flowing
loss.backward()
for name, param in model.named_parameters():
    if param.requires_grad and param.grad is None:
        print(f"WARNING: {name} has requires_grad=True but no gradient")
```

6.31.3 NaN Loss

Likely causes: learning rate too high (diverging); division by zero in a custom loss function (e.g. Dice loss without an epsilon term); mixed-precision training without gradient scaling; corrupted input data (e.g. all-zero or all-NaN images).

```
# Diagnostic: bisect which step introduces NaN
for images, labels in train_loader:
    assert not torch.isnan(images).any(), "NaN in input images"
    outputs = model(images)
```

```
    assert not torch.isnan(outputs).any(), "NaN in model output — check for exploding activations"
    loss = criterion(outputs, labels)
    assert not torch.isnan(loss), "NaN in loss — check loss function epsilon terms and LR"
    break
```

6.31.4 CUDA Out-of-Memory

Likely causes: batch size too large for available VRAM; not clearing intermediate tensors between batches; accumulating a computation graph across iterations (e.g. logging `loss` instead of `loss.item()`); validation running without `torch.no_grad()`.

```
# Common leak: this keeps the whole computation graph alive across the entire epoch
losses = []
for images, labels in train_loader:
    loss = criterion(model(images), labels)
    losses.append(loss)          # BUG — keeps the graph attached
    losses.append(loss.item())   # CORRECT — detaches to a plain float
```

6.31.5 Incorrect Image Channels / Tensor Shape

Likely causes: BGR vs. RGB mismatch; channels-first vs. channels-last mismatch between PyTorch and TensorFlow conventions; missing or extra batch dimension.

```
def sanity_check_batch(batch, expected_shape):
    assert batch.shape[1:] == expected_shape, f"Expected {expected_shape}, got {batch.shape[1:]}"
    print(f"Batch shape: {batch.shape}, dtype: {batch.dtype}, "
          f"range: [{batch.min():.3f}, {batch.max():.3f}]")
```

6.31.6 Wrong Normalization

Symptom: pretrained model performs far below its published accuracy despite a correct architecture and correct weights. **Diagnostic:** print the min/max/mean of a preprocessed batch and compare against the expected range for the normalization scheme in use (e.g. ImageNet-normalized batches should have values roughly in `[-2.5, 2.5]`, not `[0, 255]` or `[0, 1]`).

6.31.7 Bounding Boxes Shifted / Masks Misaligned

Likely causes: forgetting to map coordinates back through a resize/pad transform; `xyxy`/`xywh`/`cxcywh` format confusion; mask resized with bilinear instead of nearest-neighbor interpolation; image and mask augmented independently instead of jointly (Chapter 6.23).

6.31.8 OCR Produces Garbage

Likely causes: wrong page segmentation mode; wrong or missing language pack; no preprocessing on a noisy/tilted source image (Chapters 6.26-6.27).

6.31.9 Detection Confidence Extremely Low

Likely causes: model still under-trained (check training-set performance, not just validation); confidence threshold set inappropriately high for the current model quality; anchor configuration mismatched to actual object sizes in the dataset (anchor-based detectors); train/inference preprocessing mismatch.

6.31.10 Model Works on Training Data but Fails in Deployment

Likely causes: train/serve preprocessing skew (the single most common cause — see Chapter 6.3's emphasis on one shared preprocessing function); domain shift between training data and real deployment inputs (Chapter 6.29); export/quantization introducing accuracy loss without adequate validation; `model.eval()` not called in the deployed inference path.

• Best Practice

Keep a "debugging checklist" like this one pinned during every new project, and work through it symptom-first rather than guessing at root causes. The overwhelming majority of computer vision bugs fall into a small number of well-known categories — shape mismatches, normalization errors, train/serve skew, and leakage — and a systematic checklist finds them far faster than ad hoc investigation.

★ Quick Revision Sheet

CV Debugging, in one screen:

- One-class predictions: check label distribution, LR, and head replacement.
- NaN loss: check LR, loss epsilon terms, mixed-precision scaling, and input data validity.
- OOM: check for accumulated computation graphs (`loss` vs. `loss.item()`) and missing `no_grad()`.
- Train-fine/deploy-broken: almost always preprocessing skew — verify with a shared, tested preprocessing function.
- Print shape/dtype/range at every pipeline boundary when something looks wrong.

Practice Tasks

- **Level 1:** Given a model predicting the same class for every input, list the three most likely causes in priority order.
- **Level 2:** Add the gradient-flow diagnostic to an existing training loop as a permanent sanity check.
- **Level 3:** Implement `sanity_check_batch` and add it to the start of a training loop.
- **Level 4:** Given a training run that diverges to NaN at epoch 3, use the bisection diagnostic to identify the exact failure point.
- **Level 5:** Profile a CUDA-OOM training run to find where the computation graph is unintentionally retained.
- **Level 6:** Write an automated pre-flight check script that runs the shape/normalization/gradient diagnostics on a new project before any real training begins.

Chapter 6.32 — Computer Vision Performance Optimization

6.32.1 Batch Size and Resolution

Larger batch sizes generally improve GPU utilization (more parallel work per kernel launch) up to the point of diminishing returns or memory limits. Higher image resolution improves accuracy on small objects/details but scales compute roughly quadratically with side length — a common, effective lever is training at a smaller resolution and only increasing resolution for a final fine-tuning phase.

6.32.2 DataLoader Workers and Pin Memory

```
train_loader = DataLoader(
    dataset, batch_size=32, shuffle=True,
    num_workers=8,        # parallel CPU processes preparing batches while the GPU trains
    pin_memory=True,      # speeds up host-to-GPU transfer
    persistent_workers=True,   # avoids worker restart overhead between epochs
    prefetch_factor=2,    # batches pre-loaded per worker
)
```

- **Best Practice**

Set `num_workers` to roughly the number of physical CPU cores available, minus a couple reserved for the main process — start around 4-8 and profile from there. Too few workers starves the GPU waiting for data; too many causes CPU contention and can actually slow training down. There's no universal correct value; measure it.

6.32.3 Mixed Precision Training

```
import torch

scaler = torch.cuda.amp.GradScaler()

for images, labels in train_loader:
    images, labels = images.to(device), labels.to(device)
    optimizer.zero_grad()

    with torch.autocast(device_type="cuda", dtype=torch.float16):
        outputs = model(images)
```

```
        loss = criterion(outputs, labels)

    scaler.scale(loss).backward()   # scales loss to prevent float16 gradient underflow
    scaler.step(optimizer)
    scaler.update()
```

Mixed precision typically gives a 1.5-3x speedup on modern GPUs with tensor cores, at negligible accuracy cost for most vision tasks.

6.32.4 Gradient Accumulation

```
accumulation_steps = 4   # effective batch size = batch_size * accumulation_steps

optimizer.zero_grad()
for i, (images, labels) in enumerate(train_loader):
    outputs = model(images.to(device))
    loss = criterion(outputs, labels.to(device)) / accumulation_steps
    loss.backward()
    if (i + 1) % accumulation_steps == 0:
        optimizer.step()
        optimizer.zero_grad()
```

Use gradient accumulation when the desired effective batch size doesn't fit in available GPU memory at once.

6.32.5 Caching and Augmentation Bottlenecks

```
# If augmentation is CPU-bound and starving the GPU, profile to confirm before optimizing
import time

def profile_dataloader(loader, num_batches=20):
    start = time.time()
    for i, _ in enumerate(loader):
        if i >= num_batches:
            break
    elapsed = time.time() - start
    print(f"{num_batches} batches in {elapsed:.2f}s ({elapsed/num_batches*1000:.1f}ms/batch)")
```

- **Common Mistake**

Optimizing model architecture or GPU code when the actual bottleneck is CPU-bound data loading/augmentation. Profile the DataLoader in isolation (as above) before assuming the model or GPU utilization is the problem — a surprising fraction of "slow training" complaints trace back to an underpowered or misconfigured data pipeline, not the model itself.

6.32.6 GPU Utilization Monitoring

```
# Watch GPU utilization live while training runs
watch -n 1 nvidia-smi
```

Low GPU utilization (consistently under ~80-90%) during training strongly suggests a data-loading bottleneck rather than a compute-bound model; consistently high utilization with slow throughput suggests the model/compute itself is the limiting factor, and the next lever to reach for is mixed precision or a more efficient architecture.

6.32.7 Model Pruning and Quantization (Concepts)

- **Pruning**: removes low-magnitude or low-importance weights/channels, reducing model size and (with structured pruning) compute cost, at some accuracy cost that typically requires fine-tuning to recover.
- **Quantization**: reduces numerical precision of weights/activations (e.g. float32 to int8), reducing model size ~4x and often speeding up inference on supporting hardware (see Chapter 6.8's edge-deployment coverage).

6.32.8 ONNX and Inference Batching

```
import onnxruntime as ort

session = ort.InferenceSession("model.onnx", providers=["CUDAExecutionProvider"])
outputs = session.run(None, {"input": batch_numpy_array})
```

Batching inference requests (processing multiple images per forward pass instead of one-at-a-time) substantially improves throughput on GPU by better utilizing parallel compute, at the cost of added per-request latency for the individual requests waiting to fill a batch — a real tradeoff for latency-sensitive serving.

★ Quick Revision Sheet

CV Performance Optimization, in one screen:

- Profile the DataLoader in isolation before optimizing the model — data loading is a very common actual bottleneck.
- Mixed precision (`autocast` + `GradScaler`) is close to a free 1.5-3x speedup on modern GPUs.
- Gradient accumulation simulates a larger batch size when memory is the constraint.
- `nvidia-smi` utilization tells you whether you're data-bound or compute-bound.
- Batch inference trades individual-request latency for throughput — the right choice depends on the serving scenario.

Practice Tasks

- **Level 1:** Run `profile_dataloader` on an existing training pipeline and report ms/batch.
- **Level 2:** Add mixed-precision training to an existing training loop and measure the speedup.
- **Level 3:** Implement gradient accumulation to simulate a batch size 4x larger than what fits in memory.
- **Level 4:** Debug a training run with low GPU utilization — determine whether `num_workers` or augmentation complexity is the bottleneck.
- **Level 5:** Compare throughput of single-image vs. batched ONNX Runtime inference on the same hardware.
- **Level 6:** Set up an end-to-end optimized training pipeline (mixed precision + tuned DataLoader workers) and document the measured speedup versus the naive baseline.

Chapter 6.33 — Computer Vision Deployment

6.33.1 The Deployment Pipeline

```
Training Model
      |
      v
Validation
      |
      v
Export
      |
      v
Optimization
      |
      v
Inference API
      |
      v
Docker
      |
      v
Deployment
```

This chapter covers the practical steps to get a trained CV model behind a working API. Advanced production concerns (CI/CD, monitoring, scaling, rollback strategy) belong to the later dedicated MLOps section — this is deliberately scoped to "get a model served correctly."

6.33.2 Python Inference Script

```python
# inference.py
import torch
from PIL import Image
import torchvision.transforms as T

class InferenceEngine:
    def __init__(self, checkpoint_path, num_classes, class_names, device="cpu"):
```

```
        self.device = device
        self.model = build_transfer_model(num_classes, freeze_backbone=False)
        self.model.load_state_dict(torch.load(checkpoint_path, map_location=device))
        self.model.to(device).eval()
        self.class_names = class_names
        self.transform = T.Compose([
            T.Resize(256), T.CenterCrop(224), T.ToTensor(),
            T.Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225]),
        ])

    @torch.no_grad()
    def predict(self, image: Image.Image):
        tensor = self.transform(image).unsqueeze(0).to(self.device)
        probs = torch.softmax(self.model(tensor), dim=1)[0]
        idx = probs.argmax().item()
        return {"class": self.class_names[idx], "confidence": probs[idx].item()}
```

6.33.3 FastAPI Inference Endpoint

```
# main.py
from fastapi import FastAPI, UploadFile, File, HTTPException
from PIL import Image
import io

app = FastAPI()
engine = InferenceEngine("best_model.pt", num_classes=5, class_names=["a", "b", "c", "d", "e"])

@app.post("/predict")
async def predict(file: UploadFile = File(...)):
    if file.content_type not in ("image/jpeg", "image/png"):
        raise HTTPException(status_code=400, detail="Only JPEG/PNG images are supported")
    try:
        image = Image.open(io.BytesIO(await file.read())).convert("RGB")
    except Exception:
        raise HTTPException(status_code=400, detail="Could not decode image")

    result = engine.predict(image)
    return result

@app.get("/health")
async def health():
    return {"status": "ok"}
```

- **Common Mistake**

Building an inference endpoint without input validation — an unreadable file, a non-image upload, or a corrupted image should return a clean 400-level error, not crash the process or return a confusing 500. Validate content type and wrap decoding in a try/except that returns an informative client error.

6.33.4 Docker

```
FROM python:3.11-slim

WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

COPY main.py inference.py best_model.pt ./

EXPOSE 8000
CMD ["uvicorn", "main:app", "--host", "0.0.0.0", "--port", "8000"]
```

```
docker build -t cv-inference-api .
docker run -p 8000:8000 cv-inference-api
```

6.33.5 ONNX Runtime and TFLite/LiteRT Serving

```
import onnxruntime as ort
import numpy as np

class ONNXInferenceEngine:
    def __init__(self, onnx_path, providers=None):
        providers = providers or ["CPUExecutionProvider"]
        self.session = ort.InferenceSession(onnx_path, providers=providers)
        self.input_name = self.session.get_inputs()[0].name

    def predict(self, preprocessed_array: np.ndarray):
        outputs = self.session.run(None, {self.input_name: preprocessed_array})
        return outputs[0]
```

6.33.6 Edge and Mobile Deployment (Concepts)

Target	Typical stack
NVIDIA Jetson / edge GPU	TensorRT engine, C++/Python runtime
Android	TFLite / LiteRT model bundled in the app
iOS	Core ML model bundled in the app
Coral Edge TPU	Edge TPU-compiled TFLite model

Mobile/edge deployment specifics (app packaging, on-device model updates, platform SDK integration) are beyond this chapter's scope — the key CV-specific takeaway is that export format and quantization strategy should be decided *early*, since they influence architecture choices (e.g. avoiding ops with poor mobile-runtime support) rather than being an afterthought at the end of a project.

• Best Practice

Test the exported/quantized model's predictions against the original PyTorch/TensorFlow model on a fixed set of sample inputs before considering deployment "done." Export and quantization can silently introduce small numerical differences that occasionally flip a prediction near a decision boundary — catching this in a pre-deployment validation step is far cheaper than catching it after a user-facing incident.

★ Quick Revision Sheet

CV Deployment, in one screen:

- Pipeline: train -> validate -> export -> optimize -> inference API -> Docker -> deploy.
- Always validate input (content type, decodability) at the API boundary — never let a bad upload crash the process.
- Docker + FastAPI + a saved checkpoint is a solid minimal deployment for most projects.
- Decide export format and quantization strategy early — it can constrain architecture choices.
- Always validate exported/quantized model output against the original before calling deployment done.

Practice Tasks

- **Level 1:** Add a `/health` endpoint to an existing FastAPI inference service.
- **Level 2:** Modify the `/predict` endpoint to return top-3 predictions instead of just the top-1.
- **Level 3:** Containerize a full inference service with Docker and confirm it runs correctly from a clean image.
- **Level 4:** Debug an endpoint that crashes on a corrupted upload — add proper input validation and error handling.
- **Level 5:** Benchmark request throughput of the FastAPI endpoint under concurrent load.
- **Level 6:** Set up an ONNX Runtime-backed version of the same endpoint and validate prediction parity against the PyTorch version.

Chapter 6.34 — Video Computer Vision

6.34.1 Video Fundamentals

A video is a sequence of frames, each an image, played at a given frame rate (frames per second, FPS). Processing every single frame through a CV model is often unnecessary and expensive — most CV-on-video applications process at a reduced effective rate via frame skipping.

```
import cv2

cap = cv2.VideoCapture("video.mp4")
fps = cap.get(cv2.CAP_PROP_FPS)
frame_count = int(cap.get(cv2.CAP_PROP_FRAME_COUNT))
width = int(cap.get(cv2.CAP_PROP_FRAME_WIDTH))
height = int(cap.get(cv2.CAP_PROP_FRAME_HEIGHT))
print(f"{frame_count} frames at {fps} FPS, resolution {width}x{height}")
```

6.34.2 Frame Skipping

```
def process_video_with_skip(video_path, model, skip_frames=2):
    """Run inference on every (skip_frames+1)-th frame instead of every frame."""
    cap = cv2.VideoCapture(video_path)
    frame_idx = 0
    results = []
    while cap.isOpened():
        ret, frame = cap.read()
        if not ret:
            break
        if frame_idx % (skip_frames + 1) == 0:
            detections = model(frame)
            results.append((frame_idx, detections))
        frame_idx += 1
    cap.release()
    return results
```

- **Best Practice**

Choose the frame-skip rate based on how fast the objects of interest actually move relative to the camera's field of view, not an arbitrary fixed number. A slow-moving object in a wide static shot tolerates aggressive skipping (e.g. processing every 5th frame) with no practical loss of detection quality; a fast-moving object in a tight shot needs near-every-frame processing to avoid missing it between processed frames entirely.

6.34.3 Real-Time Webcam Object Detection

```
Webcam
 |
 v
Frame Capture
 |
 v
YOLO
 |
 v
Detection
 |
 v
Display
```

```
from ultralytics import YOLO
import cv2

model = YOLO("yolo11n.pt")
cap = cv2.VideoCapture(0)

while cap.isOpened():
    ret, frame = cap.read()
    if not ret:
        break

    results = model(frame, verbose=False)
    annotated = results[0].plot()   # BGR frame with boxes/labels drawn

    cv2.imshow("Live Detection", annotated)
    if cv2.waitKey(1) & 0xFF == ord("q"):
        break

cap.release()
cv2.destroyAllWindows()
```

6.34.4 Tracking Preview

Detection alone re-identifies objects independently on every frame, with no memory of "this is the same object as last frame." Object tracking assigns a persistent ID across frames, which is necessary for tasks like counting unique objects crossing a line, or measuring an object's trajectory over time. A minimal approach pairs a detector with a lightweight

tracker (e.g. ByteTrack, integrated directly into modern `ultralytics` workflows):

```
results = model.track(source="video.mp4", persist=True, tracker="bytetrack.yaml")
for result in results:
    for box in result.boxes:
        track_id = int(box.id) if box.id is not None else None
        print(f"Object {track_id}: class={model.names[int(box.cls)]}")
```

Full tracking algorithm coverage (Kalman filters, re-identification embeddings, multi-object tracking evaluation metrics) is out of scope for this chapter — this is a working preview sufficient for straightforward counting/trajectory use cases.

6.34.5 Temporal Considerations

- **Flicker**: a detector's confidence can fluctuate frame-to-frame even for a stationary object, causing boxes to flicker in and out. Simple temporal smoothing (requiring N consecutive detections before confirming, as mentioned in Chapter 6.16's improvement tasks) reduces this.
- **Frame rate vs. inference speed mismatch**: if model inference is slower than the video's native frame rate, either drop frames to keep up (real-time priority) or process every frame at a slower-than-real-time pace (completeness priority) — an explicit design decision, not something to leave implicit.

6.34.6 Performance Bottlenecks

Bottleneck	Diagnosis	Fix
Slow frame read	`cap.read()` itself is slow (network stream, disk I/O)	Use a separate thread/process for frame reading, decoupled from inference
Slow inference	Model forward pass exceeds frame budget	Frame skipping, smaller model variant, lower resolution, GPU/TensorRT
Slow display/write	`imshow`/`imwrite` overhead accumulates	Only display/save a subset of frames, or write asynchronously

★ Quick Revision Sheet

Video CV, in one screen:

- Frame skipping trades detection latency/completeness for speed — tune to actual object motion, not an arbitrary constant.
- `model.track(persist=True)` adds persistent object IDs across frames for counting/trajectory use cases.
- Decouple frame reading from inference (separate threads) when I/O, not compute, is the bottleneck.
- Temporal smoothing (requiring N consecutive detections) reduces flicker in per-frame confidence.

Practice Tasks

- **Level 1:** Read a video file's FPS, frame count, and resolution using OpenCV.
- **Level 2:** Modify `process_video_with_skip` to adapt the skip rate based on measured inference latency.
- **Level 3:** Implement a full webcam detection + tracking script that counts unique objects crossing a horizontal line.

- **Level 4:** Debug a live-detection script with severe display lag — profile to determine whether frame capture, inference, or display is the bottleneck.
- **Level 5:** Compare detection completeness (objects missed) at different frame-skip rates on a fixed test video.
- **Level 6:** Build a threaded video-processing pipeline that decouples frame capture from inference to maximize throughput.

Chapter 6.35 — Computer Vision Projects

This chapter presents eight progressively difficult, portfolio-ready projects spanning the techniques covered in this part. Each follows the same specification structure introduced in Chapter 6.16. Where a project reuses code already built in an earlier chapter, this chapter references it directly rather than repeating it, per the handbook's no-padding principle.

Project 1 — Image Classification From Scratch (CIFAR-10)

Problem statement: Classify 32x32 natural images into 10 categories using a CNN trained from random initialization — no transfer learning. **Architecture:** `SimpleCNN` from Chapter 6.5. **Why:** CIFAR-10 is small and simple enough that a from-scratch CNN trains well within a short time budget, and skipping transfer learning here is deliberate — the point of this project is understanding the full training loop, not maximizing accuracy. **Difficulty:** Beginner. **Estimated time:** 1-2 hours. **Portfolio value:** Moderate — table-stakes for demonstrating you understand the fundamentals, expected as a baseline rather than a differentiator on its own.

Dataset Field	Value
Dataset	CIFAR-10
Official source / URL	https://www.cs.toronto.edu/~kriz/cifar.html
Samples	60,000 (50,000 train / 10,000 test)
Classes	10
Input size	32x32x3
Format	Python pickle / binary batches
Preprocessing	Scale to `[0,1]`, no resize needed
Augmentation	Random crop with padding, horizontal flip
Evaluation	Accuracy, confusion matrix

Environment Field	Value
Python	3.10+
Framework	PyTorch
GPU	Optional — trains in a reasonable time on CPU given the small image size, faster on any GPU
VRAM	2GB+ if using GPU
RAM	8GB
Storage	~200MB
Estimated training time	10-20 minutes for 20 epochs on a single GPU; 1-2 hours on CPU

```
import torchvision
import torchvision.transforms as T

train_transform = T.Compose([
    T.RandomCrop(32, padding=4),
    T.RandomHorizontalFlip(),
    T.ToTensor(),
    T.Normalize((0.4914, 0.4822, 0.4465), (0.2470, 0.2435, 0.2616)),  # CIFAR-10-specific stats
])
test_transform = T.Compose([
    T.ToTensor(),
    T.Normalize((0.4914, 0.4822, 0.4465), (0.2470, 0.2435, 0.2616)),
])

train_set = torchvision.datasets.CIFAR10(root="./data", train=True, download=True, transform=train_transform)
test_set = torchvision.datasets.CIFAR10(root="./data", train=False, download=True, transform=test_transform)

train_loader = torch.utils.data.DataLoader(train_set, batch_size=128, shuffle=True, num_workers=4)
test_loader = torch.utils.data.DataLoader(test_set, batch_size=128, shuffle=False, num_workers=4)

model = SimpleCNN(num_classes=10).to(device)   # from Chapter 6.5
# training loop: reuse train_one_epoch / validate from Chapter 6.10
```

Evaluation & debugging: confirm accuracy exceeds ~70% within 20 epochs — if it doesn't, check normalization stats and that `RandomCrop` uses padding (unpadded crop of an already-32px image discards too much information). **Deployment:** export to ONNX and confirm parity, as a lightweight exercise — this project doesn't warrant a full production deployment.

Improvement tasks: add MixUp/CutMix (Chapter 6.4) and compare accuracy; try a deeper `SimpleCNN` variant; compare against a from-scratch small ResNet.

Interview questions:

IQ1. Why is a from-scratch CNN a reasonable choice for CIFAR-10 but usually a poor choice for a 500-image custom dataset? **DIFFICULTY: EASY** *Companies: General* CIFAR-10 has 50,000 training images — enough to learn useful features from scratch. A 500-image dataset is far too small to learn general visual features without overfitting badly; transfer learning from an ImageNet-pretrained backbone (Chapter 6.9) is the correct default there instead.

Project 2 — Transfer-Learning Image Classifier (Custom Categories)

Problem statement: Classify images into custom categories not present in ImageNet's 1,000 classes (e.g. specific plant species, product types, or architectural styles), using a small labeled dataset (a few hundred images per class). **Architecture:** `build_transfer_model` (ResNet50) from Chapter 6.9. **Why:** Small dataset, high domain similarity to natural photos — the textbook transfer-learning scenario. **Alternatives:** EfficientNet-B0 for a lighter-weight option with comparable accuracy. **Difficulty:** Beginner-intermediate. **Estimated time:** 2-4 hours including data collection/organization. **Portfolio value:** High — demonstrates the most commonly-used real-world CV workflow end-to-end.

Dataset Field	Value
Dataset	Oxford-IIIT Pet Dataset (example stand-in for any custom-category classification task)
Official source / URL	https://www.robots.ox.ac.uk/~vgg/data/pets/
Samples	~7,400 images
Classes	37 (pet breeds)
Input size	Variable, resized to 224x224
Format	JPEG + XML/text annotations
License	See dataset site (research use)
Preprocessing	Resize(256) -> CenterCrop(224), ImageNet normalization
Augmentation	RandomResizedCrop, horizontal flip, color jitter
Evaluation	Accuracy, per-class precision/recall, confusion matrix

Environment Field	Value
Python	3.10+
Framework	PyTorch, torchvision
GPU	Recommended, 4GB+ VRAM
RAM	8GB
Storage	~1GB
Estimated training time	20-40 minutes for 15 epochs (head-only + fine-tune stages) on a single GPU

```
# Reuses ImageFolderDataset (Chapter 6.10), build_transfer_model and the two-stage
# workflow (Chapter 6.9), and train_one_epoch/validate (Chapter 6.10).

model = build_transfer_model(num_classes=37, freeze_backbone=True)
# Stage 1: train head only, then unfreeze layer4 and fine-tune with discriminative LR — see Chapter 6.9.
```

Error analysis: inspect the confusion matrix for visually similar breed pairs — these account for the large majority of errors in this kind of fine-grained classification task, and are a fundamentally different (and more fixable, via more training examples of the confused pairs) problem than errors spread randomly.

Deployment: wrap in `ProductionClassifier` (Chapter 6.10) and serve via the FastAPI pattern from Chapter 6.33.

Improvement tasks: try EfficientNet-B0 instead of ResNet50 and compare accuracy/latency; add test-time augmentation (average predictions over several augmented views) and measure the accuracy gain.

Interview questions:

IQ2. In a fine-grained classification task (e.g. distinguishing 37 similar pet breeds), why might per-class precision/recall be more useful than overall accuracy for debugging? **DIFFICULTY: MEDIUM** *Companies: General*
Overall accuracy hides which specific class pairs the model confuses. Per-class metrics and the confusion matrix reveal that errors are concentrated on a few visually similar pairs rather than spread randomly — directly pointing toward the fix (more/better training data for those specific pairs) rather than a generic "train longer" response. ## Project 3 — Custom YOLO Detector

See Chapter 6.16 for the complete specification, dataset, environment, training, evaluation, error analysis, export, and deployment coverage for this project (the warehouse PPE detector). This entry exists in the index (Chapter 6.35's final index and Chapter 6.38's project index) for completeness alongside the other seven projects.

Project 4 — Real-Time Webcam Detection with Counting

Problem statement: Detect and count objects (e.g. vehicles, people) crossing a defined line in a live webcam or video feed, in real time. **Architecture:** Pretrained YOLO11n + ByteTrack (Chapter 6.34). **Why:** Real-time constraint rules out two-stage detectors; nano-size YOLO with tracking is the standard practical choice for this exact use case. **Difficulty:** Intermediate. **Estimated time:** 2-3 hours. **Portfolio value:** High — combines detection, tracking, and a genuine real-time systems constraint, a common interview/take-home project theme.

Dataset Field	Value
Dataset	None required for training — uses YOLO11's COCO-pretrained weights directly (COCO includes `person`, `car`, etc.)
Input	Live webcam feed or video file
Evaluation	Manual count validation against a short reference video with a known ground-truth count

Environment Field	Value
Python	3.10+
Framework	`ultralytics`, OpenCV
GPU	Optional — YOLO11n runs in real time on a modern CPU at reduced resolution; a GPU gives headroom for higher resolution/larger model variants
RAM	8GB
Storage	Minimal (pretrained weights only, ~5-10MB for the nano variant)

```
from ultralytics import YOLO
import cv2

model = YOLO("yolo11n.pt")
LINE_Y = 300   # horizontal counting line, in pixel coordinates
counted_ids = set()
count = 0

cap = cv2.VideoCapture(0)
while cap.isOpened():
    ret, frame = cap.read()
    if not ret:
        break

    results = model.track(frame, persist=True, tracker="bytetrack.yaml", classes=[0], verbose=False)
    for box in results[0].boxes:
        if box.id is None:
            continue
        track_id = int(box.id)
```

```
        x1, y1, x2, y2 = box.xyxy[0].tolist()
        cy = (y1 + y2) / 2
        if cy > LINE_Y and track_id not in counted_ids:
            counted_ids.add(track_id)
            count += 1

    annotated = results[0].plot()
    cv2.line(annotated, (0, LINE_Y), (annotated.shape[1], LINE_Y), (0, 0, 255), 2)
    cv2.putText(annotated, f"Count: {count}", (20, 40), cv2.FONT_HERSHEY_SIMPLEX, 1, (0, 255, 0), 2)
    cv2.imshow("Counter", annotated)
    if cv2.waitKey(1) & 0xFF == ord("q"):
        break

cap.release()
cv2.destroyAllWindows()
```

- **Common Mistake**

Counting every frame a tracked object's box crosses the line, instead of tracking which IDs have already been counted. Without the `counted_ids` set, an object lingering near the line boundary gets counted many times in a row as its box flickers across the threshold pixel-by-pixel.

Debugging: if the same object gets a new track ID after brief occlusion (double-counted), tune the tracker's association/re-identification parameters, or accept a small known error margin and document it as a limitation.

Improvement tasks: add multi-line counting (in vs. out), export counts to a time-series log for later analysis, add a minimum-dwell-time filter to avoid counting objects that briefly enter and leave the frame edge.

Project 5 — U-Net Medical/Object Segmentation

Problem statement: Segment a foreground object (e.g. a specific anatomical structure in a medical scan, or a product in a retail shelf photo) from its background at the pixel level. **Architecture:** `UNet` from Chapter 6.22. **Why:** U-Net remains a strong, well-understood default for binary/few-class segmentation tasks, especially with moderate dataset sizes. **Difficulty:** Intermediate-advanced. **Estimated time:** 3-5 hours.

Dataset Field	Value
Dataset	Carvana Image Masking Challenge (example stand-in; substitute a domain-specific dataset as needed)
Official source / URL	https://www.kaggle.com/c/carvana-image-masking-challenge
Task	Binary segmentation (foreground object vs. background)
Format	JPEG images + PNG (or gif) masks
License	Kaggle competition terms — verify before any commercial use
Preprocessing	Resize to a fixed size (e.g. 256x256), mask resized with nearest-neighbor
Augmentation	Joint image+mask flip/rotation via `albumentations` (Chapter 6.23)
Evaluation	Dice coefficient, IoU

Environment Field	Value
Python	3.10+
Framework	PyTorch, `albumentations`
GPU	Recommended, 6GB+ VRAM for reasonable batch sizes at 256x256
RAM	16GB
Storage	2-5GB depending on dataset size
Estimated training time	1-3 hours for 50 epochs on a single GPU

```
# Reuses SegmentationDataset, UNet, combined_loss, and segmentation_iou from Chapters 6.22-6.23.

model = UNet(in_channels=3, num_classes=1).to(device)
optimizer = torch.optim.Adam(model.parameters(), lr=1e-4)

for epoch in range(50):
    model.train()
    for images, masks in train_loader:
        images, masks = images.to(device), masks.to(device)
        optimizer.zero_grad()
        logits = model(images)
        loss = combined_loss(logits, masks.unsqueeze(1))
        loss.backward()
        optimizer.step()
```

Error analysis: visualize predicted masks overlaid on images (`overlay_mask`, Chapter 6.23) for the worst-Dice examples specifically — boundary errors (slightly-off edges) and gross errors (missed object entirely) point to different fixes: boundary errors often improve with more training or a loss-function adjustment (e.g. boundary-weighted loss), while gross misses often indicate a genuinely hard/ambiguous example worth a second annotation review.

Deployment: export to ONNX; for medical or otherwise high-stakes domains, always pair automated segmentation with a human-review step rather than fully automating downstream decisions.

Improvement tasks: try DeepLab as an alternative architecture and compare; add test-time augmentation; experiment with a boundary-aware loss term for sharper edges. ## Project 6 — OCR Document Extraction

Problem statement: Extract structured text (e.g. invoice fields, receipt line items) from photographed or scanned documents. **Architecture:** OpenCV preprocessing + Tesseract, per Chapters 6.26-6.27. **Why:** Tesseract remains a solid, free, well-documented choice for general document OCR; the preprocessing pipeline is the real differentiator in accuracy. **Difficulty:** Intermediate. **Estimated time:** 2-4 hours. **Portfolio value:** High — document processing is a common, immediately-relatable business use case.

Dataset Field	Value
Dataset	A small self-collected set of receipt/invoice photos (10-50 images sufficient for a demonstration project), or a public dataset such as SROIE for a more rigorous benchmark
Official source / URL (SROIE)	https://rrc.cvc.uab.es/?ch=13
Format	Images + ground-truth text/field annotations (for SROIE)
Evaluation	Character/word error rate against ground truth; field-level extraction accuracy

Environment Field	Value
Python	3.10+
Framework	`pytesseract`, OpenCV
GPU	Not required
RAM	4GB
Storage	Minimal

```
import re

def extract_invoice_fields(image_path):
    results = full_ocr_pipeline(image_path)   # from Chapter 6.26
    full_text = " ".join(r["text"] for r in results)

    total_match = re.search(r"total[:\s]*\$?(\d+\.\d{2})", full_text, re.IGNORECASE)
    date_match = re.search(r"\d{1,2}[/-]\d{1,2}[/-]\d{2,4}", full_text)

    return {
        "total": total_match.group(1) if total_match else None,
        "date": date_match.group(0) if date_match else None,
        "raw_text": full_text,
    }
```

• Best Practice

For structured field extraction (totals, dates, invoice numbers), combine OCR's raw text output with regex or a small rules layer rather than expecting the OCR engine itself to understand document structure — Tesseract recognizes characters, it doesn't understand semantics. For genuinely complex, varied document layouts, a dedicated document-understanding model is the right upgrade path beyond this project's scope.

Debugging: if field extraction fails on documents that OCR correctly, the bug is almost always in the regex/rules layer, not the OCR step — verify by inspecting `raw_text` directly before touching the extraction patterns.

Deployment: wrap in a FastAPI endpoint (Chapter 6.33 pattern) accepting an uploaded document image and returning structured JSON.

Improvement tasks: add layout-aware extraction (using `image_to_data`'s bounding boxes to associate labels with nearby values spatially, not just via regex on flattened text); add a confidence-based human-review flag for low-confidence extractions.

Project 7 — Production-Style CV API

Problem statement: Wrap any of the above models (classifier, detector, or segmentation model) in a properly production-structured API: input validation, error handling, logging, health checks, and containerization. **Architecture:** FastAPI + Docker, per Chapter 6.33. **Why:** This project is specifically about production engineering discipline, not about model novelty — reuse a model from an earlier project. **Difficulty:** Intermediate. **Estimated time:** 2-3 hours. **Portfolio value:** High — this is precisely the kind of project that differentiates "I trained a model in a notebook" from "I can ship a model."

Environment Field	Value
Python	3.10+
Framework	FastAPI, Docker
GPU	Depends on the wrapped model — CPU-only is fine for a small classifier
RAM	4-8GB
Storage	Model checkpoint size + a few hundred MB for the container image

```python
# main.py — extends the pattern from Chapter 6.33 with logging and structured errors
import logging
from fastapi import FastAPI, UploadFile, File, HTTPException
from fastapi.responses import JSONResponse

logging.basicConfig(level=logging.INFO)
logger = logging.getLogger("cv-api")

app = FastAPI(title="CV Inference API", version="1.0")

@app.exception_handler(Exception)
async def unhandled_exception_handler(request, exc):
    logger.exception("Unhandled error during request")
    return JSONResponse(status_code=500, content={"detail": "Internal server error"})

@app.post("/predict")
async def predict(file: UploadFile = File(...)):
    logger.info(f"Received prediction request: {file.filename}")
    if file.content_type not in ("image/jpeg", "image/png"):
        raise HTTPException(status_code=400, detail="Only JPEG/PNG images are supported")
    # ... decode, validate, run inference (Chapter 6.33 pattern) ...
    logger.info("Prediction complete")
    return {"result": "..."}

@app.get("/health")
async def health():
    return {"status": "ok"}

@app.get("/ready")
async def ready():
    # confirm the model is actually loaded and able to run a trivial forward pass
    return {"status": "ready"}
```

Improvement tasks: add request-ID tracing through logs; add a `/metrics` endpoint (request count, average latency, error rate) for basic observability; add rate limiting.

Interview questions:

IQ3. Why separate `/health` and `/ready` endpoints instead of one combined check? DIFFICULTY: MEDIUM
Companies: General `/health` typically confirms the process is running and responsive at all (useful for basic liveness checks); `/ready` confirms the service has finished loading its model and dependencies and can actually serve a real

request. A process can be alive but not yet ready (e.g. during a slow model load at startup) — conflating the two can cause an orchestrator to route traffic to an instance that isn't actually able to serve it yet.

Project 8 — Portfolio-Grade End-to-End CV System: Retail Shelf Analyzer

Problem statement: Given a photo of a retail shelf, detect individual products, classify each into a product category, and report shelf-level analytics (product counts, out-of-stock gaps, planogram compliance at a basic level). **Architecture:** YOLO detector (product localization) + a fine-tuned classifier (fine-grained category) chained together, served via a single API. **Why this combination:** Detection alone can't distinguish fine-grained product categories reliably at typical shelf-photo resolution; a two-stage detect-then-classify pipeline (detect all "product" boxes, then classify each cropped box) is a standard, practical pattern for exactly this kind of fine-grained-plus-localization problem. **Difficulty:** Advanced. **Estimated time:** 6-10 hours. **Portfolio value:** Highest of the eight — demonstrates architectural judgment (why two chained models, not one), full-pipeline engineering, and a directly business-relevant use case.

```
Shelf Photo
     |
     v
YOLO: detect all "product" bounding boxes
     |
     v
Crop each detected box
     |
     v
Classifier: fine-grained category per crop
     |
     v
Aggregate: counts per category, gap detection (large uncovered shelf regions), summary report
```

Field	Value
Detection dataset	A custom-annotated shelf-photo set (product-level boxes only, class-agnostic "product" label)
Classification dataset	Cropped product images labeled by fine-grained category (can reuse/extend Project 2's workflow)
Environment	Same as Projects 2-4 combined — PyTorch, `ultralytics`, GPU recommended for both training and reasonably fast inference
Estimated training time	Detector: 1-3 hours; classifier: 20-40 minutes; combined pipeline validation: additional 1-2 hours

```python
def analyze_shelf(image_path, detector, classifier):
    results = detector.predict(image_path, conf=0.4)
    image = cv2.imread(image_path)
    report = {"total_products": 0, "by_category": {}}

    for box in results[0].boxes:
        x1, y1, x2, y2 = map(int, box.xyxy[0].tolist())
        crop = image[y1:y2, x1:x2]
```

```
        crop_pil = Image.fromarray(cv2.cvtColor(crop, cv2.COLOR_BGR2RGB))
        prediction = classifier.predict(crop_pil)   # InferenceEngine-style predict, Chapter 6.33

        report["total_products"] += 1
        cat = prediction["class"]
        report["by_category"][cat] = report["by_category"].get(cat, 0) + 1

    return report
```

Industry Case Study: Retail Shelf Analytics

A grounded version of this pipeline: a retailer wants automated out-of-stock detection across store shelves. The detect-then-classify architecture described here is the standard practical pattern rather than a single end-to-end detector directly predicting fine-grained SKU classes, because product catalogs change far more often than it's practical to retrain a large detector — retraining only the lightweight classifier when new products launch, while the "detect a product-shaped region" detector stays stable, is a significant operational advantage. Failure modes in practice include heavy occlusion between adjacent products, reflective/glossy packaging confusing the classifier, and extreme lighting variation across different store locations — all of which argue for a diverse training set collected across multiple real stores rather than one clean reference set.

Improvement tasks: add a lightweight gap-detection heuristic (large horizontal regions of shelf with no detected product); add confidence-based routing to human review for low-confidence classifications; evaluate end-to-end pipeline accuracy (not just each model's standalone metric) since errors compound across the two stages.

Final Project Index

Project	Dataset	Task	Model	Framework	Difficulty	GPU	Deployment
1. CIFAR-10 Classifier	CIFAR-10	Classification	SimpleCNN	PyTorch	Beginner	Optional	ONNX export exercise
2. Transfer-Learning Classifier	Oxford-IIIT Pets (example)	Classification	ResNet50 (transfer)	PyTorch	Beginner-Int.	Recommended	FastAPI
3. Custom YOLO Detector	Roboflow PPE dataset	Detection	YOLO11	Ultralytics	Intermediate	Recommended	ONNX/TensorRT
4. Webcam Counter	None (pretrained)	Detection + tracking	YOLO11n + ByteTrack	Ultralytics	Intermediate	Optional	Local real-time app
5. U-Net Segmentation	Carvana (example)	Segmentation	U-Net	PyTorch	Int.-Advanced	Recommended	ONNX export
6. OCR Extraction	Self-collected / SROIE	OCR	Tesseract + rules	pytesseract	Intermediate	Not required	FastAPI

Project	Dataset	Task	Model	Framework	Difficulty	GPU	Deployment
7. Production CV API	Any prior model	Serving	FastAPI + Docker	FastAPI	Intermediate	Depends on model	Docker container
8. Retail Shelf Analyzer	Custom shelf + product datasets	Detection + Classification	YOLO11 + ResNet50	PyTorch/Ultralytics	Advanced	Recommended	FastAPI, chained models

Organized by track:

- **Beginner:** Projects 1, 2
- **Intermediate:** Projects 3, 4, 6, 7
- **Advanced:** Projects 5, 8
- **Portfolio standout:** Project 8
- **Hackathon/industry-style:** Project 4 (real-time constraint, demoable live) and Project 6 (immediately relatable business value)

★ Quick Revision Sheet

CV Projects, in one screen:

- Progress: from-scratch classifier -> transfer learning -> detection -> tracking -> segmentation -> OCR -> production API -> chained multi-model system.
- Reuse code across projects rather than re-deriving it — the project chapter is about applying, not repeating, earlier chapters.
- Project 8's detect-then-classify pattern is the standard real-world answer to "fine-grained classification at unknown locations in a larger image."
- Every project needs a genuine evaluation and error-analysis step, not just a training run.

Chapter 6.36 — Computer Vision Model Selection

6.36.1 Decision Tree

```
What is the task?
       |
       +-- Classification
       |      +-- Small dataset -> Transfer Learning (frozen or partially fine-tuned backbone)
       |      +-- Large dataset -> Larger CNN, more freedom to fully fine-tune or train from scratch
       |
       +-- Detection
       |      +-- Real-time / edge deployment -> YOLO
       |      +-- Accuracy-focused, offline -> Faster R-CNN / RetinaNet
       |      +-- Severe class imbalance, small/dense objects -> RetinaNet specifically
       |      +-- Legacy/embedded constraint -> SSD
       |
       +-- Segmentation
       |      +-- Semantic -> U-Net / DeepLab
       |      +-- Instance -> Mask R-CNN
       |
       +-- Text in image
              +-- OCR pipeline (preprocessing + Tesseract, or a dedicated OCR model for complex layouts)
```

- **Best Practice**

Treat this decision tree as a starting point for a conversation, not a lookup table with one correct answer. Real projects usually have at least two competing constraints (e.g. ”real-time AND maximum accuracy”) that the tree doesn’t resolve on its own — resolving that tension with an explicit tradeoff (a smaller high-accuracy model, a two-tier pipeline, a relaxed latency budget) is the actual engineering judgment being exercised.

6.36.2 Choosing a Backbone (Classification and as a Detector/Segmenter Base)

Constraint	Backbone choice
General-purpose baseline, no strong constraint	ResNet50 or EfficientNet-B0
Mobile/edge, latency-critical	MobileNetV3
Best accuracy achievable, compute is not the constraint	ConvNeXt or a larger EfficientNet variant

Constraint	Backbone choice
Memory-constrained training (not just inference)	MobileNet or a smaller EfficientNet variant

6.36.3 Choosing a Detector

Revisit the full comparison and decision checklist in Chapter 6.21 — the short version: YOLO is the default; deviate only when a specific constraint (severe imbalance, legacy hardware, offline maximum-accuracy priority) points elsewhere.

6.36.4 Choosing a Segmentation Approach

Scenario	Choice
Single or few classes, moderate dataset size	U-Net
Many classes, need strong multi-scale context (e.g. urban scene parsing)	DeepLab
Need to distinguish individual object instances, not just classes	Mask R-CNN

6.36.5 A Practical Selection Checklist

```
1. What is the actual task? (classification / detection / segmentation / OCR)
2. What is the latency budget? (real-time, near-real-time, offline/batch)
3. What is the deployment target? (server GPU, server CPU, mobile, edge device)
4. How large and how similar-to-ImageNet is the dataset?
5. Is there severe class imbalance or a small-object-dominant distribution?
6. What deployment tooling/export format does the target actually support?
```

Answering these six questions before naming an architecture produces a defensible, constraint-driven recommendation — the same discipline tested in Chapter 6.21's system-design interview trap.

★ Quick Revision Sheet

Model Selection, in one screen:

- Task first, then dataset size/similarity, then latency budget, then deployment target — in that order.
- YOLO default for detection; RetinaNet for imbalance/small-objects; Faster R-CNN for offline max-accuracy; SSD for legacy constraints only.
- U-Net/DeepLab for semantic segmentation; Mask R-CNN when individual instances matter.
- Always resolve competing constraints explicitly rather than picking one and ignoring the other.

Practice Tasks

- **Level 1:** Walk through the decision tree for a hypothetical "detect defects on a factory conveyor belt in real time" scenario.
- **Level 2:** Walk through it for "segment tumors in offline-processed MRI scans, accuracy is the only priority."
- **Level 3:** Write a one-page justified model recommendation for a real or hypothetical CV problem of your choosing.

- **Level 4:** Given a scenario with a stated but infeasible combination of constraints, identify the infeasibility and propose a resolution.
- **Level 5:** Compare two defensible model choices for the same scenario and articulate the tradeoff between them.
- **Level 6:** Build a lightweight internal decision-support doc (like this chapter) tailored to your own team's typical deployment constraints.

Chapter 6.37 — Computer Vision Interview Preparation

60 questions spanning conceptual understanding, coding, debugging, architecture selection, and system design. Answers are intentionally concise — this is a revision reference, not a substitute for the full chapters.

CNN Fundamentals

IQ1. What does a convolutional kernel actually compute? **DIFFICULTY: EASY** *Companies: General* A weighted sum of the input values in a local receptive-field window, producing one output value per spatial position — the same learned weights are reused (shared) across every position, which is what makes convolution parameter-efficient compared to a fully-connected layer.

IQ2. Why does padding matter, and what does "same" padding mean? **DIFFICULTY: EASY** *Companies: General* Without padding, each convolution shrinks the spatial size, and repeated layers quickly reduce a feature map to nothing. "Same" padding adds enough border zeros so the output spatial size matches the input (for stride 1), letting you control downsampling explicitly via stride/pooling instead of it happening as an unavoidable side effect of every conv layer.

IQ3. What's the effect of stride 2 vs. stride 1 in a convolution? **DIFFICULTY: EASY** *Companies: General* Stride 2 moves the kernel two positions at a time, roughly halving the output spatial resolution and reducing compute — commonly used for downsampling instead of, or alongside, pooling.

IQ4. Why do modern CNNs often use several stacked 3x3 convolutions instead of one larger kernel? **DIFFICULTY: MEDIUM** *Companies: Google, Meta* Two stacked 3x3 convolutions have the same receptive field as one 5x5, but fewer parameters (`2*(3*3) = 18` vs. `5*5 = 25` per channel pair) and an extra nonlinearity between them, giving the network more representational capacity for a similar or lower compute cost.

IQ5. What is the receptive field, and why does it grow with depth? **DIFFICULTY: MEDIUM** *Companies: General* The receptive field is the region of the original input that can influence one output unit. Each additional convolutional layer lets that unit "see" a wider input region than the layer before it, because it's computed from a window of the previous layer's outputs, each of which already aggregated its own local window.

IQ6. Why use global average pooling instead of flattening before the final classifier? **DIFFICULTY: MEDIUM** *Companies: General* Flattening a large feature map into a fully-connected layer creates a huge number of parameters and fixes the required input resolution. Global average pooling collapses each channel to a single value, drastically cutting parameters and letting the same architecture handle variable input sizes.

IQ7. What's the difference between max pooling and average pooling, and when would you prefer one? **DIFFICULTY: MEDIUM** *Companies: General* Max pooling keeps the strongest activation in each window (good for detecting "is this feature present anywhere in this region"); average pooling smooths over the window (can lose sharp, localized signals). Max pooling is the more common default for intermediate downsampling; average pooling is standard for the final global reduction before classification.

IQ8. Why does BatchNorm behave differently in train vs. eval mode, and what breaks if you forget `model.eval()`? **DIFFICULTY: MEDIUM** *Companies: Amazon, Meta* In train mode, BatchNorm normalizes using the current batch's statistics and updates a running average; in eval mode it uses the accumulated running statistics instead. Forgetting `model.eval()` at inference/validation time means predictions depend on whatever batch happens to be passed in, producing noisy, non-reproducible, and often noticeably worse results.

Architectures & Transfer Learning

IQ9. What problem do residual (skip) connections solve? **DIFFICULTY: MEDIUM** *Companies: Google, Meta, Amazon* They address the degradation problem in very deep plain networks, where adding more layers makes training accuracy worse (not just a vanishing-gradient issue, though that's related). The `F(x) + x` formulation gives the network an easy path to an identity mapping and gives gradients a direct route back through the shortcut.

IQ10. Why does ResNet50 use a bottleneck block (1x1-3x3-1x1) instead of two stacked 3x3s like ResNet18/34? **DIFFICULTY: HARD** *Companies: Google* The bottleneck first reduces channel depth with a 1x1 conv, applies the expensive 3x3 conv at that reduced depth, then restores depth with another 1x1 — cutting compute substantially at greater depths while preserving representational capacity, which is what makes deeper variants (50/101/152 layers) computationally feasible.

IQ11. What is depthwise separable convolution, and why does it matter for mobile deployment? **DIFFICULTY: MEDIUM** *Companies: General* It splits a standard convolution into a depthwise step (per-channel spatial filtering) and a pointwise 1x1 step (channel mixing), cutting compute roughly 8-9x for typical 3x3 kernels versus a standard convolution — the core trick that makes MobileNet efficient enough for phones and edge devices.

IQ12. What does EfficientNet's "compound scaling" mean? **DIFFICULTY: HARD** *Companies: Google* Instead of arbitrarily scaling depth, width, or input resolution independently, EfficientNet scales all three together by a fixed coefficient, found empirically to produce better accuracy-per-FLOP than scaling any single dimension alone.

IQ13. When would you freeze the entire backbone during transfer learning vs. fully fine-tune it? **DIFFICULTY: MEDIUM** *Companies: General* Freeze the whole backbone for small datasets similar in domain to the pretraining data (natural photos vs. ImageNet). Fully fine-tune for larger datasets or domains very different from the pretraining distribution (medical, satellite), where the pretrained features need real adaptation.

IQ14. Why use a much smaller learning rate when fine-tuning a pretrained model than when training from scratch? **DIFFICULTY: MEDIUM** *Companies: General* A large learning rate applied to already-good pretrained weights can destroy the useful features they encode within a few batches ("catastrophic forgetting"). A small learning rate lets the model adapt gradually instead.

IQ15. What are discriminative learning rates, and why apply smaller rates to earlier layers? **DIFFICULTY: HARD** *Companies: Meta* Discriminative learning rates apply different learning rates per layer group — smaller for early layers (which learn generic, broadly-transferable features like edges and textures) and larger for later layers and the new head (which need to adapt more to the specific task).

Detection

IQ16. What is IoU, and why is it the standard box-matching metric? **DIFFICULTY: EASY** *Companies: General* Intersection over Union: the overlap area between two boxes divided by their combined (union) area. It's scale-invariant and intuitively captures "how well do these two boxes agree," making it the standard way to decide whether a prediction matches a ground-truth box or whether two predictions are duplicates.

IQ17. Walk through what Non-Maximum Suppression does and why it's necessary. **DIFFICULTY: MEDIUM** *Companies: General* A detector's raw output has many overlapping candidate boxes around the same object. NMS sorts

candidates by confidence, keeps the highest-confidence box, discards other boxes that overlap it above an IoU threshold, and repeats — leaving roughly one box per real object instead of a cluster of duplicates.

IQ18. What's the difference between the confidence threshold and the IoU threshold in detection post-processing? **DIFFICULTY: MEDIUM** *Companies: General* The confidence threshold filters out low-confidence raw predictions before NMS even runs. The IoU threshold, used within NMS, controls how much overlap is tolerated before two boxes are considered duplicates of each other. They're independent knobs with different effects.

IQ19. Why is YOLO called a "one-stage" detector, and how does that differ from Faster R-CNN? **DIFFICULTY: EASY** *Companies: General* YOLO predicts boxes, classes, and confidences directly from a single forward pass. Faster R-CNN is two-stage: a Region Proposal Network first generates candidate regions, then a second stage classifies and refines each one — a more expensive but often more accurate approach, especially on small/crowded objects.

IQ20. Why can Faster R-CNN be slower than YOLO even though both use a CNN backbone? **DIFFICULTY: MEDIUM** *Companies: Google, Amazon* Faster R-CNN runs two networks sequentially (the RPN, then a per-region classification/regression head), and the per-region head's cost scales with however many proposals the RPN generates — a variable, often large number — rather than being a single fixed-cost forward pass like a one-stage detector.

IQ21. What problem does focal loss solve, and how? **DIFFICULTY: HARD** *Companies: Google, Meta* Dense one-stage detectors evaluate huge numbers of mostly-background candidate locations; standard cross-entropy lets this flood of easy background examples dominate the loss. Focal loss multiplies cross-entropy by `(1-p_t)^gamma`, which shrinks toward zero for easy, well-classified examples, letting the loss focus on hard examples instead.

IQ22. Why does RetinaNet use a Feature Pyramid Network? **DIFFICULTY: MEDIUM** *Companies: General* FPN combines high-resolution, spatially precise early-layer features with low-resolution, semantically rich late-layer features via a top-down pathway with lateral connections, giving every pyramid level good coverage for both small and large objects.

IQ23. What's the practical difference between anchor-based and anchor-free detection? **DIFFICULTY: MEDIUM** *Companies: General* Anchor-based detectors predict offsets relative to a predefined set of box shapes tiled across the image, requiring anchor sizes to be tuned to the dataset (often via k-means). Anchor-free detectors predict box coordinates or distances directly, simplifying configuration at the cost of some design flexibility.

IQ24. In what scenario would you choose SSD over a modern YOLO variant today? **DIFFICULTY: HARD** *Companies: General* Essentially only when a specific deployment constraint — mature, well-tested SSD support in a legacy or fixed-function embedded toolchain that lacks equivalent support for newer YOLO architectures — outweighs SSD's generally weaker accuracy, especially on small objects. ## Segmentation

IQ25. What's the difference between semantic and instance segmentation? **DIFFICULTY: EASY** *Companies: General* Semantic segmentation labels every pixel with a class but doesn't distinguish separate objects of the same class. Instance segmentation additionally separates individual object instances, each with its own mask.

IQ26. Why does U-Net use skip connections between the encoder and decoder? **DIFFICULTY: MEDIUM** *Companies: General* The encoder compresses spatial detail as it downsamples; without skip connections, the decoder would have to reconstruct precise object boundaries purely from the heavily-compressed bottleneck representation. Concatenating each encoder stage's features into the matching-resolution decoder stage restores that lost spatial detail.

IQ27. Why is Dice loss often preferred over plain cross-entropy for segmentation? **DIFFICULTY: MEDIUM** *Companies: Meta, medical imaging companies* Dice loss directly optimizes mask overlap and handles foreground/background class imbalance (a small object against a large background) better than pixel-wise cross-entropy, which can be dominated by the easy, large background class.

IQ28. How does Mask R-CNN extend Faster R-CNN? **DIFFICULTY: MEDIUM** *Companies: General* It adds a third parallel head — a small fully-convolutional network that predicts a per-instance binary mask — alongside the existing classification and box-regression heads, all operating on ROI-Align'd features.

IQ29. Why does Mask R-CNN use ROI Align instead of ROI Pooling? **DIFFICULTY: HARD** *Companies: Google* ROI Pooling's coarse quantization misaligns features slightly from their true spatial location, which is tolerable for box classification but noticeably degrades pixel-precise mask prediction. ROI Align uses bilinear interpolation to avoid that quantization, producing better-aligned features for the mask head.

IQ30. Why is mean IoU preferred over pixel accuracy as a segmentation summary metric? **DIFFICULTY: MEDIUM** *Companies: General* Pixel accuracy is dominated by whichever class covers the most pixels (often background), so a model that ignores a small foreground class entirely can still score misleadingly high. Mean IoU averages per-class overlap, surfacing poor performance on minority classes that pixel accuracy would hide.

OCR

IQ31. Why does preprocessing usually matter more than OCR engine choice for real-world document images? **DIFFICULTY: MEDIUM** *Companies: General* OCR engines are trained on relatively clean, upright text. Deskewing, denoising, and binarizing a noisy or tilted real-world photo typically improves recognition accuracy far more than switching engines or tuning engine-specific configuration.

IQ32. What does Tesseract's Page Segmentation Mode (PSM) control, and why does the wrong setting hurt accuracy? **DIFFICULTY: MEDIUM** *Companies: General* PSM tells Tesseract what layout assumption to make about the image (a full page, a single block, a single line, sparse scattered text, etc.). A mismatched PSM causes Tesseract to segment the image incorrectly before recognition even runs, producing garbled or incomplete output regardless of image quality.

IQ33. Why can OCR fail silently when the wrong language pack is used? **DIFFICULTY: MEDIUM** *Companies: General* Tesseract doesn't detect a language mismatch and error out — it applies its character/language model as configured and produces its best (often wrong) guess, which can look plausible enough to go unnoticed without a ground-truth check.

OpenCV

IQ34. Why does OpenCV load images in BGR instead of RGB, and what goes wrong if you forget to convert? **DIFFICULTY: EASY** *Companies: General* It's a historical convention from OpenCV's early design. Feeding a BGR array into any RGB-expecting library (matplotlib, PyTorch, most pretrained models) without conversion produces images with visibly swapped red/blue channels — colors look subtly, confusingly wrong rather than causing an outright error.

IQ35. Why must kernel sizes for blur/erode/dilate be odd numbers? **DIFFICULTY: EASY** *Companies: General* An odd kernel size has a well-defined single center pixel that the operation is centered on; an even kernel size has no single center, which OpenCV doesn't support for these operations and will raise an assertion error for.

IQ36. What's the difference between MORPH_OPEN and MORPH_CLOSE, and when would you use each? **DIFFICULTY: MEDIUM** *Companies: General* Opening (erode then dilate) removes small bright noise specks while preserving the overall shape. Closing (dilate then erode) fills small dark holes/gaps within a shape. Use opening to clean up salt-noise; use closing to connect small gaps in an otherwise continuous region.

Deployment & GPU Optimization

IQ37. Why should you benchmark full end-to-end latency instead of just the model's forward-pass time? **DIFFICULTY: MEDIUM** *Companies: Amazon, Meta* Preprocessing (resize, letterbox) and post-processing (NMS, especially with many raw candidate boxes) can be a meaningful fraction of total latency. Reporting only the forward pass misrepresents the actual latency a user or downstream system experiences.

IQ38. What's the practical benefit of mixed-precision (FP16) training, and what's the risk? DIFFICULTY: MEDIUM *Companies: General* It roughly halves memory usage and can give a 1.5-3x speedup on GPUs with tensor cores, with usually-negligible accuracy impact. The main risk is gradient underflow in FP16, which is why `GradScaler` (loss scaling) is used alongside `autocast`.

IQ39. When would you choose gradient accumulation over simply reducing batch size? DIFFICULTY: MEDIUM *Companies: General* When the effective batch size needed for stable training/good convergence doesn't fit in available GPU memory, gradient accumulation simulates a larger batch by summing gradients over several smaller forward/backward passes before stepping the optimizer, preserving the larger effective batch size's training dynamics without needing more memory at once.

IQ40. How would you decide whether a slow training run is data-bound or compute-bound? DIFFICULTY: MEDIUM *Companies: Google, Meta* Monitor GPU utilization during training (e.g. `nvidia-smi`); consistently low utilization (well under ~80-90%) points to the DataLoader/CPU pipeline as the bottleneck, while consistently high utilization with still-slow throughput points to the model/compute itself.

IQ41. Why does quantization (e.g. INT8) speed up inference, and what's the tradeoff? DIFFICULTY: MEDIUM *Companies: General* Lower-precision integer arithmetic is cheaper and faster on supporting hardware, and the model becomes roughly 4x smaller (float32 to int8). The tradeoff is some accuracy loss, which calibration on representative data typically keeps small but should always be validated, not assumed.

IQ42. Why validate an exported/quantized model's output against the original before deploying it? DIFFICULTY: MEDIUM *Companies: General* Export and quantization can introduce small numerical differences that occasionally flip a prediction near a decision boundary. Validating parity on a fixed sample set before deployment catches this cheaply, rather than discovering it via a user-facing incident.

Debugging

IQ43. A classifier predicts the same class for every input. What do you check first? DIFFICULTY: MEDIUM *Companies: General* Label distribution actually reaching the model (severe imbalance or a labeling bug), whether the classifier head was correctly replaced and left trainable, and the learning rate (too high can collapse to a trivial solution).

IQ44. Training loss is NaN starting at some point in training. What's your debugging process? DIFFICULTY: MEDIUM *Companies: Amazon, Meta* Check input data for NaN/corruption first, then check for a missing epsilon term in a custom loss (e.g. Dice loss dividing by zero), then check the learning rate for divergence, then check mixed-precision gradient scaling if in use.

IQ45. You hit CUDA out-of-memory partway through training, not at the start. What's the likely cause? DIFFICULTY: HARD *Companies: Google, Meta* Most commonly, the computation graph is being unintentionally retained across iterations — e.g. appending `loss` (a tensor still attached to the graph) instead of `loss.item()` to a running list, or running validation without `torch.no_grad()`.

IQ46. A model trains and validates well but performs poorly in production. What's your first hypothesis? DIFFICULTY: HARD *Companies: Amazon, Google, Meta* Train/serve preprocessing skew is the single most common cause — the production inference pipeline doesn't exactly match the training-time preprocessing (different resize method, missed normalization, different color-channel order). Domain shift between training data and real production inputs is the second most likely cause.

IQ47. Bounding boxes drawn on an output image are all shifted or oddly scaled. What do you check? DIFFICULTY: MEDIUM *Companies: General* Whether resize/letterbox scale and padding offsets were correctly tracked and inverted when mapping predictions back to original image coordinates, and whether normalized vs. absolute-pixel coordinate conventions were mixed up somewhere in the pipeline.

IQ48. A segmentation model has a fine loss curve but near-zero IoU. What's a likely cause? DIFFICULTY: HARD

Companies: General Image/mask misalignment — often from applying augmentation independently to the image and the mask instead of jointly, so the loss is being computed against a technically-valid-shaped but spatially wrong mask.

Data & Leakage

IQ49. Why does a random train/test split fail on a dataset with multiple images per patient? **DIFFICULTY: HARD** *Companies: medical imaging companies, Amazon* Images from the same patient can share patient-specific, non-diagnostic artifacts (equipment quirks, positioning) that correlate with the label for that patient. A random split can place some of a patient's images in train and others in test, letting the model partly "memorize" the patient rather than learn generalizable features — inflating test metrics in a way that won't hold on genuinely new patients.

IQ50. Why must preprocessing statistics (e.g. normalization mean/std) be computed from the training split only? **DIFFICULTY: MEDIUM** *Companies: General* Computing them from the full dataset lets validation/test data subtly influence how training data gets transformed — a form of leakage, since information from the evaluation set has bled into the training pipeline.

IQ51. Why should augmentation be applied after, not before, the train/val split? **DIFFICULTY: MEDIUM** *Companies: General* Augmenting before splitting can let augmented copies of what should be a validation image end up in the training set (or vice versa), giving the model indirect exposure to validation examples and inflating apparent performance.

System Design / Architecture Selection

IQ52. Design an object detection system for a phone app that needs real-time performance. What do you choose and why? **DIFFICULTY: HARD** *Companies: Google, Meta, startups* Establish latency budget and target hardware first. For real-time, on-device inference: a small YOLO variant (nano/small), exported to TFLite/Core ML with quantization, at a resolution tuned to balance small-object accuracy against the frame-rate budget.

IQ53. Design an offline pipeline to audit a large archive of satellite images for a specific object class, with no real-time constraint. What changes vs. IQ52? **DIFFICULTY: HARD** *Companies: General* With latency no longer a constraint, a two-stage detector (Faster R-CNN) or RetinaNet (if the target objects are small/sparse against a large background) becomes preferable for its typically higher accuracy, since the throughput cost is fully absorbable in a batch/offline setting.

IQ54. A stakeholder asks for "the most accurate model, and it also needs to run in real time." How do you handle the conflict? **DIFFICULTY: HARD** *Companies: Amazon, Meta* Surface the tradeoff explicitly rather than silently picking one side — options include a smaller architecture with a strong training recipe, a two-tier pipeline (fast coarse filter, slower accurate model only on flagged regions/frames), or renegotiating which constraint has genuine flexibility.

IQ55. Why might a two-stage detect-then-classify pipeline outperform a single end-to-end fine-grained detector for a retail product-recognition system? **DIFFICULTY: HARD** *Companies: Amazon, retail tech companies* Product catalogs change far more often than it's practical to retrain a large detector. Keeping a stable "detect a product-shaped region" detector and retraining only a lightweight classifier when new products launch is operationally far cheaper than retraining an end-to-end fine-grained detector on every catalog change.

IQ56. How would you approach a CV project where the customer's real-world images look meaningfully different from your available training data? **DIFFICULTY: HARD** *Companies: General* Treat it explicitly as a domain-shift risk: audit a sample of real target-domain images against the training distribution before committing to a training plan, prioritize collecting or augmenting toward that real distribution, and validate on genuinely representative held-out data rather than trusting metrics from a mismatched validation set.

IQ57. What's your process for choosing between training from scratch, partial fine-tuning, and full fine-tuning for a new classification task? **DIFFICULTY: MEDIUM** *Companies: General* Primarily driven by dataset size and do-

main similarity to the pretraining data: small + similar domain favors a frozen backbone; small + different domain favors partial fine-tuning; large dataset (of either domain) affords full fine-tuning, where pretrained weights mainly help initialization and convergence speed rather than being strictly necessary.

IQ58. How would you evaluate whether a defect-detection model is ready for production on a factory line? **DIFFICULTY: HARD** *Companies: manufacturing tech companies* Prioritize recall for the defect classes (missing a real defect is costlier than a false alarm), validate on genuinely held-out production-representative data (checking for leakage across the same physical unit/batch appearing in train and test), and pair automated flagging with a human-review step rather than fully automating the reject decision, at least initially.

IQ59. A model's offline mAP looks excellent, but a stakeholder reports it "misses things" in practice. What do you investigate? **DIFFICULTY: HARD** *Companies: General* Whether the offline evaluation set is representative of real deployment conditions (lighting, camera angle, object scale/distance) — a strong offline mAP on a curated or leakage-affected validation set doesn't guarantee real-world recall; also check whether the deployed confidence threshold matches the one used during offline evaluation.

IQ60. When would you recommend NOT using a deep learning model at all for a computer vision task? **DIFFICULTY: HARD** *Companies: General* When a classical OpenCV-based approach (thresholding, contour detection, template matching) reliably solves a well-constrained problem — e.g. detecting a fixed-shape object under controlled, consistent lighting — a deep model adds training data requirements, compute cost, and maintenance burden without a commensurate accuracy benefit. Reach for deep learning when the visual variability genuinely exceeds what hand-engineered rules can robustly handle.

★ Quick Revision Sheet

Interview Prep, in one screen:

- Conceptual questions test whether you understand *why*, not just *what* — always be ready to explain the mechanism, not just name it.
- System-design questions test constraint-gathering discipline: latency, deployment target, dataset, accuracy priority, before naming an architecture.
- Debugging questions reward a systematic, symptom-first process over guessing.
- Data-leakage questions are disproportionately common in senior interviews — know the grouped-split pattern cold.

Chapter 6.38 — Computer Vision Cheat Sheets

Rapid-fire revision sheets for every major topic in this part. Each is meant as a pre-interview or pre-project refresher, not a first introduction — see the referenced chapter for full explanations.

★ OpenCV Cheat Sheet

- `cv2.imread` loads BGR; always `cv2.cvtColor(img, cv2.COLOR_BGR2RGB)` before anything RGB-expecting.
- `cv2.imread` returns `None` on failure — always check.
- Slicing crops are `[y1:y2, x1:x2]`; drawing functions take `(x, y)` points.
- Kernel sizes for blur/erode/dilate must be odd.
- Blur before Canny edge detection; `high` threshold ≈ 2-3x `low`.
- `MORPH_OPEN` removes small noise; `MORPH_CLOSE` fills small holes.
- `cv2.imshow` needs a real display — use `imwrite` on headless machines.

★ Image Preprocessing Cheat Sheet

- Match normalization to the pretrained backbone: ImageNet mean/std for ImageNet-pretrained models.
- Detection/segmentation: preserve aspect ratio (letterbox), track scale/pad offsets to invert at inference.
- Segmentation masks: resize with nearest-neighbor only, never bilinear.
- OCR: prioritize sharpness — deskew and binarize rather than aggressively resizing/blurring.
- Preprocessing runs everywhere (train/val/test/production) and must be deterministic and identical across all of them.

★ Augmentation Cheat Sheet

- Augmentation is training-only and randomized; never apply to validation/test.
- Flip/rotate/crop/color-jitter are the default safe set; check flip validity for text and asymmetric classes.
- MixUp blends two images and labels linearly; CutMix pastes a patch and mixes labels proportionally.
- Apply identical joint transforms to image+mask for segmentation (e.g. via `albumentations`).

★ CNN Layers Cheat Sheet

- Output size: `floor((H + 2*pad - dilation*(k-1) - 1) / stride) + 1`.
- Conv2d params: `(kH * kW * C_in + 1) * C_out`.
- Stacked small kernels (3x3) beat one large kernel: same receptive field, fewer parameters, more nonlinearity.
- Global average pooling replaces flattening for the final classifier — fewer params, resolution-independent.
- Always `model.eval()` before validation/inference (BatchNorm + Dropout behave differently in eval mode).

★ CNN Architectures Cheat Sheet

- ResNet: residual connections (`F(x)+x`), default general-purpose baseline (ResNet50).
- MobileNet: depthwise-separable convolutions, built for mobile/edge (MobileNetV3).
- EfficientNet: compound scaling of depth/width/resolution together.
- DenseNet: concatenated (not added) skip connections, strong feature reuse, higher training memory.
- ConvNeXt: modernized pure-CNN design, competitive with vision transformers.
- Each family names its classifier head differently (`.fc`, `.classifier[-1]`, etc.) — `print(model)` before replacing it.

★ Transfer Learning Cheat Sheet

- Flow: freeze backbone -> replace head -> train head -> selectively unfreeze -> fine-tune (small LR).
- Freeze depth depends on dataset size and domain similarity to ImageNet.
- Discriminative LRs: smaller for early/generic layers, larger for late/task-specific layers and the new head.
- Train the new head alone first before unfreezing anything.
- Fine-tuning LR should be 10-100x smaller than a from-scratch training LR.

★ Classification Metrics Cheat Sheet

- Accuracy is misleading under class imbalance — always pair with precision/recall/F1.
- Medical/safety domains: prioritize recall. Fraud/high-false-positive-cost domains: prioritize precision.
- PR-AUC beats ROC-AUC as a summary metric on imbalanced datasets.
- Always inspect the confusion matrix, not just the scalar metric.

★ IoU & NMS Cheat Sheet

- IoU = intersection area / union area.
- NMS: sort by confidence, keep highest, discard overlapping boxes above the IoU threshold, repeat.
- Confidence threshold and IoU threshold are independent knobs, opposite-direction tradeoffs.
- Run NMS per class (class-aware) to avoid cross-class suppression errors.
- Use `torchvision.ops.nms/batched_nms` in production, not a from-scratch loop.

★ Object Detection Cheat Sheet

- Box formats: `xyxy`, `xywh` (COCO), `cxcywh` normalized (YOLO) — always know which one an API expects.
- Confidence is often objectness $\times$ class probability.
- Anchor-based needs tuned anchors; anchor-free predicts coordinates directly.
- Pipeline: backbone -> detection head -> confidence threshold -> NMS -> final boxes.

★ YOLO Cheat Sheet

- One-stage, multi-scale detection heads (feature pyramid) for small and large objects together.
- Always start from a pretrained checkpoint, never random weights.
- Label format: `class_id cx cy w h`, normalized `[0,1]`, one `.txt` per image.
- `conf` and `iou` are tunable at inference time without retraining.
- Use `stream=True` for video/large-batch inference to avoid loading all results into memory.

★ SSD Cheat Sheet

- One-stage, predicts directly from multiple backbone feature-map resolutions.
- Anchor-based, multiple aspect ratios/scales per feature map location.
- Weaker on small objects than modern YOLO; mostly relevant today for legacy/embedded toolchains.

★ Faster R-CNN Cheat Sheet

- Two-stage: Region Proposal Network generates candidates, then a head classifies + refines each.
- ROI Align beats ROI Pooling for precise localization (bilinear, no quantization).
- Higher typical accuracy, especially on small/crowded objects; slower — proposal count scales inference cost.
- torchvision: `num_classes` = actual classes + 1 (background).

★ RetinaNet Cheat Sheet

- Focal loss: $-\alpha(1-p_t)^{\gamma}\log(p_t)$ — down-weights easy examples, fixes class imbalance in dense one-stage detection.
- Built on a Feature Pyramid Network for multi-scale coverage.
- Best fit: severe foreground/background imbalance + small/dense objects.

★ Detector Selection Cheat Sheet

- Real-time / edge: YOLO.
- Offline, max accuracy: Faster R-CNN.
- Severe imbalance, small/dense objects: RetinaNet.
- Legacy/embedded constraint only: SSD.
- Always establish latency budget and deployment target before naming a model.

★ Segmentation Cheat Sheet

- Output: per-pixel class-ID mask, same spatial size as input. Save masks as PNG, never JPEG.
- Semantic: one class mask. Instance: per-object mask + box + class (Mask R-CNN).
- IoU and Dice are the standard overlap metrics; pixel accuracy alone is misleading under imbalance.
- U-Net: encoder-decoder with concatenated skip connections. DeepLab: dilated convs + multi-scale pooling.

★ U-Net Cheat Sheet

- Skip connections carry spatial detail from encoder to decoder — essential for sharp boundaries.
- Transposed convs can cause checkerboard artifacts; bilinear upsample + conv is a common fix.
- Combine BCE + Dice loss for more stable training under class imbalance.
- Apply identical random transforms to image and mask together — never independently.

★ Mask R-CNN Cheat Sheet

- Faster R-CNN + a parallel mask head on ROI-Align'd features.
- Raw mask output is a `[0,1]` probability map — threshold (typically 0.5) before using as binary.
- Fine-tuning needs replacing both the box predictor and the mask predictor.

★ OCR / Tesseract Cheat Sheet

- Pipeline: preprocess (deskew/denoise/binarize) -> detect regions -> recognize -> post-process -> structured output.
- Preprocessing is usually the highest-leverage accuracy lever, not the OCR engine.
- Always install/specify the correct language pack — wrong language fails silently, not loudly.
- PSM controls layout assumptions; match it to the actual document structure.
- Debug in order: preprocessing -> PSM -> language pack -> only then consider a different engine.

★ GPU Optimization Cheat Sheet

- Profile the DataLoader in isolation before optimizing the model — data loading is a common actual bottleneck.
- Mixed precision (`autocast` + `GradScaler`): near-free 1.5-3x speedup on modern GPUs.
- Gradient accumulation simulates a larger batch size when memory is the constraint.
- `nvidia-smi` utilization tells you whether you're data-bound or compute-bound.
- `num_workers` $\approx$ physical core count minus a couple — profile, don't guess.

★ Deployment Cheat Sheet

- Pipeline: train -> validate -> export -> optimize -> inference API -> Docker -> deploy.
- Validate input at the API boundary (content type, decodability) — never let a bad upload crash the process.
- Decide export format (ONNX/TensorRT/TFLite/Core ML) and quantization strategy early.
- Always validate exported/quantized model output against the original before calling deployment done.
- Benchmark full end-to-end latency (preprocessing + inference + NMS/post-processing), not just the forward pass.

One-Page CV Roadmap

```
OpenCV Basics
      |
      v
Image Processing / Preprocessing / Augmentation
      |
      v
```

```
CNN Fundamentals
      |
      v
Transfer Learning
      |
      v
Classification
      |
      v
Object Detection (IoU, NMS)
      |
      v
YOLO
      |
      v
RetinaNet / Faster R-CNN / SSD
      |
      v
Segmentation (U-Net)
      |
      v
Mask R-CNN
      |
      v
OCR
      |
      v
Video CV
      |
      v
Deployment
      |
      v
Portfolio Project
```

Model-Selection Decision Tree

See Chapter 6.36 for the full decision tree and selection checklist.

Environment Checklist

```
[ ] Python version confirmed and matches project requirements.txt
[ ] Framework (PyTorch/TensorFlow) version confirmed
[ ] GPU detected and CUDA version compatible, if applicable
[ ] Dataset directory structure verified (images/labels present, counts sane)
[ ] Free disk space sufficient for dataset + checkpoints + logs
[ ] requirements.txt pinned to actually-used versions
[ ] environment_check.py run successfully before any training
```

Project Checklist

```
[ ] Data quality audit run (corrupted files, duplicates, class balance, resolution) — Ch. 6.29
[ ] Leakage audit run (grouped/temporal splits where relevant) — Ch. 6.30
[ ] Annotations visualized/sanity-checked before training — Ch. 6.15
[ ] Baseline model trained and evaluated before any architecture tuning
[ ] Correct metric chosen for the domain (recall/precision/F1/mIoU/Dice as appropriate)
[ ] Confusion matrix / per-class breakdown inspected, not just the aggregate metric
[ ] Error analysis performed on the worst-performing examples
[ ] Preprocessing function shared and tested identically across train/val/test/production
[ ] Exported/quantized model validated against the original before deployment
[ ] Input validation and error handling present at the deployment API boundary
```

Interview Question Bank

See Chapter 6.37 for 60 fully-answered interview questions spanning CNN fundamentals, architectures, transfer learning, detection, segmentation, OCR, OpenCV, deployment, GPU optimization, debugging, data leakage, and system design — organized by topic for targeted review.

★ Part 6 Closing Summary

- The CV pipeline (load -> preprocess -> augment -> model -> post-process -> evaluate -> deploy) is the mental model underlying every project in this part.
- Transfer learning is the default starting point for nearly every practical task — from-scratch training is the exception, not the rule.
- Detector and segmentation architecture choice is a constraint-driven decision (latency, deployment target, object-size distribution, class imbalance), not a matter of picking the ”best” model in the abstract.
- Data quality and leakage audits are cheap insurance against the most damaging and hardest-to-detect class of CV bugs.
- Train/serve preprocessing skew is the most common cause of ”works in training, fails in production.”